

1 Furijeov red (Fourier Series)

Koristi se za periodične funkcije $G(T)$. Ideja: svaka periodična funkcija se može izraziti kao zbir sinusa i kosinusa različitih frekvencija (harmonika).

Opšta forma Furijeove serije:

$$G(T) = a_0 + \sum_{n=1}^{\infty} [a_n \cos(n\omega_0 t) + b_n \sin(n\omega_0 t)]$$

Gde je šta:

- $G(T)$ – originalna periodična funkcija vremenskog signala
- a_0 – srednja (DC) vrednost funkcije
- a_n – koeficijent kosinusa, pokazuje amplitudu kosinusne komponente n -te harmonike
- b_n – koeficijent sinusa, pokazuje amplitudu sinusne komponente n -te harmonike
- $\omega_0 = \frac{2\pi}{T_0}$ – osnovna (fundamentalna) ugaona frekvencija, gde je T_0 period signala
- n – red harmonike (prva, druga, treća...)

2 Furijeova transformacija (Fourier Transform)

Koristi se za neperiodične ili kontinuirane signale, da ih prevedemo iz vremenskog domena u frekvencijski domen:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt$$

- $x(t)$ – signal u vremenskom domenu
- $X(f)$ – spektar signala u frekvencijskom domenu
- f – frekvencija
- $e^{-j2\pi ft}$ – kompleksna eksponencijalna funkcija (kombinuje sinus i kosinus)

Inverzna Fourierova transformacija:

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df$$

3 Srednjekvadratna amplituda (RMS)

Mera efektivne amplitude signala.

a) Diskretni signal $x[n]$ $x_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=1}^N x[n]^2}$

- N – broj uzoraka
- $x[n]$ – vrednost signala u n -tom uzorku

b) Kontinualni signal $x(t)$ $x_{\text{RMS}} = \sqrt{\frac{1}{T} \int_0^T [x(t)]^2 dt}$

- T – period signala
- $x(t)$ – vrednost signala u trenutku t

Značenje:

- Kvadrat RMS-a (x_{RMS}^2) je proporcionalan **energiji** ili **snazi signala**.
- RMS uzima u obzir oscilacije i varijacije, daje efektivnu "prosečnu snagu".

4 Koraci Furijeove analize

1. Uzimanje signala u vremenskom domenu

- Diskretan $x[n]$ ili kontinualan $x(t)$

2. Razlaganje signala na sinusne i kosinusne komponente

- Fourierova serija za periodične signale
- Fourierova transformacija za neperiodične

3. Izračunavanje Fourierovih koeficijenata

- a_n, b_n za seriju
- $X(f)$ za transformaciju

4. Dobijanje spektra signala u frekvencijskom domenu

- Amplituda vs frekvencija
- Faza vs frekvencija

5. (Po potrebi) Inverzna transformacija

- Rekonstrukcija signala u vremenskom domenu iz njegovih frekvencija

5 Primena srednjekvadratne amplitude (RMS)

Primena

Objašnjenje

Električni signali (AC)

Efektivna vrednost napona ili struje (npr. 220 V RMS)

Zvuk i vibracije

Merenje energije ili snage signala, intenziteta vibracija

Statistika

Standardna devijacija oscilatornih ili promenljivih podataka

Opšta primena

Daje "prosečnu veličinu" signala uzimajući u obzir oscilacije i varijacije

$$g(t) = \frac{1}{2}c + \sum_{n=1}^{\infty} a_n \cdot \sin(2\pi n f t) + \sum_{n=1}^{\infty} b_n \cdot \cos(2\pi n f t) - \text{Фурјеова серија за периодичну функцију } g(t)$$

c, a_n, b_n - коефицијенти који зависе од функције $g(t)$ и рачунају се по основном интегралу

f - основна фреквенција сигнала

$\cos(2\pi n f t)$ и $\sin(2\pi n f t)$ - хармонички сигнал где n означава редни број хармоника

Imate sinusoidalni električni signal koji je prikazan kao funkcija vremena: $V(t) = 10 \cdot \sin(2\pi \cdot 50 \cdot t) + 5 \cdot \sin(2\pi \cdot 100 \cdot t)$ Ovaj signal predstavlja kombinaciju dve sinusne komponente sa frekvencijama od 50 Hz i 100 Hz. Izračunajte RMS vrednost ovog signala.

✓ **Dati signal:** $V(t) = 10 \sin(2\pi \cdot 50t) + 5 \sin(2\pi \cdot 100t)$

Dakle, imamo dve **sinusne komponente**:

- Prva: amplituda $A_1 = 10$, frekvencija $f_1 = 50 \text{ Hz}$
- Druga: amplituda $A_2 = 5$, frekvencija $f_2 = 100 \text{ Hz}$

1 RMS jedne sinusne komponente

Za čistu sinusoidu $V(t) = A \sin(\omega t)$: $V_{rms} = \frac{A}{\sqrt{2}}$

2 RMS svake komponente posebno

$$V_{rms1} = \frac{10}{\sqrt{2}} = 7.071 \quad V_{rms2} = \frac{5}{\sqrt{2}} = 3.536$$

3 Kombinacija RMS vrednosti (ne sme se sabrati direktno!)

Pošto su frekvencije različite (50 Hz i 100 Hz), signali su **međusobno ortogonalni** (ne utiču jedan na drugi), pa se RMS vrednosti kombinuju **kvadratnim sabiranjem**:

$$V_{rms} = \sqrt{V_{rms1}^2 + V_{rms2}^2}$$

$$V_{rms} = \sqrt{(7.071)^2 + (3.536)^2}$$

$$V_{rms} = \sqrt{50 + 12.5} = \sqrt{62.5} = 7.906$$

4 Računamo:

◆ **Rezultat:**

$V_{rms} = 7.91 \text{ V}$

Imate niz diskretnih vrednosti napona V_i koji predstavljaju uzorke analognog signala. Niz uzoraka je dat kao: $V = [2, 4, 6, 8, 10]$
Izračunajte RMS vrednost ovog diskretnog signala.

RMS (Root Mean Square) za **diskretni signal** se računa po formuli:

$$V_{rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N V_i^2}$$

gde je • V_i — svaka pojedinačna vrednost napona, • N — ukupan broj uzoraka.

■ Imamo niz: $V = [2, 4, 6, 8, 10]$ Dakle, $N = 5$.

Korak 1. Kvadriramo svaku vrednost: $2^2 = 4$, $4^2 = 16$, $6^2 = 36$, $8^2 = 64$, $10^2 = 100$

Korak 2. Saberemo kvadrate: $4 + 16 + 36 + 64 + 100 = 220$

Korak 3. Podelimo sa brojem uzoraka (5): $\frac{220}{5} = 44$

Korak 4. Izvadimo kvadratni koren: $\sqrt{44} \approx 6.633$

KASNJENJE

PAKETA !!!!!

Kada paket putuje od računara A do računara B kroz mrežu, **ukupno kašnjenje (d)** koje paket doživljava se

sastoji iz više komponenti: $d = d_{\text{obrade}} + d_{\text{baferovanja}} + d_{\text{prenosa}} + d_{\text{propagacije}}$

⚙️ 1 Kašnjenje obrade — d_{obrade}

- ◆ **Šta predstavlja:** Vreme koje ruter ili čvor potroši da **obradi paket** pre nego što odluči gde će ga poslati dalje.
- ◆ **Šta se dešava u tom vremenu:**
 - Provera greške u zaglavlju (checksum, CRC)
 - Čitanje zaglavlja paketa (da vidi odredište)
 - Izbor izlaznog linka (koji port vodi ka odredištu)
 - Eventualno rutiranje, filtriranje ili obrada tabela
- ◆ **Tipične vrednosti:**
 - Vrlo mala — **manja od 1 milisekunde (ms)** po paketu
 - Ali ako je ruter opterećen ili ima složene operacije (npr. firewall, NAT), može biti veća
- ◆ **Zavisí od:**
 - Performansi rutera
 - Broja ruta koje treba proveriti
 - Vrste operacija (npr. samo prosleđivanje vs. sigurnosna analiza)

2 Kašnjenje čekanja u baferu — $d_{\text{baferovanja}}$

◆ Šta predstavlja:

Vreme koje paket čeka u redu (baferu) pre nego što dođe na red da se prenese preko linka.

◆ Zašto nastaje:

- Ako više paketa stiže nego što link može trenutno da pošalje, oni se **nakupljaju u baferu** i čekaju svoj red.
- Dakle, kašnjenje se povećava kada je mreža **zagušena**.

◆ Zavisi od: $d_{\text{baferovanja}}$ zavisi od odnosa brzine dolaznih i odlaznih paketa

- Ako paketi stižu **brže nego što odlaze**, bafer se puni i kašnjenje raste.
- Ako paketi stižu **sporije nego što odlaze**, kašnjenje je minimalno.

◆ Tipične vrednosti:

- Od **mikrosekundi do sekundi**, u zavisnosti od opterećenja mreže
- Ako se bafer prepuni, dolazi do **gubitka paketa**

3 Predajno (transmisiono) kašnjenje (d_{prenosa})

- Šta opisuje: vreme potrebno da svi bitovi paketa budu poslani na link.
- Formula:

$$d_{\text{prenosa}} = \frac{\text{veličina paketa (bit)}}{\text{propusnost linka (bit/s)}}$$

- Napomena: Veći paket → veće kašnjenje, brži link → manje kašnjenje.
Ne zavisi od udaljenosti, već od brzine linka i veličine paketa.

4 Propagaciono kašnjenje ($d_{\text{propagacije}}$)

- Šta opisuje: vreme koje signal troši da fizički pređe link između dva uređaja.

- Formula:
$$d_{\text{propagacije}} = \frac{\text{udaljenost linka (m)}}{\text{brzina širenja signala (m/s)}}$$

- Napomena: Ne zavisi od veličine paketa, zavisi od udaljenosti i medijuma.

💡 Dakle, ukratko:

Vrsta kašnjenja	Simbol	Opis	Tipično trajanje
Obrada	d_{obrade}	vreme za proveru greške i izbor izlaznog linka	< 1 ms
Baferovanje	$d_{\text{baferovanja}}$	čekanje paketa da dođe na red za slanje	zavisi od zagušenja (od μs do s)
Prenos	d_{prenosa}	slanje svih bitova na link	zavisi od veličine paketa i brzine linka
Propagacija	$d_{\text{propagacije}}$	fizičko putovanje signala	zavisi od udaljenosti i medijuma

VAZNE

FORMULE

1 Predajno (transmisiono) kašnjenje (T_{pred})

- Šta opisuje: vreme potrebno da svi bitovi paketa X budu poslani na link.
- Oznake: X — veličina paketa (bit), R — brzina linka (bit/s)

$$T_{\text{pred}} = \frac{X}{R}$$

2 Propagaciono kašnjenje (T_{prop})

- Šta opisuje: vreme koje signal troši da pređe link.
- Oznake: L — dužina linka, V — brzina širenja signala

$$T_{\text{prop}} = \frac{L}{V}$$

3 Dužina bita (W_b)

- Šta opisuje: fizička dužina koju jedan bit zauzima na linku
- Oznake: V — brzina širenja signala, R — brzina prenosa (bit/s)

$$W_b = \frac{V}{R}$$

ZADACI

Neka je data veza tipa tačka-ka-tački (point-to-point) dužine 50km. Pri kom će propusnom opsegu (tj. pri kojoj bitskoj brzini) propagacino kašnjenje biti jednako predajno kašnjenju za slučaj da je paket obima 200B? Kolika treba da je bitska brzina prenosa, ako je paket obima 1024B, pri istim uslovima definisanim u prethodnom pristupu? Napomena: Signal se prenosi brzinom od $2 \cdot 10^8$ m/s.

◆ Podaci iz zadatka

- Dužina linka: $L = 50 \text{ km} = 50\,000 \text{ m}$
- Brzina signala: $V = 2 \times 10^8 \text{ m/s}$
- Paket:
 1. prvi slučaj: $X_1 = 200 \text{ B} = 200 \times 8 = 1600 \text{ bitova}$
 2. drugi slučaj: $X_2 = 1024 \text{ B} = 1024 \times 8 = 8192 \text{ bitova}$

Traži se bitska brzina R kod koje je: $T_{\text{prop}} = T_{\text{pred}}$

◆ 1. Izrazimo obe formule $T_{\text{prop}} = \frac{L}{V}$ $T_{\text{pred}} = \frac{X}{R}$

Po uslovu: $\frac{L}{V} = \frac{X}{R}$ $R = \frac{V \cdot X}{L}$

$$R_1 = 6.4 \text{ Mb/s}$$

$$R_2 = 32.77 \text{ Mb/s}$$

Pretpostavimo da su dva host-a A i B međusobno razdvojeni direktnim link-om čija je dužina $L = 10\,000\text{ km}$. Brzina prenosa podataka po link-u iznosi $R = 1\text{ Mbps}$, a brzina propagacije signala po linku iznosi $S = 2.5 \times 10^8\text{ m/s}$. 1. Koliko je vremena potrebno da se pošalje fajl obima $400\,000\text{ b}$ ako pretpostavimo da se isti predaje kontinualno? 2. Neka se fajl podeli na 10 paketa pri čemu svaki paket sadrži $40\,000\text{ b}$. Pretpostavimo da se svaki paket potvrđuje od strane prijemnika i da je transmisiono vreme paketa potvrde zanemarljivo. Konačno, usvojimo da predajnik ne može poslati novi paket sve dok se prijem prethodnog ne potvrdi. Koliko vremena je potrebno za slanje celog fajla? 3. Uporedi rezultate iz stavki 1) i 2).

◆ Podaci iz zadatka

Oznaka	Značenje	Vrednost
L	dužina linka	$10\,000\text{ km} = 10^7\text{ m}$
R	bitska brzina	$1\text{ Mbps} = 10^6\text{ bit/s}$
S	brzina propagacije	$2.5 \times 10^8\text{ m/s}$
X	obim fajla	$400\,000\text{ bitova}$

◆ 1) Kontinualno slanje fajla

Ovde nema čekanja potvrda

Prenosno (transmisiono) kašnjenje:

$$T_{pred} = \frac{X}{R} = \frac{400\,000}{1\,000\,000} = 0.4\text{ s}$$

Propagaciono kašnjenje:

$$T_{prop} = \frac{L}{S} = \frac{10^7}{2.5 \times 10^8} = 0.04\text{ s}$$

Pošto se fajl šalje kontinuirano, ukupno vreme do potpunog prijema na strani B:

$$T_{\text{ukupno}} = T_{\text{pred}} + T_{\text{prop}} = 0.4 + 0.04 = \boxed{0.44 \text{ s}}$$

◆ 2) Slanje po paketima uz potvrdu (stop-and-wait ARQ)

Podela fajla:

- 10 paketa, svaki po 40 000 bitova
- svaki paket šalje se → propagira → čeka potvrdu
- sledeći se ne može poslati dok prethodni nije potvrđen

Dakle, za svaki paket imamo ciklus:

$$T_{\text{paket}} = T_{\text{pred},p} + 2 \times T_{\text{prop}}$$

(gde 2× znači: propagacija do prijemnika + propagacija potvrde nazad)

Prenosno vreme jednog paketa:

$$T_{\text{pred},p} = \frac{40\,000}{1\,000\,000} = 0.04 \text{ s}$$

Propagaciono vreme:

$$T_{\text{prop}} = 0.04 \text{ s}$$

Vreme po paketu:

$$T_{\text{paket}} = 0.04 + 2 \times 0.04 = 0.12 \text{ s}$$

Ukupno vreme za 10 paketa:

$$T_{\text{ukupno}} = 10 \times 0.12 = \boxed{1.2 \text{ s}}$$

◆ 3) Uporedi rezultate

Scenarijo	Ukupno vreme prenosa
Kontinualni prenos	0.44 s
Paketno (sa potvrdom)	1.2 s



Dakle:

$$T_{\text{paketno}} \approx 2.7 \times T_{\text{kontinualno}}$$

◆ Zaključak

- Kod kontinualnog slanja veza se koristi maksimalno efikasno (nema čekanja).
- Kod paketnog sa potvrdom (stop-and-wait), mnogo vremena se gubi na čekanje potvrda jer je **propagaciono kašnjenje veliko u odnosu na vreme prenosa paketa**.
- Zato se u praksi koriste **pipeline protokoli (slanje više paketa pre potvrde)** da bi se poboljšala iskorišćenost linka.

$$L = 10000000 \text{ m} \quad R = 1000000 \text{ b/s} \quad S = 2.5 * 10^8 \frac{\text{m}}{\text{s}} \quad \text{Део зајатка под 1: } X = 400000 \text{ b}$$

$$T_{\text{prop}} = \frac{L}{S} = \frac{10000000 \text{ m}}{250000000 \text{ m/s}} = \frac{4}{100} \text{ s} \quad T_{\text{pred}} = \frac{X}{R} = \frac{400000}{1000000} = \frac{40}{100} \text{ s} \quad \text{Укупно време је:}$$

$$T_{\text{fail_укупно}} = T_{\text{prop}} + T_{\text{pred}} = \frac{4}{100} + \frac{40}{100} = \frac{44}{100} \text{ s}$$

$$\text{Део зајатка под 2: Преносно време једног пакета је: } T_{\text{pred.p}} = \frac{40000}{1000000} = \frac{4}{100} \text{ s}$$

$$\text{Циклус једног пакета: } T_{\text{paket}} = T_{\text{pred.p}} + 2 * T_{\text{prop}} = \frac{4}{100} + 2 * \frac{4}{100} = \frac{4}{100} + \frac{8}{100} = \frac{12}{100} \text{ s}$$

$$\text{Циклус за свих 10 пакета: } \frac{12}{100} * \frac{10}{1} = \frac{120}{100} \text{ s. Дакле, } T_{\text{paket_укупно}} = \frac{120}{100} \text{ s.}$$

Само је важно да се напомене да у формули за T_{paket} изабра $2 * T_{\text{prop}}$ јер нам је једно T_{prop} пропација до пријемника, а друго је пропација потврде.

$$\text{Ако их упоредимо, добија се: } \frac{T_{\text{paket_укупно}}}{T_{\text{fail_укупно}}} = \frac{\frac{120}{100}}{\frac{44}{100}} = \frac{120 * 100}{44 * 100} = \frac{120}{44} \approx 2.7$$

Pretpostavimo da su dva host-a A i B međusobno udaljeni za $L = 10\,000\text{ km}$ i povezani link-om čija je brzina prenosa $R = 1\text{ Gbps}$. Neka propagaciono kašnjenje signala po vezi (link-u) iznosi $S = 2.5 \times 10^8\text{ m/s}$. 1. Izračunati proizvod propusnost-kašnjenje, $W = R \times t_{\text{prop}}$. 2. Neka host A predaje host-u B fajl obima $400\,000\text{ b}$. Fajl se predaje kao jedinstvena neprekidna poruka. Koliki je maksimalni broj bitova koji može biti prisutan na link-u u datom trenutku? 3. Koja je dužina bitova u metrima, W_{b} , na link-u?

◆ **Zadato**

Oznaka	Značenje	Vrednost
L	dužina linka	$10\,000\text{ km} = 10^7\text{ m}$
R	bitska brzina	$1\text{ Gbps} = 10^9\text{ bit/s}$
S	brzina propagacije	$2.5 \times 10^8\text{ m/s}$
X	obim fajla	$400\,000\text{ bitova}$

- ◆ 1. Propagaciono kašnjenje t_{prop} $t_{\text{prop}} = \frac{L}{S} = \frac{10^7}{2.5 \times 10^8} = 0.04\text{ s}$
- ◆ 2. Proizvod propusnost-kašnjenje $W = R \times t_{\text{prop}}$ $W = 10^9 \times 0.04$

Ovaj broj pokazuje koliko bitova može istovremeno biti "na putu" između A i B — tj. koliko ih „fizički postoji u linku“ dok signal putuje.

◆ 3. Maksimalan broj bitova na linku

To je upravo vrednost proizvoda $W = R \times t_{\text{prop}}$,
jer to predstavlja broj bitova u „cevi“ u svakom trenutku.

Dakle: Na linku može biti do 40 000 000 bitova istovremeno.

Napomena:

Pošto fajl ima samo 400 000 bitova, celi fajl je mnogo manji od kapaciteta linka, znači link nikada neće biti „pun“.

◆ 4. Dužina jednog bita u metrima W_b

Formula: $W_b = \frac{S}{R} \quad W_b = \frac{2.5 \times 10^8}{10^9} = 0.25 \text{ m}$

◆ Rezime rezultata

Veličina	Formula	Rezultat
Propagaciono kašnjenje	$t_{\text{prop}} = L/S$	0.04 s
Proizvod propusnost–kašnjenje	$W = R \times t_{\text{prop}}$	4×10^7 bitova
Broj bitova na linku	$= W$	40 000 000 bitova
Dužina bita	$W_b = S/R$	0.25 m

$$L = 10000000 \text{ m} \quad R = 1000000000 \text{ b/s} \quad S = 250000000 \frac{\text{m}}{\text{s}} \quad t_{\text{prop}} = \frac{L}{S} = \frac{10000000 \text{ m}}{250000000 \text{ m/s}} = \frac{4}{100} \text{ s}$$

$$W = R \cdot t_{\text{prop}} = \frac{1000000000 \text{ b/s}}{1} \cdot \frac{4}{100} \text{ s} = 4000000 \text{ b} - \text{ово је максимални број бита који могу бити у једном тренутку на линији. Капацитет линије неће бити пуна ни у једном тренутку јер } 4000000 < 40 \text{ mil b.}$$

$$W_b = \frac{S}{R} = \frac{250000000 \text{ m/s}}{1000000000 \text{ b/s}} = \frac{25}{100} \frac{\text{m}}{\text{b}} \quad \text{Поделе се само именници ових кацбења и то је то.}$$

BER (Bit Error Rate) se definiše kao verovatnoća da će bit na određenoj poziciji u povorci bitova biti pogrešan. Neka računarski sistem prihvata povorku bitova sa serijske komunikacione linije, bitskom brzinom od 10^6 bps, a neka je $BER = 10^{-10}$. Odrediti broj pogrešnih prenetih bitova u sekundi.

◆ **Zadato:**

- Bitska brzina: $R = 10^6$ bps
- Verovatnoća greške (BER): $P_e = 10^{-10}$

◆ **Izračunavanje:** $N_e = 10^6 \times 10^{-10} = 10^{-4}$

◆ **Tumačenje:**

◆ **Traži se:** Broj pogrešnih bitova u sekundi N_e

◆ **Formula:** $N_e = R \times P_e$

To znači da bi, u proseku, došlo do jedne greške na svakih:

$$\frac{1}{N_e} = \frac{1}{10^{-4}} = 10^4 = 10\,000 \text{ sekundi}$$

Dakle — jedna greška na svakih 10.000 sekundi, što je otprilike jedna greška na svaka 2 sata i 46 minuta.

$$BER = \frac{\text{pogresni bitovi}}{\text{primljeni bitovi}} = \frac{10^{-10}}{10^6} = \frac{10^{-10}}{1} \cdot \frac{10^6}{1} \quad \text{pogresni bitovi} = 10^{-4} \text{ E sad kad}$$

ako ovo dobili mi trebamo da nađemo recipročnu vrednost ovog, a to je 10000. Znači, na svakih 10000 sekundi pojavi se 1 pogrešan bit. Logično je da je BER verovatnoća pojavljivanja pogrešnog bita. To je jasno iz konteksta. Dakle ovo je količnik pogrešnih bitova i ukupnog broja primljenih bitova.

Razlike između: Mili i mikro. LSK.

◆ **Zadato:**

- Brzina prenosa: $R = 2048 \text{ Mbps} = 2.048 \times 10^9 \text{ bps}$
- Broj pogrešnih bitova u sekundi: $N_e = 2$

◆ **Formula:**
$$\text{BER} = \frac{N_e}{R}$$

◆ **Izračunavanje:**
$$\text{BER} = \frac{2}{2.048 \times 10^9}$$

◆ **Tumačenje:**

To znači da je, u proseku, jedan bit od oko milijardu prenetih bitova pogrešan.

Takav BER je izuzetno dobar i karakterističan za optičke veze ili vrlo stabilne digitalne komunikacione kanale.

Kod prenosa fajlova između dva računara moguće su (najmanje) dve strategije potvrđivanja prijema poruka. 1.Kod prve, fajl se deli na pakete čiji se prijem individualno potvrđuje od strane prijemnika, ali prijem celog fajla se ne potvrđuje. 2.Kod drugog slučaja, paketi se ne potvrđuju individualno, ali se potvrđuje fajl kada se isti u celosti prihvati. Ukazati na prednosti i nedostatke ova dva pristupa za slučaj kada se u toku prenosa javlja veći broj grešaka i za slučaj kada je prenos skoro idealan (ne javljaju se greške u prenosu). Razmotriti konkretan slučaj i predložiti superiornije rešenje kada je fajl koji se prenosi dužine 100 000 bitova, BER = 1×10^{-4} , a paketi dužine 1000 bitova. Bitska brzina prenosa podataka je 10000 bps. Odrediti prosečno ukupno vreme prenosa fajla. Usvojiti da je vreme prenosa povratne poruke veoma kratko (zanemarljivo).

◆ 1 **Kratak pregled dva pristupa**

Pristup	Opis	Prednosti	Nedostaci
(1) Individualni ACK	Svaki paket se potvrđuje zasebno.	- Greška se brzo otkriva i ponavlja se samo taj paket. - Manje gubitka podataka.	- Više signalizacije (mnogo potvrda). - Sporije kod idealnog kanala (jer se stalno čeka ACK).
(2) Grupni ACK (na kraju)	ACK se šalje tek po prijemu celog fajla.	- Brz prenos ako nema grešaka. - Mala signalizacija (samo jedan ACK).	- Ako se javi i <i>jedna</i> greška, ceo fajl mora da se ponovo pošalje.

◆ 2 Dat zadatak

Veličina	Oznaka	Vrednost
Veličina fajla	X_f	100 000 b
Veličina paketa	X_p	1 000 b
Brzina prenosa	R	10 000 bps
BER	P_e	10^{-4}
Povratna poruka (ACK)	—	zanemarljivo vreme

Broj paketa:
$$N = \frac{X_f}{X_p} = \frac{100\,000}{1\,000} = 100$$

◆ 3 Verovatnoća greške po paketu

Greška u paketu postoji ako barem jedan bit bude pogrešan.

Za male P_e , aproksimacija je:

$$P_{\text{paket}} \approx 1 - (1 - P_e)^{X_p} \approx X_p \cdot P_e$$

$$P_{\text{paket}} = 1\,000 \times 10^{-4} = 0.1$$

👉 Dakle, 10 % paketa je pogrešno preneto (u proseku).

◆ 4 Pristup 1 – potvrda svakog paketa (individualni ACK)

Svaki paket koji stigne loše, ponavlja se.

Očekivani broj pokušaja po paketu:

$$E[\text{pokušaja}] = \frac{1}{1 - P_{\text{paket}}} = \frac{1}{0.9} \approx 1.11$$

Ukupan broj paketa koji se prenose (uključujući ponovljene):

$$N_{\text{ef}} = N \times 1.11 = 111$$

Vreme prenosa jednog paketa:

$$t_p = \frac{X_p}{R} = \frac{1\,000}{10\,000} = 0.1\text{ s}$$

Ukupno vreme prenosa fajla:

$$T_1 = N_{\text{ef}} \times t_p = 111 \times 0.1 = 11.1\text{ s}$$

◆ 5 Pristup 2 – potvrda celog fajla (ACK na kraju)

Sada nema pojedinačnih potvrda, ali ako postoji i jedna greška u bilo kom paketu, ceo fajl se mora ponoviti.

Verovatnoća da se ceo fajl prenese ispravno:

$$P_{\text{fajl_OK}} = (1 - P_{\text{paket}})^N = (0.9)^{100} \approx 2.65 \times 10^{-5}$$

👉 Dakle, šansa da prođe sve bez greške je 0.00265 %, praktično nikakva.

Fajl će se morati ogromno mnogo puta ponavljati.

Očekivani broj pokušaja:

$$E[\text{pokušaja}] = \frac{1}{P_{\text{fajl_OK}}} \approx 37\,700$$

Ukupno vreme prenosa:

$$T_2 = E[\text{pokušaja}] \times \frac{X_f}{R}$$
$$T_2 = 37\,700 \times \frac{100\,000}{10\,000} = 37\,700 \times 10 = 377\,000 \text{ s} \approx 104.7 \text{ h}$$

6 Uporedni pregled

Pristup	Očekivano vreme prenosa	Pouzdanost	Pogodno za
Individualni ACK	≈ 11.1 s	Veoma visoka	Kanali sa greškama
ACK na kraju	≈ 104 h	Veoma loša	Samo idealni kanali

✓ Zaključak

■ Kada je $BER = 10^{-4}$ (kanal sa dosta grešaka) →

Individualni ACK je mnogo efikasniji.

Ponavljaju se samo oštećeni paketi, a ne ceo fajl.

■ Kada bi kanal bio idealno pouzdan ($BER \approx 0$) →

Grupni ACK (na kraju) bi bio brži jer nema kašnjenja između potvrda.

Важне напомене: Индивидуални АСК: се шаље пакет по пакет, али се не шаље на крају потврда о послатом фајлу, већ сваки пакет поједино кад се пошаље он се проверава и ако се пронађе грешка, понавља се само тај пакет. Много је мање губитка података. Много је потврда, и спорије је код ижељног канала. Групи АСК: шаље се цео фајл одједном, а АСК се шаље тек по пријему целог фајла. Преко је брз ~~ако~~ нема грешака, али ако се јави и једна грешка, онда цео фајл мора поново да се пошаље.

Дужина фајла: 100000b BER = $\frac{1}{10^4}$ Дужина пакета: 1000b Број пакета: $\frac{100000}{1000} = 100$ $R = 10^4 \text{ b/s}$

Грешка у пакету постоји ако је макар и 1 бит погрешан. За мале P_e апроксимација је:

$P_{\text{paket}} \approx 1 - (1 - P_e)^{X_p} \approx X_p \cdot P_e \approx 1000 \cdot 10^{-4} \approx \frac{10^3}{10^4} \approx \frac{10^3}{10^3 \cdot 10^1} = \frac{1}{10}$. Ово значи да је 10% шансе да пакет буде погрешан. Такође је важно да се каже да важи: $BER \approx P_e$. Сада покажујемо оба поступка:

Пristup 1 је потврда сваког пакета: $E[\text{pokusaja}] = \frac{1}{1 - P_{\text{paket}}} = \frac{1}{\frac{10}{10} - \frac{1}{10}} = \frac{1}{\frac{9}{10}} = \frac{10}{9}$. Ово је очекивани број покушаја по пакету, дакле $10/9$.

Укупан број пакета који се преносе је: $N_{\text{ef}} = \frac{100}{1} \cdot \frac{10}{9} = \frac{1000}{9}$

Време преноса 1 пакета: $t_p = \frac{X_p}{R} = \frac{1000}{10000} = \frac{1}{10} = \frac{10}{100} \text{ s}$

Укупно време преноса целог фајла: $T_1 = N_{\text{ef}} \cdot t_p = \frac{1000}{9} \cdot \frac{1}{10} = \frac{1000}{90} = \frac{100}{9} \text{ s}$

Пristup 2 је потврда целог фајла. Овде нема појединачних потврда, већ се каже цео фајл одређеном.

Ако се испостави да постоји и најмања грешка у целом фајлу, онда се цео фајл мора поново послати.

Вероватноћа да се цео фајл пренесе исправно: $P_{\text{fail-ok}} = (1 - P_{\text{paket}})^N = \left(\frac{10}{10} - \frac{1}{10}\right)^{100} \approx 2.65 \cdot 10^{-5}$ — никаква шанса.

Фајл ће се морати слати огроман број пута, а то је: $E[\text{pokusaja}] = \frac{1}{P_{\text{fail-ok}}} \approx 37700$

Укупно време преноса: $T_2 = E[\text{pokusaja}] \times \frac{X_f}{R} = 37700 \times \frac{10^5}{10^4} = 377000 \text{ s}$

NOVA

LEKCIJA

◆ 1. Nyquist-ova teorema (za bešumni kanal)

Formula je: $D_N = 2 \cdot BW \cdot \log_2(V)$

- gde je:
- D_N — maksimalna brzina prenosa podataka (bps),
 - BW — propusni opseg kanala (Hz),
 - V — broj nivoa (diskretnih amplituda) signala.

◆ 2. Shannon-ova teorema (za šumni kanal)

Formula je:

$$D_S = BW \cdot \log_2\left(1 + \frac{S}{N}\right)$$

- gde je:
- D_S — maksimalna brzina prenosa podataka (bps),
 - BW — propusni opseg (Hz),
 - $\frac{S}{N}$ — odnos signal-šum (Signal-to-Noise Ratio).

Nyquist ti kaže kolika bi bila idealna brzina.

Shannon ti kaže kolika je realna maksimalna brzina zbog šuma.

Sampling rate ti kaže koliko puta moraš uzorkovati signal.

Baud rate ti kaže koliko simbola (stanja signala) se prenosi u sekundi.

 Značenje	 Kada se koristi
Maks. brzina prenosa bez greške u idealnom kanalu	Kada nema šuma (idealni uslovi)
Maksimalni kapacitet kanala u realnim uslovima	Kada postoji šum (realni kanali)

Brzina odmeravanja
(Sampling rate)

$f_s \geq 2f_{max}$

Minimalna frekvencija uzorkovanja signala

Kod digitalizacije analognih signala (npr. zvuk, slika)

Baud Rate (simbolna brzina)

$Baud = \frac{R_b}{\log_2 M}$

Broj simbola u sekundi (koliko puta se signal menja)

Kod modulacija (ASK, PSK, QAM itd.)

Zadatak 11:

Neka je $D_{N(S)}$ - maksimalna brzina prenosa podataka, BW - propusni opseg bešumnog kanala po kome se prenose podaci, V - odgovara broju diskretnih nivoa signala.

Na osnovu Nyquist-ove teoreme brzina prenosa podataka po bešumnom kanalu se računa kao: $D_N = 2 \cdot BW \cdot \log_2 V \text{ [bps]}$

Kada postoji slučajan šum situacija se drastično razlikuje. Na osnovu Shannon-ove teoreme maksimalna brzina, tj. maksimalni broj bitova u sekundi koji može da se prenese kroz šumni kanal čiji je propusni opseg BW , a odnos signal-šum (SNR – Signal-to-Noise Ratio) S/N , u oznaci D_S , iznosi:

$$D_S = BW \cdot \log_2 \left(1 + \frac{S}{N} \right) \text{ [bps]} = BW \cdot \log_2 (1 + SNR) \text{ [bps]}$$

a) ako je bešumni kanal propusnog opsega širine 3kHz , a kroz kanal se prenose binarni impulsi odrediti maksimalnu brzinu prenosa podataka;

b) Ako je propusni opseg kanala 3 kHz, a odnos signal-šum 30 dB odrediti maksimalni broj bitova koji se u sekundi može preneti kroz ovaj kanal;

c) Televizijski kanal je širine 3kHz. Koliko se bitova u sekundi mogu poslati putem ovog kanala ako se za prenos koriste digitalni signali sa 4 nivoa (multi-nivovski prenos)? Usvojiti da je kanal bešumni.

◆ 1. Nyquist-ova teorema (za bešumni kanal)

Formula je: $D_N = 2 \cdot BW \cdot \log_2(V)$

gde je:

- D_N — maksimalna brzina prenosa podataka (bps),
- BW — propusni opseg kanala (Hz),
- V — broj nivoa (diskretnih amplituda) signala.

◆ 2. Shannon-ova teorema (za šumni kanal)

Formula je:

$$D_S = BW \cdot \log_2\left(1 + \frac{S}{N}\right)$$

gde je:

- D_S — maksimalna brzina prenosa podataka (bps),
- BW — propusni opseg (Hz),
- $\frac{S}{N}$ — odnos signal-šum (Signal-to-Noise Ratio).

a)

Bešumni kanal,

$$BW = 3 \text{ kHz} = 3000 \text{ Hz},$$

binarni prenos $V = 2$.

$$D_N = 2 \cdot 3000 \cdot \log_2(2)$$

$$\log_2(2) = 1$$

$$D_N = 2 \cdot 3000 \cdot 1 = 6000 \text{ bps}$$

✓ Odgovor (a): 6000 bps (6 kbps)

b)

Šumni kanal,

$$BW = 3 \text{ kHz} = 3000 \text{ Hz},$$

odnos signal-šum $S/N = 30 \text{ dB}$.

Prvo pretvaramo decibele u odnos:

$$\frac{S}{N} = 10^{\frac{SNR_{dB}}{10}} = 10^{\frac{30}{10}} = 10^3 = 1000$$

Sad ubacujemo u Shannonovu formulu:

$$D_S = 3000 \cdot \log_2(1 + 1000)$$

$$\log_2(1001) \approx 9.97$$

$$D_S \approx 3000 \cdot 9.97 = 29910 \text{ bps}$$

✓ Odgovor (b): $\approx 29.9 \text{ kbps}$

c)

Bežumni kanal,

$$BW = 3 \text{ kHz} = 3000 \text{ Hz},$$

broj nivoa $V = 4$.

$$D_N = 2 \cdot 3000 \cdot \log_2(4)$$

$$\log_2(4) = 2$$

$$D_N = 2 \cdot 3000 \cdot 2 = 12000 \text{ bps}$$

✓ Odgovor (c): 12 kbps

◆ Sažetak rezultata:

Deo	Uslovi	Formula	Rezultat
a)	Bešumni, $V=2$	$2BW \log_2 V$	6000 bps
b)	Šumni, SNR=30 dB	$BW \log_2(1 + S/N)$	$\approx 29,910$ bps
c)	Bešumni, $V=4$	$2BW \log_2 V$	12,000 bps

Никвистова теорема: $D_N = 2 \cdot BW \cdot \log_2(V)$ Шеннонова теорема: $D_S = BW \cdot \log_2\left(1 + \frac{N}{S}\right)$ $f_s \geq 2 \cdot f_{\max}$

2) Пошто нема шума, онај иде Никвистова теорема: $D_N = 2 \cdot 3000 \cdot 1 = 6000 \text{ b/s}$.

~~3) Пошто има шума, онај иде Шеннонова теорема: $D_S = 3000 \cdot 10^{\frac{30}{10}} = 3000 \cdot 1000$~~

BW -прописан опсег V - број нивоа сигнала (у овом случају је то 2)

3) Пошто има шума, онај иде Шеннонова теорема: $D_S = BW \cdot \log_2\left(1 + \frac{N}{S}\right)$ 30 dB претварамо у однос $\frac{N}{S}$
на следећи начин: $10^{\frac{30}{10}} = 10^3 = 1000$ па нам је онај $\log_2(1000 + 1) \approx 10$. Е сад можемо правити рачун:
 $D_S = 3000 \cdot 10 = 30000 \text{ Hz} = 30 \text{ kHz}$

4) $BW = 3000 \text{ Hz}$ $V = 4 \Rightarrow \log_2(V) = \log_2(4) = 2$ Бесшумни канал $\rightarrow$ Никвистова теорема: $D_N = 2 \cdot BW \cdot \log_2(V)$
 $D_N = 2 \cdot 3000 \cdot 2 = 12000 \text{ Hz} = 12 \text{ kHz}$

Definisati odnos koji postoji između bitske brzine prenosa (bit rate) N i signalne brzine prenosa (baud rate) S .

◆ Osnovni pojmovi:

1. Bitska brzina (bit rate) N

- Broj bitova koji se prenose po jedinici vremena.
- Jedinica: bps (bitova u sekundi).

2. Signalna brzina (baud rate) S

- Broj promena (simbola) po sekundi u prenosnom signalu.
- Jedinica: baud (ili simboli/sekundi).

💡 **Važno:** Jedan simbol može prenositi **više bitova**, zavisno od broja diskretnih nivoa signala.

◆ Veza između N i S :

Ako signal koristi V diskretnih nivoa (npr. amplituda, faza, polaritet): $N = S \cdot \log_2(V)$

gde: • N = bit rate (bps) • S = baud rate (simboli/s) • V = broj diskretnih nivoa po simbolu

◆ Intuicija:

- Ako je $V = 2$ (binarni signal, 0 ili 1): $N = S \cdot \log_2(2) = S \cdot 1 = S$

→ bit rate = baud rate, jer svaki simbol prenosi 1 bit.

- Ako je $V = 4$: $N = S \cdot \log_2(4) = S \cdot 2$

→ svaki simbol prenosi 2 bita, pa je bit rate dvostruko veći od baud rate-a.

- Ako je $V = 8$: svaki simbol prenosi 3 bita, itd.

Dakle, formula je jednostavno "bit rate = signalna brzina × broj bitova po simbolu".

Analogni signal prenosi četiri bita po signalnom elementu. Ako se u sekundi predaje 1000 signalnih elemenata, odrediti bitsku brzinu prenosa.

◆ **Podaci iz zadatka:**

- Broj bitova po signalnom elementu: $\log_2(V) = 4$ bita/simbol
- Broj signalnih elemenata po sekundi (baud rate): $S = 1000$ simbol/s

◆ **Formula:** $N = S \cdot \text{broj bitova po simbolu}$ $N = S \cdot \log_2(V)$

U našem slučaju: $N = 1000 \cdot 4$ $N = 4000$ bps

Dajemo vam binarni niz koji treba kodirati pomoću Manchester kodiranja. Binarni niz je: 1011001101 Vaš zadatak je da pretvorite ovaj binarni niz u odgovarajući Manchester kod i prikažete rezultirajući signal. Koje su prednosti a koje mane ovog načina kodiranja?

3 Manchester kod

Korak po korak, bit po bit:

Bit	Manchester kod
1	Low → High
0	High → Low
1	Low → High
1	Low → High
0	High → Low
0	High → Low
1	Low → High
1	Low → High
0	High → Low
1	Low → High

Manchester kodiranje se koristi da binarni signal prenese sa samosinhronizacijom. Pravila su:

- 0 → tranzicija visok → nizak (High → Low) u sredini bita
- 1 → tranzicija nizak → visok (Low → High) u sredini bita

Svaki bit traje jedan vremenski interval, ali unutar njega postoji promena napona, što omogućava prijemniku da lakše sinhronizuje signal.

2 Dat niz Binarni niz: 1 0 1 1 0 0 1 1 0 1

4 Prednosti Manchester kodiranja

1. Samosinhronizacija – svaka promena napona unutar bita omogućava prijemniku da ostane sinhronizovan.
2. Lakše detektovanje grešaka – iznenadne promene napona ukazuju na moguće greške.
3. Eliminacija DC komponente – prosečna vrednost napona je blizu nule, pogodno za dugolinijske prenose.

5 Mane Manchester kodiranja

1. Dupla širina pojasa (bandwidth) – potrebno je dvostruko više frekvencije u odnosu na običan NRZ signal.
2. Veća kompleksnost – više promena napona, što može povećati opterećenje na prenosnoj liniji.
3. Manje energetski efikasno – više tranzicija znači više potrošnje energije kod nekih medija.

Dat vam je binarni niz podataka koji treba kodirati pomoću 4B/5B kodiranja. Binarni niz izgleda ovako:

110110001010101011101001. Vaš zadatak je da pretvorite ovaj binarni niz u odgovarajući 4B/5B kod i zapišete rezultirajući kodirani niz.

Prednosti i mane? Evo tabele mapiranja 4B/5B kodiranja:

4B vrednost	5B vrednost
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101

1 Dat niz

1101 1000 1010 1010 1110 1001

2 Prevod svakog 4-bitnog bloka u 5-bitni blok

Koristimo tabelu:

4B	5B
1101	11011
1000	10010
1010	10110
1010	10110
1110	11100
1001	10011

4 Prednosti 4B/5B kodiranja

1. Bez dugih nizova istih bitova (run-length) – omogućava sinhronizaciju jer u kodu nema više od 3 uzastopna nula.
2. Lakša detekcija grešaka – nelegalne 5-bitne kombinacije mogu ukazati na grešku.
3. Efikasna integracija sa fizičkim slojem – obično se koristi sa NRZI signalizacijom, smanjuje DC komponentu.

5 Mane 4B/5B kodiranja

1. Povećava broj bitova za 25% – svaki 4-bitni blok postaje 5 bita, što zahteva veću propusnost kanala.
2. Veća složenost koda/dekoda – potreban je mapping i konverzija sa 4B na 5B i obrnuto.

Dajemo vam binarni niz podataka koji treba kodirati pomoću bipolarnog kodiranja. Binarni niz izgleda ovako: 100101001101 Vaš zadatak je da pretvorite ovaj binarni niz u odgovarajući bipolarni kod i zapišete rezultirajući kodirani niz. Koje su prednosti a koje mane ovog načina kodiranja?

1 Bipolarno kodiranje – osnovna pravila

Bipolarno kodiranje (takođe poznato i kao Alternate Mark Inversion – AMI) radi ovako:

- $0 \rightarrow 0\text{ V}$ (nema napona)
- $1 \rightarrow +V, -V, +V, -V...$ naizmenično za svaki uzastopni 1
(tj. pozitivna i negativna amplituda se smenjuju kako bi se eliminisala DC komponenta)

4 Rezultujući bipolarni signal

+V 0 0 -V 0 +V 0 0 -V +V 0 -V

5 Prednosti bipolarnog kodiranja

- 1. Nema DC komponente** – pozitivni i negativni signali se balansiraju.
- 2. Samosinhronizacija** – na osnovu promena napona moguće je sinhronizovati prijemnik.
- 3. Otkrivanje grešaka** – ako se pojavljuju dva uzastopna ista znaka za 1, signal ukazuje na grešku.

6 Mane bipolarnog kodiranja

- 1. Složenija implementacija** – potrebna kontrola naizmeničnog znaka.
- 2. Ne eliminiše sve moguće duge nizove nula** – ako ima puno uzastopnih 0, prijemnik može izgubiti sinhronizaciju.
- 3. Veća širina pojasa** – zbog naizmeničnih promena napona, potrebno je više frekvencije za prenos.

Ako je bitska brzina analognog signala 8000 bps, a baud-ova 1000 Baud, odrediti koliko se elemenata podataka prenosi po signalnom elementu i koliko je signalnih elemenata potrebno?

1 Podaci iz zadatka

- Bitska brzina (bit rate): $N = 8000$ bps
- Signalna brzina (baud rate): $S = 1000$ Baud

Treba odrediti:

1. Broj bitova po signalnom elementu
2. Broj signalnih elemenata potrebnih za prenos

2 Broj bitova po signalnom elementu

Formula: broj bitova po elementu = $\frac{N}{S}$

$$\frac{8000}{1000} = 8 \text{ bita po elementu}$$

3 Broj signalnih elemenata potrebnih

Formula:

$$\text{broj signalnih elemenata} = \frac{\text{ukupan broj bitova}}{\text{bitova po elementu}}$$

Ako želimo preneti N bitova, po elementu prenesemo 8 bita, znači:

$$\text{broj elemenata} = \frac{N}{8} = \frac{8000}{8} = 1000 \text{ signalnih elemenata po sekundi}$$

Tvoje rešenje sa $L = 2^8 = 256$ zapravo nije broj signalnih elemenata, već **maksimalni broj različitih kombinacija koje jedan signalni element može da prenese**. To je druga strana iste priče:

Objašnjenje

1. Broj bitova po signalnom elementu: $n = \frac{N}{S} = \frac{8000}{1000} = 8$ bita po elementu

✓ To smo već izračunali.

2. Ako jedan signalni element prenosi $n = 8$ bitova, tada **maksimalan broj različitih stanja (nivoi signala)** po elementu je: $L = 2^n = 2^8 = 256$

- To znači da signal može da ima **256 različitih nivoa** da bi prenio 8 bitova po elementu.

Dakle:

- Bitova po signalnom elementu: 8
- Signalnih elemenata po sekundi: 1000
- Maksimalan broj različitih signala po elementu: 256 ✓

- a) Neka je propusni opseg govornog signala od 50 Hz do 10 kHz, a muzičkog signala od 15 Hz do 20 kHz. Odrediti bitsku brzinu prenosa podataka koja se generiše od strane procedure za digitalizaciju signala za svaki od ova dva slučaja, ako usvojimo da je zadovoljen Nyquist-ov kriterijum uzorkovanja sa 12 bitova po uzorku za govorni signal i 16 bitova po uzorku za muzički signal.
- b) Odrediti kapacitet memorije potreban za zapis desetominutne stereofonske muzike.

Podaci iz zadatka

Govorni signal

- Propusni opseg: $f_g = 50 \text{ Hz} - 10 \text{ kHz}$
- Broj bita po uzorku: $b_g = 12 \text{ bita}$

Muzički signal

- Propusni opseg: $f_m = 15 \text{ Hz} - 20 \text{ kHz}$
- Broj bita po uzorku: $b_m = 16 \text{ bita}$

a) Bitska brzina prenosa podataka

Po Nyquist-ovom kriterijumu za uzorkovanje: $f_s \geq 2 \cdot f_{\max}$

gde je $f_{\max}$ najveća frekvencija signala.

1 Govorni signal Najveća frekvencija: $f_{\max} = 10 \text{ kHz}$

Uzorkovanje: $f_s = 2 \cdot f_{\max} = 2 \cdot 10\,000 = 20\,000 \text{ uzoraka/s}$

Bitska brzina: $N = f_s \cdot b_g = 20\,000 \cdot 12 = 240\,000 \text{ bps} = 240 \text{ kbps}$

✓ Govorni signal: 240 kbps

2 Muzički signal

Najveća frekvencija: $f_{\max} = 20\,000\text{ Hz}$

Uzorkovanje: $f_s = 2 \cdot f_{\max} = 2 \cdot 20\,000 = 40\,000\text{ uzoraka/s}$

Bitska brzina: $N = f_s \cdot b_m = 40\,000 \cdot 16 = 640\,000\text{ bps} = 640\text{ kbps}$

✓ Muzički signal: 640 kbps po kanalu

Za stereo zvuk (dva kanala): $N_{\text{stereo}} = 640\,000 \cdot 2 = 1\,280\,000\text{ bps} = 1.28\text{ Mbps}$

b) Kapacitet memorije za desetominutnu stereo muziku

1 minuta = 60 s, dakle 10 minuta = 600 s.

Broj bitova = $N_{\text{stereo}} \cdot t = 1\,280\,000 \cdot 600 = 768\,000\,000\text{ bita}$

U bajtovima: $768\,000\,000 \div 8 = 96\,000\,000\text{ B} \approx 96\text{ MB}$

$$\begin{aligned}
 f_{g\min} &= 50 \text{ Hz} & f_{g\max} &= 10000 \text{ Hz} & f_{m\min} &= 15 \text{ Hz} & f_{m\max} &= 20000 \text{ Hz} & f_{gs} &= 2 * f_{g\max} = 2 * 10000 = 20000 \text{ Hz} \\
 f_{ms} &= 2 * f_{m\max} = 2 * 20000 = 40000 \text{ Hz} & D_{Ng} &= b_g * f_{gs} = 12 * 20000 = 240000 \frac{\text{b}}{\text{s}} & D_{Nm} &= b_m * f_{ms} = \\
 &= 16 * 40000 = 640000 \text{ b/s.} & \text{За стерео ЗВК постоје 2 канала: зато кажемо: } N_{\text{stereo}} &= 640000 * 2 \text{ што је} \\
 &1280000 \text{ b/s.} & \text{Сада можемо да изведемо закључак за 10-минутну стерео музику: } 1280000 * 600 = \\
 &= 768000000 \text{ b}
 \end{aligned}$$

HDTV koristi digitalne signale za emisiju kvalitetnih video signala. Po jednoj slici se prenosi 1920×1080 piksela, a slike se obnavljaju 25 puta u sekundi. Svaki piksel se prenosi sa 24 bita. Odrediti bitsku brzinu prenosa.

Podaci iz zadatka

- Rezolucija: 1920×1080 piksela
- Broj slika u sekundi: $f = 25 \text{ Hz}$
- Broj bita po pikselu: 24 bita

1 Broj piksela po slici

Broj piksela po slici $= 1920 \times 1080 = 2\,073\,600$ piksela

2 Broj bita po slici

Biti po slici $= 2\,073\,600 \times 24 = 49\,766\,400$ bita

3 Bitska brzina prenosa

Slike se prenose 25 puta u sekundi:

$$N = 49\,766\,400 \times 25 = 1\,244\,160\,000 \text{ bps} \approx 1.244 \text{ Gbps}$$

Sliku čini 1024x768 piksela sa 3 bajta po pikselu. Usvojimo da slika nije komprimovana. Koliko vremena je potrebno da se prenese preko: a) 56 kbps modemskeg kanala, b) 1 Mbps Ethernet-a, c) 10 Mbps Ethernet-a, d) 100 Mbps Ethernet-a?

1 Veličina slike u bitovima

1 piksel = 3 bajta = 24 bita

Broj bitova po slici = $1024 \cdot 768 \cdot 24 = 18\,874\,368$ bita

✓ Slika = 18,874,368 bita $\approx$ 18.87 Mbit

2 Vreme prenosa

Vreme prenosa: $t = \frac{\text{Broj bitova}}{\text{Brzina kanala}}$

a) 56 kbps modem $t = \frac{18\,874\,368}{56\,000} \approx 337$ sekundi

a) 56 kbps modem

$$t = \frac{18\,874\,368}{56\,000} \approx 337 \text{ sekundi}$$

b) 1 Mbps Ethernet

$$t = \frac{18\,874\,368}{1\,000\,000} \approx 18.87 \text{ sekundi}$$

c) 10 Mbps Ethernet

$$t = \frac{18\,874\,368}{10\,000\,000} \approx 1.887 \text{ sekundi}$$

d) 100 Mbps Ethernet

$$t = \frac{18\,874\,368}{100\,000\,000} \approx 0.1887 \text{ sekundi}$$

FORMULE

Podaci:

- Talasna dužina: λ
- Brzina svetlosti (ili brzina talasa u datom medijumu): c
- Frekvencija signala: f

Formula: $\lambda = \frac{c}{f}$

Objašnjenje:

- Talasna dužina λ je rastojanje koje talas pređe u jednom ciklusu.
- Brzina talasa c je koliko metara talas pređe u jednoj sekundi.
- Frekvencija f je broj ciklusa u jednoj sekundi.
- Zato delimo brzinu talasa sa frekvencijom da dobijemo dužinu jednog talasa.

ZADACI

Koliki frekventni spektar odgovara promeni talasne dužine signala od $0,1\mu\text{m}$ pri promeni talasne dužine od $1\mu\text{m}$?

1 Veza između frekvencije i talasne dužine

Frekvencija f i talasna dužina λ su povezane:

$$f = \frac{c}{\lambda}$$

gde je $c \approx 3 \cdot 10^8 \text{ m/s}$ brzina svetlosti u vakuumu.

2 Promena frekvencije zbog promene talasne dužine

Ako talasna dužina promeni za $\Delta\lambda$, frekvencija se menja:

$$f + \Delta f = \frac{c}{\lambda - \Delta\lambda} \quad (\text{ako smanjujemo talasnu dužinu})$$

Za male promene, aproksimacija diferencijalom: $df = -\frac{c}{\lambda^2} d\lambda$

Znak minus znači da povećanje talasne dužine smanjuje frekvenciju,

ali mi tražimo samo apsolutnu vrednost spektra: $\Delta f \approx \frac{c \cdot \Delta\lambda}{\lambda^2}$

3 Ubacivanje podataka

Imamo: $\lambda = 1 \mu\text{m} = 1 \cdot 10^{-6} \text{ m}$

$$\Delta\lambda = 0.1 \mu\text{m} = 0.1 \cdot 10^{-6} \text{ m} = 1 \cdot 10^{-7} \text{ m}$$

Dakle:

$$\Delta f \approx \frac{3 \cdot 10^8 \cdot 1 \cdot 10^{-7}}{(1 \cdot 10^{-6})^2}$$

4 Izračunavanje

Izracunavanje:

$$(1 \cdot 10^{-6})^2 = 1 \cdot 10^{-12}$$

$$3 \cdot 10^8 \cdot 1 \cdot 10^{-7} = 3 \cdot 10^1 = 30$$

$$\Delta f = \frac{30}{1 \cdot 10^{-12}} = 3 \cdot 10^{13} \text{ Hz}$$

$$\lambda = 10^{-6} \text{ m} \quad \Delta \lambda = 10^{-7} \text{ m} \quad \Delta f = \frac{c \cdot \Delta \lambda}{\lambda^2} \quad c = 3 \cdot 10^8 \frac{\text{m}}{\text{s}} \quad \Delta f = \frac{3 \cdot 10^8 \cdot 10^{-7}}{10^{-12}} = 3 \cdot 10^{13} \text{ Hz}$$

Potrebno je poslati sekvencu računarskih slika preko optičkog vlakna. Slike su formata 480*640 piksela, pri čemu se svaki piksel predstavlja u RGB formatu sa 24 bita. U sekundi je potrebno preneti 60 slika. (Frekvencija osvežavanja). Kolika je potrebna propusnost tj. koliko mikrona talasne dužine je potrebno za ovaj propusni opseg pri talasnoj dužini nosioca od 1.30μm?

1 Podaci iz zadatka

- Format slike: 480 × 640 piksela
- Svaki piksel: RGB, 24 bita
- Frekvencija osvežavanja: 60 slika/s
- Nosioc: talasna dužina $\lambda = 1.30 \mu\text{m}$
- Pitanje: potrebna propusnost i koliko to znači u mikronima talasne dužine.

2 Korak 1: Ukupan broj bitova po slici

$$\text{Broj piksela} = 480 \cdot 640 = 307\,200$$

$$\text{Broj bitova po slici} = 307\,200 \cdot 24 = 7\,372\,800 \text{ bita}$$

3 Korak 2: Ukupan broj bitova po sekundi (bit rate)

$$R = 7\,372\,800 \text{ bita/sliku} \cdot 60 \text{ slika/s}$$

$$R = 442\,368\,000 \text{ bps} \approx 442.37 \text{ Mbps}$$

✓ Dakle, propusnost digitalnog signala je 442.37 Mbps.

4 Korak 3: Prelazak sa propusnosti u talasni spektar

Veza između promene frekvencije Δf i promene talasne dužine $\Delta\lambda$:

$$\Delta f = \frac{c}{\lambda^2} \Delta\lambda$$

Gde je:

- $c \approx 3 \times 10^8 \text{ m/s}$ (brzina svetlosti u vakuumu)
- $\lambda = 1.30 \text{ }\mu\text{m} = 1.3 \times 10^{-6} \text{ m}$
- $\Delta f \approx \text{bit rate} = 442.37 \times 10^6 \text{ Hz}$

Rešavamo za $\Delta\lambda$:

$$\Delta\lambda = \frac{\lambda^2 \Delta f}{c}$$

5 Korak 4: Umetanje vrednosti

$$\Delta\lambda = \frac{(1.3 \cdot 10^{-6})^2 \cdot 442.37 \cdot 10^6}{3 \cdot 10^8}$$

Prvo kvadriramo λ :

$$(1.3 \cdot 10^{-6})^2 = 1.69 \cdot 10^{-12}$$

Množimo sa Δf :

$$1.69 \cdot 10^{-12} \cdot 442.37 \cdot 10^6 \approx 7.472 \cdot 10^{-4}$$

Delimo sa c :

$$\Delta\lambda \approx \frac{7.472 \cdot 10^{-4}}{3 \cdot 10^8} \approx 2.49 \cdot 10^{-12} \text{ m}$$

$$\Delta\lambda \approx 0.00249 \text{ }\mu\text{m} \approx 2.5 \text{ nm}$$

Формат слике: 480×640 Имамо и 24 бита по пикселу. Аимензија 1 слике: 307200 пиксела
 Сваки пиксел се представља са 24 бита. Ајме: $307200 \cdot 24 = 7372800$ бита — оволико по слици
 $7372800 \cdot 60 = 442368000$ бита $\Delta f = 442368000$ $\Delta f = \frac{c}{\lambda^2} \cdot \Delta \lambda$
 $442368000 = \frac{3 \cdot 10^8}{(1.30 \cdot 10^{-6})^2} \cdot \Delta \lambda$ $3 \cdot 10^8 \cdot \Delta \lambda = 1.69 \cdot 10^{-12} \cdot 442368 \cdot 10^3 = 1.69 \cdot 10^{-12} \cdot 4.42 \cdot 10^8$

Једна слика: 480×640 пиксела. Један пиксел је 24 бита. За 1 слику треба: $480 \cdot 640 \cdot 24 = 7372800$ б
 За 60 слика треба: $7372800 \cdot 60 = 442368000$ б/с $\Delta f = \frac{c \cdot \Delta \lambda}{\lambda^2}$ $442368000 = \frac{3 \cdot 10^8 \cdot \Delta \lambda}{(1.30 \cdot 10^{-6})^2}$

Deset signala, svakom je potreban opseg od 4000 Hz, se multipleksiraju u jedinstven kanal koristeći FDM (Frequency Division Multiplex). Koliki je minimalni propusni opseg potreban za multipleksiranje kanala? Usvojiti da je opseg razdvajanja (guard band) između kanala širine 400Hz.

1 Podaci iz zadatka

- Broj signala: $N = 10$
- Svaki signal zahteva: $B = 4000 \text{ Hz}$
- FDM guard band između kanala: $G = 400 \text{ Hz}$

Cilj: minimalni propusni opseg za multiplexing.

2 Logika

Kod FDM-a, svaki kanal dobija svoj opseg + guard band između susednih kanala.

- Za prvi kanal guard band ide samo sa desne strane.
- Za srednje kanale guard band se deli sa levom i desnom stranom → ali jednostavno možemo računati kao G između kanala.
- Za poslednji kanal nema guard band sa desne strane (ili se ignoriše minimalno).

Dakle, formula:

$$B_{\text{total}} = N \cdot B + (N - 1) \cdot G$$

3 Umetanje vrednosti

$$B_{\text{total}} = 10 \cdot 4000 + (10 - 1) \cdot 400$$

$$B_{\text{total}} = 40\,000 + 9 \cdot 400$$

$$B_{\text{total}} = 40\,000 + 3\,600$$

$$B_{\text{total}} = 43\,600 \text{ Hz}$$



Zaključak

Minimalni propusni opseg kanala za FDM:

43.6 kHz

Zašto je formula:

$$B_{\text{total}} = N \cdot B_s + (N - 1) \cdot B_g$$

1 Opseg jednog signala

Svaki signal zahteva B_s Hz.

Ako imamo 1 signal, minimalni propusni opseg je samo taj signal:

$$B_{\text{total}} = B_s$$

2 Guard band (razmak)

Da signali ne bi preklapali, između svakog susednog signala stavljamo razmak B_g .

- Ako imamo 2 signala, jedan razmak između njih: $B_{\text{total}} = B_s + B_g + B_s = 2B_s + 1B_g$
- Ako imamo 3 signala, razmaci su između 1-2 i 2-3, dakle 2 razmaka:

$$B_{\text{total}} = B_s + B_g + B_s + B_g + B_s = 3B_s + 2B_g$$

Vidiš obrazac?

- Za N signala imamo $N - 1$ razmaka između njih.

Da sumiramo:

- $B_s = 4000 \text{ Hz}$ → opseg jednog signala
- $N = 10$ → broj signala
- $B_g = 400 \text{ Hz}$ → razmak između signala (guard band)
- $N - 1 = 9$ → broj razmaka između 10 signala

Formula:

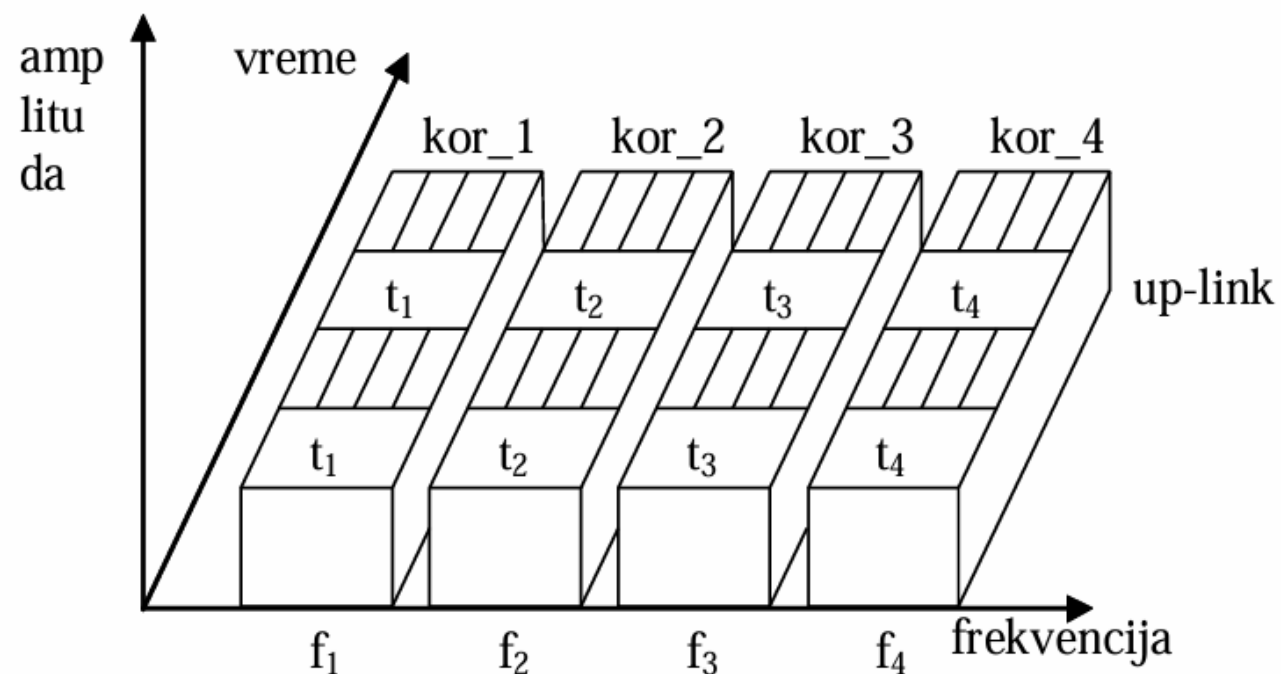
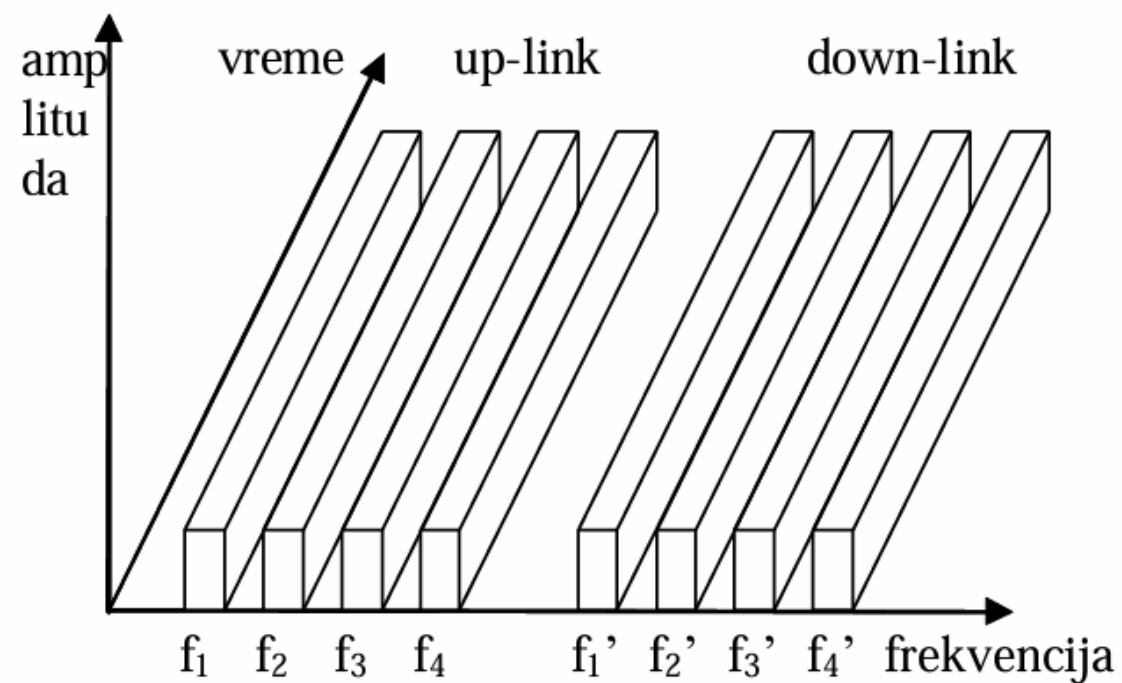
$$B_{\text{total}} = N \cdot B_s + (N - 1) \cdot B_g = 10 \cdot 4000 + 9 \cdot 400 = 43\,600 \text{ Hz} = 43.6 \text{ kHz}$$

Sve se slaže. 

Kada imamo N signala, onda imamo $N-1$ razmaka između njih, pa je formula onda tako da se izvede

$$BW = N * B + (N-1) * G = 10 * 4000 + 9 * 400 = 40000 + 3600 = 43600 \text{ Hz}$$

Koja od Slika a) i b) odgovara FDMA/FDD i FDMA/TDD tehnici prenosa?



2 Analiza

◆ 2.1 FDMA/FDD

- FDMA = *Frequency Division Multiple Access* → različiti korisnici dobijaju različite frekvencije.
- FDD = *Frequency Division Duplex* → up link i down link koriste različite frekvencije (odvojene frekvencijski).

Ključni znakovi:

- Up link i down link su razdvojeni frekvencijski
- Svaki korisnik ima svoju frekvenciju

→ To odgovara Slika 1, jer vidiš $f_1 \dots f_4$ za up link i $f_1' \dots f_4'$ za down link.

◆ 2.2 FDMA/TDD

- TDD = *Time Division Duplex* → up link i down link koriste isti frekvencijski opseg, ali se vreme deli u slotove
- Brojevi koraka (time slots) pokazuju koji slot pripada up linku, koji down linku

→ To odgovara Slika 2, gde vidiš korak 1, 2, 3, 4 (tj. različite vremenske slotove).

FDMA/FDD се односи на фреквенцију, а FDMA/TDD се односи на време.

Zadatak 26-27 :	CDMA A: 0 B: 1 C: 0 D: idle a)	A: (-1 -1 -1 +1 +1 -1 +1 +1) B: (-1 -1 +1 -1 +1 +1 +1 -1) C: (-1 +1 -1 +1 +1 +1 -1 -1) D: (-1 +1 -1 -1 -1 -1 +1 -1) b)
-----------------	--	--

◆ 1 Tumačenje podataka

Svaki korisnik ima:

- svoj **kodni niz** (tzv. *chip sequence*) dužine 8,
- i stanje:
 - **1** → šalje $+1 \times$ (kodni niz)
 - **0** → šalje $-1 \times$ (kodni niz)
 - **idle** → ne šalje ništa (0 niz).

Dakle:

Korisnik	Bit	Kodni niz	Prenos
A	0	$(-1 -1 -1 +1 +1 -1 +1 +1)$	invertuje kod: $(+1 +1 +1 -1 -1 +1 -1 -1)$
B	1	$(-1 -1 +1 -1 +1 +1 +1 -1)$	šalje kod onakav kakav jeste
C	0	$(-1 +1 -1 +1 +1 +1 -1 -1)$	invertuje kod: $(+1 -1 +1 -1 -1 -1 +1 +1)$
D	idle	$(-1 +1 -1 -1 -1 -1 +1 -1)$	šalje nulu $(0 0 0 0 0 0 0 0)$

◆ 2 Sabiranje svih prenetih nizova (superpozicija signala)

Zbir svih signala (A + B + C + D):

Chip	A	B	C	D	Zbir
1	+1	-1	+1	0	+1
2	+1	-1	-1	0	-1
3	+1	+1	+1	0	+3
4	-1	-1	-1	0	-3
5	-1	+1	-1	0	-1
6	+1	+1	-1	0	+1
7	-1	+1	+1	0	+1
8	-1	-1	+1	0	-1

📡 Rezultat prenetog signala (zbirni signal):

👉 (+1, -1, +3, -3, -1, +1, +1, -1)



◆ **3** Ako bi primalac želeo da dekodira recimo korisnika B

Množi primljeni signal sa kodnim nizom korisnika B i sabira proizvode:

Ako $\text{zbir} > 0 \rightarrow \text{bit} = 1$,

ako $\text{zbir} < 0 \rightarrow \text{bit} = 0$.

Isto se može proveriti i za A, C, D (D će dati nulu jer je idle).

ADSL – Asymetric Digital Subscriber Line sistem koji koristi DMT (Discrete multitone) dodeljuje $\frac{3}{4}$ od dostupnih kanala downstream linku. Po svakom kanalu ADSL koristi se modulacija tipa QAM-64. Koliki je kapacitet downstream linka?

◆ **1 DMT sistem u ADSL-u ima 256 podnosača (tonova)**

To je nominalno broj koji dolazi iz 512-tačke FFT (polovina od njih je simetrična, pa se koristi 256 tonova).

Ali — ne koristi se svih 256 za prenos podataka.

◆ **2 Prvi tonovi su rezervisani**

U realnim ADSL sistemima:

- Ton 0 (DC komponenta) se ne koristi — on je samo referenca (DC nosilac).
- Nekoliko donjih tonova (1–6) se koriste za POTS (Plain Old Telephone Service), tj. analogni telefonski signal.
- Dakle, otprilike 8 tonova ukupno ne prenose digitalne podatke, već su rezervisani.

Zato efektivni broj upotrebljivih kanala nije 256, nego:

$$N_{\text{korisno}} = 256 - 8 = 248$$

◆ **3 Downstream deo koristi $\frac{3}{4}$ od toga**

$$N_d = \frac{3}{4} \times 248 = 186$$

↓

(tačno 186 kanala za downlink)

◆ 4 Sada računamo kapacitet

Za QAM-64 imamo 6 bitova/simbol, i po kanalu 4000 simbola/s:

$$C_{\text{kanal}} = 6 \times 4000 = 24,000 \text{ b/s}$$

Ukupno:

$$C_{\text{down}} = 186 \times 24,000 = 4,464,000 \text{ b/s}$$

$$C_{\text{down}} = 4.46 \text{ Mb/s}$$

◆ 5 Zaključak

Dakle, razlika između 256 i 248 dolazi od **rezervisanih tonova** (za sinhronizaciju, kontrolu i telefonski opseg).

U realnim ADSL sistemima se **uvek uzima oko 248 tonova** kao korisnih.

DMT систем. ADSL користи 256 подносача (тонова). Не користи се свих 256 тонова за пренос. Тон 0 је DC компонента и он се не користи. Тонове од 1 до 6 се користе за POTS (телефонски сигнал). Дакле, рећемо да се користи 248 тонова за пренос. Ефективни број употребљивих канала није 256, већ 248. Запишаћемо то овако: $N_{\text{korisno}} = 248$. $N_u = \frac{3}{4} \cdot 248 = 186$. Downstream користи 186 канала. За QAM-64 важи да имамо 6 b/symbol, што је добијено као $\log_2(64) = 6$. Значи, 6 b/symbol, и 4000 symbol/s, е то важи за QAM-64. Добија се по једном каналу: $C_{\text{kanal}} = 4000 \cdot 6 = 24000$ b/s. А за 186 канала (downstream) је: $C_d = 24000 \cdot 186 = 4464000$ b/s.

ADSL има 256 канала, али први канал је DC компонента и он се не користи, а канали од 2 до 8 су телефонски сигнали. Значи, ефективно, ADSL користи 248 канала. Три четвртине од тога је 186 канала. $N_{\text{downstream}} = 186$ канала. За QAM-64 имамо 6 bitova/symbol и 4000 symbol/s, па се тако добија: $C_{\text{kanal}} = 6 \cdot 4000 = 24000$ b/s. Дакле, ово је битска брзина по једном каналу. А имамо их 186. Према томе: $C_{\text{downstream}} = 24000 \cdot 186 = 4464000$ b/s.

Tri paketno komutirane mreže, različitih topologija imaju po jednak broj čvorova n . Prva mreža je topologije tipa zvezda (star), druga je bidirekcionni prsten (ring), a treća potpuno povezana mreža (mesh) sa ostvarenim vezama od čvora ka svim ostalim čvorovima. Kod koje mreže su najbolji, prosečni i najgori prenosni putevi sa aspekta broja preskoka?

1 Podaci iz zadatka

- Tri mreže, isteg broja čvorova n :
 1. Zvezda (star)
 2. Bidirekcionni prsten (ring)
 3. Potpuno povezana mreža (mesh)
- Pitanje: koja mreža ima najbolji, prosečni i najgori prenosni put u smislu broja preskoka (hops).

2 Analiza po mrežama

◆ 2.1 Star (zvezda)

- Ima centralni čvor.
- Svaki čvor je povezan samo sa centralnim čvorom.

Broj preskoka:

- Iz čvora do centralnog čvora: 1 hop
- Iz jednog perifernog čvora do drugog perifernog čvora: 2 hops (put preko centralnog)

Zaključak:

- Najbolji put: 1 hop (iz periferije ka centru ili obrnuto)
- Najgori put: 2 hops (iz periferije ka periferiji)
- Prosečan put: između 1 i 2 hops, zavisi od n.

◆ 2.2 Ring (prsten)

- Svaki čvor povezan sa **dva susedna čvora**, formira zatvoreni krug.
- Broj preskoka između čvorova zavisi od **kraće strane prstena**.

Broj preskoka:

- Najkraći put između dva čvora: $\leq n/2$ hops
- **Najbolji put:** 1 hop (između susednih čvorova)
- **Najgori put:** $\lfloor n/2 \rfloor$ hops (dva čvora su nasuprot prstena)
- **Prosečan put:** $\approx n/4$ hops (srednja vrednost)

◆ 2.3 Fully Connected Mesh (potpuno povezana)

- Svaki čvor direktno povezan sa **svim ostalim čvorovima**.

Broj preskoka:

- Svaka komunikacija ide direktno između dva čvora → **uvek 1 hop**
- **Najbolji put:** 1 hop
- **Najgori put:** 1 hop
- **Prosečan put:** 1 hop

3 Uporedni zaključak

Topologija	Najbolji put (hops)	Prosečan put (hops)	Najgori put (hops)
Star	1	$\sim 1-2$	2
Ring	1	$\sim n/4$	$n/2$
Mesh	1	1	1

Zaključak:

- **Mesh:** optimalna u svim aspektima, ali skupa za realizaciju (potrebno $n(n - 1)/2$ veza).
- **Star:** dobra centralizovana topologija, ali prosečan put veći nego mesh.
- **Ring:** najmanje efikasna sa aspekta preskoka, prosečni i najgori put rastu sa brojem čvorova.

Знам како изгледа топологија звезде, и знам да без обзира на n (број чворова) увек је 2 хопа.

Знам и како изгледа mesh топологија (магистрала) где је увек 1 хоп.

Топологија прстена, најкраћи пут је 1 хоп, а најдужи пут је $n/2$ хопова.

Uporediti kašnjenje kod slanja x -bitne poruke preko puta (linka) koji ima k -preskoka (hops) kod mreže sa komutacijom kola (circuit switching) i kod slabo opterećene mreže sa komutacijom paketa (packet switching).

Vreme uspostavljanja veze (circuit setup time) iznosi s sekundi, propagaciono kašnjenje po preskoku je d sekundi, obim paketa je p bitova, a brzina prenosa podataka iznosi b bitova po sekundi. Pod kojim uslovima mreža sa komutacijom paketa ima manje kašnjenje?

1 Circuit Switching (komutacija kola)

Kod circuit switching (mreža sa komutacijom kola):

- Pre prenosa, mora da se **uspostavi veza** kroz sve preskoke.
- Vreme kašnjenja se računa kao:

$$T_{\text{circuit}} = s + (k + 1) \cdot \frac{x}{b} + (k + 1) \cdot d$$

Gde je:

- s = vreme uspostavljanja veze (circuit setup),
- k = broj preskoka (hops),
- x = broj bitova poruke,
- b = brzina prenosa (bps),
- d = propagaciono kašnjenje po hopu.

Objašnjenje:

1. s — vreme da se uspostavi put kroz sve preskoke.
2. $(k + 1) \cdot x/b$ — vreme predaje cele poruke kroz sve preskoke (circuit switching predaje celu poruku „kontinuirano“).
3. $(k + 1) \cdot d$ — propagaciono kašnjenje kroz svaki hop.

2 Packet Switching (komutacija paketa)

Kod packet switching:

- Poruka se deli na pakete od p bitova.
- Svaki paket se šalje čim je spreman (store-and-forward).
- Vreme kašnjenja za slabo opterećenu mrežu (ne čekamo redove) je:

$$T_{\text{packet}} = (k + 1) \cdot \frac{p}{b} + (k + 1) \cdot d + \left(\frac{x}{p} - 1 \right) \cdot \frac{p}{b}$$

Gde:

- $(k + 1) \cdot p/b$ — vreme predaje prvog paketa kroz sve hopove.
- $(k + 1) \cdot d$ — propagaciono kašnjenje prvog paketa.
- $\left(\frac{x}{p} - 1 \right) \cdot p/b$ — vreme predaje preostalih paketa (one koji „slede“ iza prvog).

Napomena: ovo važi kada mreža nije zagušena, tj. nema redova čekanja na svaki hop.

3 Uslovi kada packet switching ima manje kašnjenje

Da bi packet switching bio brži od circuit switching:

$$T_{\text{packet}} < T_{\text{circuit}}$$

Ako ubacimo formule i pojednostavimo:

$$(k+1) \cdot \frac{p}{b} + (k+1) \cdot d + \left(\frac{x}{p} - 1\right) \cdot \frac{p}{b} < s + (k+1) \cdot \frac{x}{b} + (k+1) \cdot d$$

Propagaciona kašnjenja $(k+1)d$ se poništavaju, pa ostaje:

$$\frac{x}{b} < s + (k+1) \cdot \frac{x}{b} - \frac{x-p}{b}?$$

Bolje da pojednostavimo korak po korak:

1. Levo: $T_{\text{packet}} = \frac{x}{b} + k \cdot \frac{p}{b}$ (nakon sabiranja svih članova)
2. Desno: $T_{\text{circuit}} = s + (k+1) \cdot \frac{x}{b}$

Zaključak: Packet switching ima manje kašnjenje ako je: broj bitova poruke x dovoljno veliki, vreme uspostavljanja veze s kod circuit switching nije zanemarljivo.

Circuit switching - Цео пакет се предаје континуирано. Пре преноса, мора да се успостави веза кроз све прескоке. Зато имамо: s - време да се успостави веза кроз све хопове. $(k+1)^*d$ - пропатационо кашњење. $(k+1)^*(X/b)$ - време испоруке целе поруке кроз све хопове (цео пакет се предаје истовремено) кроз сваки хоп.

Packet switching - $(\frac{X}{p} - 1)^* \frac{p}{b}$ - време које је потребно да се подаци преостали пакети, после првог. $(k+1)^*d$ - пропатационо кашњење првог пакета. $(k+1)^*(p/b)$ - време предаје првог пакета кроз све хопове.

Заједничко за оба ова је пропатационо кашњење: $(k+1)^*d$. Када напишемо изразе, следеће што треба израдити је да напишемо: $T_{\text{packet}} < T_{\text{circuit}}$. Потребно је разумети из контекста шта се односи на шта.

$$(k+1)^*\left(\frac{p}{b}\right) + (k+1)^*d + \left(\frac{X}{p} - 1\right)^*\frac{p}{b} < s + (k+1)^*\frac{X}{b} + (k+1)^*d$$

Код circuit switching, нигде немамо $\frac{p}{b}$, али зато имамо $\frac{X}{b}$, и имамо s које означава успостављање везе кроз сваки од хопова. Имамо и $(k+1)^*d$, а то је пропатационо кашњење кроз сваки хоп. Код packet switching, нигде немамо $\frac{X}{b}$, али зато имамо $\frac{p}{b}$, а из то $\frac{p}{b}$ још имамо и -1 . Такође, имамо и $(k+1)^*d$, што нам је пропатационо кашњење првог пакета. То $(k+1)^*d$ нам се крати јер га имамо код оба.

Neka se x bitova korisničkih podataka predaju preko mreže tipa komutacija paketa (packet switching) koja na putu od izvorišta do odredišta ima k preskoka (hops). Podaci se predaju kao niz paketa pri čemu se svaki paket sastoji od p bitova podataka (payload) i h bitova zaglavlja (header), pri čemu važi da je $x \gg p+h$. Bitska brzina prenosa podataka po liniji iznosi b bitova po sekundi (odnosi se na sve čvorove u nizu), a propagaciono kašnjenje je zanemarljivo. Za koju vrednost p se dobija minimalno ukupne kašnjenje?

Postavka problema

Tražimo idealnu veličinu korisnih podataka u paketu (p) tako da ukupno kašnjenje bude minimalno.

- Ukupno podataka: x
- Veličina paketa: $p + h$ (podaci + zaglavlje)
- Broj paketa: $n = \frac{x}{p}$
- Broj skokova (linkova): k
- Brzina prenosa: b
- Propagacija: zanemarljiva (0)

1. Formula za ukupno kašnjenje (T)

Ukupno vreme je vreme potrebno da **svi** paketi napuste izvor, plus vreme koje je potrebno da **poslednji** paket prođe kroz preostale rutere do cilja.

1. **Vreme da svi paketi napuste izvor:** Svaki paket je veličine $p + h$, ima ih $\frac{x}{p}$, pa je ovo vreme: $\frac{x}{p} \cdot \frac{p+h}{b}$
2. **Vreme da poslednji paket prođe kroz preostalih $k - 1$ rutera:** Taj poslednji paket mora da se "pretoči" kroz još $k - 1$ čvorova, a svaki ga zadržava za vreme transmisije jednog paketa: $(k - 1) \cdot \frac{p+h}{b}$

Ukupna jednačina:

$$T = \frac{(x/p) \cdot (p + h)}{b} + \frac{(k - 1) \cdot (p + h)}{b}$$

$$T = \frac{1}{b} \left(x + \frac{xh}{p} + (k - 1)p + (k - 1)h \right)$$

2. Izvod i optimizacija (Pronalaženje minimuma)

Da bismo našli minimalno T , tražimo izvod funkcije po promenljivoj p i izjednačavamo ga sa nulom ($T'(p) = 0$):

$$\frac{dT}{dp} = \frac{1}{b} \left(-\frac{xh}{p^2} + (k - 1) \right) = 0$$

Iz ovoga sledi: $\frac{xh}{p^2} = k - 1 \qquad p^2 = \frac{xh}{k - 1}$

Konačno rešenje:

Minimalno ukupno kašnjenje dobija se za vrednost p :

$$p = \sqrt{\frac{xh}{k-1}}$$

Šta ovo znači?

Ova formula nam kaže da idealna veličina paketa zavisi od:

- **Količine podataka (x):** Više podataka traži veće pakete.
- **Veličine zaglavlja (h):** Što je zaglavlje "teže", paketi moraju biti veći da ne bismo trošili previše vremena na same heder-e.
- **Broja skokova (k):** Što je put duži (više rutera), paketi treba da budu manji kako bi brže krenuo "pipeline" efekat (dok prvi paket već stiže, drugi je odmah iza njega).

GSM je baziran na konceptu TDMA/FDD, a koristi 25 MHz za prenos u jednom smeru (forward link), pri čemu se taj opseg deli na radio kanale širine 200 kHz. Ako se 8 govornih kanala podržava od strane jednog radio kanala, pri čemu ne postoji zaštitni opseg za razdvajanje kanala, odrediti broj simultanih korisnika koji mogu istovremeno koristiti GSM.

$BW = 25 \text{ MHz}$ $BW_{rch} = 200 \text{ kHz}$ 1 rch = 8 govornih kanala $GB = 0$ Broj korisnika istovremeno?

1 Podaci iz zadatka

- Ukupan opseg za forward link: 25 MHz
- Širina jednog radio kanala: 200 kHz
- Broj govornih kanala po radio kanalu: 8
- Zaštitni opseg: nema (zanemarujemo)

Cilj: broj simultanih korisnika.

2 Broj radio kanala

Ukupan broj radio kanala N_{radio} se dobija kao:

$$N_{\text{radio}} = \frac{\text{ukupan opseg}}{\text{širina jednog kanala}} = \frac{25\,000 \text{ kHz}}{200 \text{ kHz}} = 125$$

Dakle, 125 radio kanala.

3 Broj simultanih korisnika po radio kanalu

Svaki radio kanal podržava 8 govornih kanala, znači:

$$N_{\text{user/channel}} = 8$$

4 Ukupan broj simultanih korisnika

Ukupan broj korisnika koji mogu simultano koristiti GSM:

$$N_{\text{total}} = N_{\text{radio}} \times N_{\text{user/channel}} = 125 \times 8 = 1000$$

✓ Odgovor:

1000 simultanih korisnika

Логично је: Укупан пренос је 25000000 Hz , а ширина једног канала је 200000 Hz . Број радио канала:

$$N_{\text{radio}} = \frac{25000000}{200000} = \frac{250}{2} = 125 \quad N_{\text{user/channel}} = 8 \quad \text{Онда је просто добијемо: } 125 * 8 = 1000.$$

$BW = 25000000 \text{ Hz}$ $BW_{\text{ch}} = 200000 \text{ Hz}$ $N_{\text{kanala}} = \frac{25000000}{200000} = 125$ $\text{Portimarno} \approx$ што веће могуће.
Ако је од стране 1 радио канала подржано 8 говорних канала, онда је од 125 радио канала заправо подржано 1000 говорних канала. Ајакле, решење задатка је 1000 симултаних корисника.

NOVA

LEKCIJA

Zadatak 33:

Na nivou data-link protokola koristi se sledeći način kodiranja podataka:

A:	0100 0111
B:	1110 0011
FLAG :	0111 1110
ESC:	1110 0000

Prikazati kako izgleda predajna bit sekvenca (u binarnom obliku) u slučaju kada okvir podataka (frame) čini niz od sledeća četiri karaktera: **A B ESC FLAG**. Kod predaje se koriste sledeće framing metode ili metode uokviravanja:

- 1) brojanje karaktera
- 2) flag bajtovi (start i end flag) sa bajt stuffing
- 3) start i end flag bajtovi sa bit stuffing-om

Okvir koji treba poslati je: A B ESC FLAG.

a)

1) Brojanje karaktera (Character Count)

- Ideja: prvi bajt (ili polje) u okviru govori koliko karaktera (bajta) sledi u okviru.
- Naš okvir ima 4 karaktera, dakle **count = 4** (pretpostavimo da je bajt).

Predajna sekvenca:

yaml

 Копирај ко

```
0000 0100  <- broj karaktera = 4
0100 0111  <- A
1110 0011  <- B
1110 0000  <- ESC
0111 1110  <- FLAG
```

- Nema potrebe za escape-ovanjem jer se koristi brojanje bajtova da primalac zna koliko bajtova čitati.

b)

2) Flag bajtovi sa byte stuffing-om (nastavak)

- Okvir podataka: A B ESC FLAG
- Flag = 0111 1110
- ESC = 1110 0000

Pravila byte-stuffing-a:

- Ako se u podacima pojavi FLAG, šalje se ESC + FLAG
- Ako se pojavi ESC, šalje se ESC + ESC

Predajna sekvenca:

```
0111 1110    <- start FLAG
0100 0111    <- A
1110 0011    <- B
1110 0000    <- ESC -> byte-stuffing -> 1110 0000 1110 0000
0111 1110    <- FLAG -> byte-stuffing -> 1110 0000 0111 1110
0111 1110    <- end FLAG
```

Dakle, konačna sekvenca (spojeno):

yaml

```
0111 1110 0100 0111 1110 0011 1110 0000 1110 0000 1110 0000 0111 1110 0111 1110
```

📌 Šta se koristi

- Start / End FLAG: 01111110
- Bit stuffing:
 - 👉 posle SVAKIH 5 uzastopnih jedinica (11111) u payload-u ubacuje se 0
- FLAG se NE dira (na njemu se ne radi bit stuffing)

1 Podaci u okviru (payload)

Redosled karaktera:

A B ESC FLAG

Njihovi bitovi:

- A = 01000111
- B = 11100011
- ESC = 11100000
- FLAG (kao podatak!) = 01111110

Spojimo payload bez flagova okvira: 01000111 11100011 11100000 01111110

Bez razmaka: 01000111111000111110000001111110

c)

c)

2 Bit stuffing – tražimo nizove 11111

Sada prolazimo bit po bit.

Payload:

```
01000111111000111110000001111110
```

Obeležimo gde ima 5 uzastopnih jedinica:

- 01000111111 → ✓ 5 jedinica → ubacuje se 0
- dalje: 00011111 → ✓ opet 5 jedinica → ubacuje se 0
- dalje u 01111110 (FLAG kao podatak) → opet 11111 → ubacuje se 0

3 Payload POSLE bit stuffing-a

Tačno ispravan rezultat je:

```
0100011111|0|100011111|0|00000011111|0|10
```

4 Konačna predajna bit sekvenca

Sada dodajemo start i end FLAG (oni se ne diraju):

```
01111110
```

```
0100011110100011110000001111010
```

```
01111110
```

Bez razmaka:

```
0111111001000111101000111100000011110100111110
```

✓ Konačna potvrda

- ✓ FLAG se ne dira
- ✓ Bit stuffing se radi samo nad payload-om
- ✓ Posle svakih 11111 → ubacuje se 0
- ✓ Tvoje razmišljanje i zapis sa |0| je 100% ispravan.



c)

За бројање карактера нема шта посебно да се каже. Видимо да нам је 1 карактер 1 бајт, тј. 8 битова. То значи да треба у форми 8 битова записати број карактера, а то је 4. Дакле: 0000 0100. Затим, да нагласимо да мора да се испостави следећи распоред: A B ESC FLAG. То је много важније нагласити, тј не сме да се погрешно. Дакле, једноставно не сме. Дакле, ово није бајт stuffing. Дакле, овде НЕМА почетног и крајњег флага. Коначно:

00000100 01000111 11100011 11100000 01111110

2) бајт stuffing Овде је важно одмах напоменути 2 ствари. Прва је та да код бајт stuffing УВЕК ИМАМО START FLAG и END FLAG. Вредност је увек иста: 01111110. Дакле, кад видимо ESC, мењамо га са ESC ESC, а кад видимо FLAG, мењамо га са ESC FLAG. ↓ зато је зато: A B ESC FLAG. Након што уредимо бајт stuffing добија се: FLAG A B ESC ESC ESC FLAG FLAG. После само заменимо вредности.

ВАЖНО!!! За бајт ступинг и за бит ступинг важи потпуно исто правило што се тиче овог правила за START FLAG и END FLAG. За бајт ступинг смо рекли како се ради, а за бит ступинг правило 5 јединица важи за сваку поворку битова појасособ.

Napisati sledeće nizove koristeći ASCII kodiranje: a) "SOS" b) "zdravo" c) "ETF-SA " Napisati konačne odgovore u heksadecimalnom kodu. Napomena: Smatrati da se svaki ASCII niz završava kontrolnim karakterom NUL (00h). Ukoliko se niz karaktera navede između dvostrukih znaka navoda onda imamo ASCII kodirane karaktere, po istom principu ako imamo karakter naveden između jednostrukih znaka navoda onda imamo ASCII kodirani karakter.

♦ ASCII tabela (delimično)

Karakter	ASCII (decimalno)	ASCII (heksadecimalno)
A	65	41
B	66	42
C	67	43
D	68	44
E	69	45
F	70	46
S	83	53
O	79	4F
z	122	7A
d	100	64
r	114	72
a	97	61

v	118	76
o	111	6F
E	69	45
T	84	54
F	70	46
-	45	2D
S	83	53
A	65	41
(space)	32	20
NUL	—	00

◆ a) "SOS"

Slova: S, O, S, NUL

👉 ASCII HEX kodovi:

r

53 4F 53 00

✓ Odgovor:

arduino

"SOS" → 53 4F 53 00

◆ b) "zdravo"

Slova: z, d, r, a, v, o, NUL

👉 ASCII HEX kodovi:

r

7A 64 72 61 76 6F 00

✓ Odgovor:

arduino

"zdravo" → 7A 64 72 61 76 6F 00

◆ c) "ETF-SA "

Slova: E, T, F, -, S, A, (space), NUL

👉 ASCII HEX kodovi:

mathematica

45 54 46 2D 53 41 20 00

✓ Odgovor:

mathematica

"ETF-SA " → 45 54 46 2D 53 41 20 00

Важна напомена: Када се записује у хексадецималном облику, стави се обавезно испред 0x, и нема наводника. Наравно, на испит се носи ASCII табела. Наравно да се не учи напамет.

Sledeći fragment podataka se javlja u okviru niza podataka koji treba da se preda A B ESC C ESC FLAG FLAG D..... Predajnik modifikuje fragment podataka koristeći framing metodu tipa bajt stuffing (umetanje bajtova). Pokazati kako će izgledati izlaz nakon primene bajt stuffing metode.

ZADATAK

Ulazni niz podataka je:

A B ESC C ESC FLAG FLAG D

Znači, u korisničkim podacima se pojavljuju kontrolni bajtovi:

- ESC — escape bajt (koristi se za označavanje da sledeći bajt ima posebno značenje),
- FLAG — označava početak i kraj frejma.

Problem je što se FLAG i ESC ne smeju pojaviti „prirodno“ u podacima, jer bi to zbunilo prijemnik (mislio bi da je kraj okvira).

Pravilo „Byte Stuffing“

Kada u korisničkim podacima naiđeš na:

- ESC → ubaci ispred još jedan ESC
- FLAG → zameni ga nizom ESC FLAG

Dakle, svaki put kad se pojavi ESC ili FLAG u podacima ↓ bacuje se dodatni ESC ispred.

◆ Korak po korak:

Ulaz	Izlaz nakon „byte stuffing”-a
A	A
B	B
ESC	ESC ESC
C	C
ESC	ESC ESC
FLAG	ESC FLAG
FLAG	ESC FLAG
D	D 

✓ **Konačni rezultat (izlazni niz):**

A B ESC ESC C ESC ESC ESC FLAG ESC FLAG D

📦 **Ako se dodaju start i end flag (što se obično radi kod celog frejma):**

Kompletan okvir bi izgledao ovako:

FLAG A B ESC ESC C ESC ESC ESC FLAG ESC FLAG D FLAG

Sto se tice start i end flega, za njih znamo da se inace po standardu stavljaju, ali u ovom zadatku to nije naglaseno pa se ne stavlja. Treba valjda samo kad je bas naglaseno da se stave, inace ne. Ovde recimo nije naglaseno, pa ne treba.

Niz podataka: 011110111110111110 treba poslati preko data link nivoa. Kakav će biti oblik stvarnog niza na izlazu predajnika nakon primene framing metode bit stuffing? Uraditi destuffing ukoliko na stranu odredišta stigne: 011110111110111110.

Prvo podsetnik pravila bit-stuffinga (bit-framing):

- Start/end FLAG = 01111110 (ne menja se).
- U podatkovnom delu: svaki put kada uočiš 5 uzastopnih jedinica (11111), predajnik ubacuje jednu nulu (0) odmah posle tih pet 1 .
- Na prijemu se ta ubačena 0 uklanja (destuffing): ako posle pet 1 sledi 0 , prijemnik taj 0 izbací.

1) Stuffing za dati podatak

Ulazni podatak:

D = 01111011111101111110

Primenićemo bit-stuffing: ubaci 0 posle svake pojave 11111 .

Rezultat stuffing-a (podatak posle umetanja nula):

D_stuffed = 01111011111100111111010

(produžina sa 19 → 21 bita)



Ako okvir dodajemo FLAG-ovima (standardno), kompletan niz koji ide na kanal je:

```
FRAME = FLAG  D_stuffed  FLAG  
= 01111110 011110111110011111010 01111110
```

2) Destuffing za stigli niz

Stigla sekvenca (data-deo koji si dao za destuffing):

```
R = 011110111110111110
```

Primenom pravila destuffing (ukloni 0 koja odmah sledi pet 1):

- Skidamo svaki umetnuti 0 koji sledi pet 1.

Rezultat destuffing-a (podatak na strani odredišta):

```
R_destuffed = 0111101111111111
```

(dužina 18 → 16 bita)

Kratko objašnjenje razlike

- Originalni podatak koji si tražio da pošalješ bio je `011110111110111110`. Nakon pravilnog stuffing-a dobili smo `011110111110011111010`.
- Niz koji je stigao `011110111110111110` nije identičan tom pravilno poslanom stuffed nizu — zato je i destuffed rezultat (što prijemnik izvlači) različit od originala. Destuffing uklanja samo `0` koje slede pet `1`; ako je u prenosu došlo do greške (npr. bit izgubljen ili dodat), destuffed rezultat neće nužno vratiti original.

Sažetak (koncizno)

- Stuffed poslato (data-deo): `011110111110011111010`
- Poslato u okviru sa flagovima: `01111110 011110111110011111010 01111110`
- Stiglo (dato za destuff): `011110111110111110`
- Nakon destuffinga: `0111101111111111`

011110111110111110 Ово је низ података. Након што урадимо bit stuffing добијамо:

011110111110011111010 **ВАЖНО!!!** Ово се НЕ шаље на пријемник. Фале флегови.

0111110, 011101111110011111010 0111110 Ово се шаље на пријемник. Има флегова.
Start flag End flag

Други део задатка је destuffing за: 011110111110111110. Изгледа то овако:
01111011111111. Само је потребно обрисати нуле које се јављају после 5 јединица.

Objasniti funkciju i princip rada kodova za proveru parnosti.

◆ Funkcija kodova za proveru parnosti

Kodovi za proveru parnosti (Parity Check Codes) služe za otkrivanje grešaka u prenosu podataka.

- Kada se podaci šalju digitalnim kanalima, može doći do greške u jednom ili više bitova (npr. bita koji se promenio sa 0 na 1 ili obrnuto).
- Parity bit je dodatni bit koji se dodaje na kraju svakog bajta ili niza bita kako bi se mogla proveriti integritet podataka.

Funkcija:

- Otkriva greške u prenesenim podacima (najčešće jednobitne greške).
- Omogućava predajniku i prijemniku da se dogovore da li je podatak ispravan ili je došlo do greške.

◆ Princip rada

Postoje dve osnovne vrste: parnost parna (even parity) i parnost neparna (odd parity).

1 Parna parnost (Even Parity)

- Zbir svih bita (uključujući parity bit) treba da bude paran (deljiv sa 2).
- Ako bajt već ima paran broj jedinica, parity bit je 0.
- Ako bajt ima neparan broj jedinica, parity bit je 1 da bi zbir bio paran.

Primer:

Bajt: 1011001 → ima 4 jedinice (parno)

- Parity bit: 0
- Poslati: 10110010

Bajt: 1011010 → ima 3 jedinice (neparno)

- Parity bit: 1
- Poslati: 10110101

2 Neparna parnost (Odd Parity)

- Zbir svih bita (uključujući parity bit) treba da bude **neparan**.
- Ako bajt ima neparan broj jedinica, parity bit je 0.
- Ako bajt ima paran broj jedinica, parity bit je 1.

Primer:

Bajt: 1011001 → ima 4 jedinice (parno)

- Parity bit: 1
- Poslati: 10110011

◆ Kako se proverava parnost

1. Prijemnik broji broj jedinica u primljenom bajtu.
2. Ako broj jedinica ne odgovara dogovorenoj parnosti (parna/neparna), prijemnik zna da je došlo do greške.
3. Kodovi parnosti ne ispravljaju greške, samo ih detektuju.

💡 Ograničenja:

- Mogu detektovati samo **neparan broj grešaka** (jedan ili tri bita, itd.).
- Ne mogu detektovati kada se promeni **paran broj bitova**.

Функција кодова за проверу парности - кодови за проверу парности служе за откривање грешака у преносу података. Када се подаци шаљу дигиталним каналима, може доћи до грешке у једном или више бита. Парити бит је бит који се додаје на крај сваког бјита или низа бита како би се могло проверити интегритет података. Открива грешке у пренесеним подацима (најчешће једнобитне грешке). Штопуњава предајник и пријемник да се договоре да ли је податак исправан или је дошло до грешке. Постоје две основне врсте: парност парна и парност непарна. И сада је ово веома важно: збир свих бита треба да буде паран (дефинисано са 2). То значи да, ако је на пример, број 1 паран, онда је parity bit једнак нула. Ако је број 1 непаран, онда је parity bit једнак један. Све ово што сам написао важи за парну парност. За непарну парност важи следеће: ако је број 1 непаран, онда је parity bit једнак 0, а ако је број 1 паран, онда је parity bit једнак 1. Кодови за проверу парности не исправљају грешке, само их детектују. Пријемник пребројава број јединица у примљеном бјиту, и ако тај број не одговара договореној парности (парна/непарна) онда јавља грешку. Кодови за проверу парности могу детектовати непаран број грешака, односно непаран број промена 1 и 0 и обрнуто. Паран број грешака не могу детектовати.

Када се шаље поворка бита од предајника ка пријемнику, може доћи до грешке, а смисао да се претвори 0 у 1 и обрнуто (такве грешке на путу од предајника ка пријемнику).

Постоји парна парност и непарна парност. Постоји и бит парности, ево од чега зависи: ако нам треба парна парност, и број 1 у поворци бита је иначе паран, бит парности је 0 а ако је број 1 непаран, бит парности је 1. Иста логика је и за непарну парност.

Кодови парности не исправљају грешке, само их детектују.

Могу детектовати само непаран број грешака (паран број грешака не).

И за LRC, и за VRC, и за кодове за проверу парности, и за кораке које прави предајник док формира чексуми за слање пријемнику, и за пријемник чексуме, увек је мера 1 бајт.

Na koji način se formira LRC (Longitudinal Redundancy Check)?

◆ LRC (Longitudinal Redundancy Check)

Cilj:

Detektuje greške u okviru podataka (frame) tako što koristi dodatni **redundantni bajt** na kraju okvira.

1 Kako se formira:

1. Pretpostavimo da imamo okvir podataka koji se sastoji od više bajtova:

Bajt1, Bajt2, Bajt3, ..., Bajtn

2. LRC bajt se formira tako što se izvrši **bit-ekskluzivno-ILI (XOR)** po koloni svih bajtova.

- Zamislimo da svaki bajt pišemo u kolone po bitovima:

makefile

Bajt1: b11 b12 b13 b14 b15 b16 b17 b18

Bajt2: b21 b22 b23 b24 b25 b26 b27 b28

...

Bajtn: bn1 bn2 bn3 bn4 bn5 bn6 bn7 bn8

- LRC bajt se računa kao XOR svih bajtova po kol



- LRC bajt se računa kao XOR svih bajtova po koloni.

To znači da je svaki bit LRC-a XOR svih bitova iz iste kolone:

vbnet

 Copy code

$$\text{LRC} = (b_{11} \text{ XOR } b_{21} \text{ XOR } \dots \text{ XOR } b_{n1}, b_{12} \text{ XOR } b_{22} \text{ XOR } \dots \text{ XOR } b_{n2}, \dots, b_{18} \text{ XOR } b_{28} \text{ XOR } \dots \text{ XOR } b_{n8})$$

2 Šta to znači praktično:

- Ako se pojavi greška u jednom bitu u okviru, LRC će to detektovati jer će XOR biti drugačiji od originalnog LRC-a.
- LRC se dodaje na kraj okvira i primač računa XOR svih primljenih bajtova (uključujući LRC). Ako je rezultat 0, okvir je bez grešaka.

3 Zaključak:

- LRC je jedan bajt, računa se XOR svih bajtova okvira.
- Dovoljno je jednostavan, ali može detektovati samo neke tipove grešaka (nije toliko robustan kao CRC).

LRC детектује грешке у оквиру података тако што користи додатни редуцирани бајт на крају оквира. Претпоставимо да имамо оквир података који се састоји од више бајтова: Бајт1, Бајт2, Бајт3..., Бајт n .

LRC бајт се формира тако што рачунамо XOR по колони свих бајтова. Ако се појави грешка у једном биту у оквиру, LRC ће то детектовати јер ће XOR бити другачији од оригиналног LRC-а. LRC се додаје на крају оквира и прималач рачуна XOR свих примљених бајтова (укључујући LRC). Ако је резултат 0, оквир је без грешака. LRC је 1 бајт, рачуна се XOR свих бајтова оквира.

И за LRC, и за VRC, и за кодове за проверу парности, и за кораке које прави предајник и одр формира чексуми за слабе пријемнике, и за пријемник чексуме, увек је мера 1 бајт.

И LRC, и VRC, и чексум, и бит парности, све се то ставља на крају поруке.

Pronaci LRC za niz bitova 11100111 11011101 00111001 10101001

Podaci:

Okvir se sastoji od 4 bajta:

ini

B1 = 11100111

B2 = 11011101

B3 = 00111001

B4 = 10101001

Cilj: formirati LRC bajt kao XOR svih bajtova.

B1 XOR B2

11100111

XOR 11011101

00111010

- Rezultat XOR-ujemo sa trećim bajtom:

00111010

XOR 00111001

00000011

- Rezultat XOR-ujemo sa četvrtim bajtom:

00000011

XOR 10101001

10101010



2

Konačni LRC bajt

ini

LRC = 10101010

LRC bajt se dodaje na kraj okvira. Primalac izvršava XOR svih primljenih bajtova (uključujući LRC). Ako je rezultat 00000000, okvir je bez greške.

Na koji način se formira VRC (Vertical Redundancy Check)?

1 Osnovna ideja

VRC je paritetni bit koji se dodaje svakom bajtu podataka.

Cilj: da se otkriju greške pri prenosu podataka.

- Ako se koristi **paritetna prosta (even parity)**: broj jedinica u bajtu + VRC mora biti **paran**.
- Ako se koristi **paritetna neparna (odd parity)**: broj jedinica + VRC mora biti **neparan**.

2 Kako se formira

1. Pogledaj bajt podataka (8 bita).
2. Izbroj koliko ima jedinica.
3. Odaberi tip pariteta (even/odd).
4. Dodaj VRC bit tako da ukupan broj jedinica bude paran (ili neparan).

Primer:

Bajt: 10110010

- Broj jedinica = 4
- Ako koristimo **even parity**, VRC = 0 (jer je već paran broj jedinica).
- Ako koristimo **odd parity**, VRC = 1 (da broj jedinica + VRC postane neparan).

3 Za više bajtova

VRC se računa po bajtu, svaki bajt dobija svoj VRC bit.

Upravo zbog toga se zove *vertical* — ako zamislimo bajtove jedan ispod drugog, VRC čini "vertikalni" dodatak.

VRC је паритетни бит који се додаје сваком байту податка. Циљ: да се открију грешке при преносу податка. Ако се користи паритетна парна: број јединица у байту + VRC мора бити паран. Ако се користи паритетна непарна: број јединица + VRC мора бити непаран. Јасно је из контекста како се формира, да не морам то сва посебно да објашњавам. Ако има више бајтова, онда се за сваки посебно одређује VRC.

И за LRC, и за VRC, и за кодове за проверу парности, и за кораке које прави предајник и одк формира чексуму за слање пријемнику, и за пријемник чексуме, увек је мера 1 бајт.

И LRC, и VRC, и чексум, и бит парности, све се то ставља на крај поруке.

Pronaci VRC za 11100111 11011101 00111001 10101001

1 Prvi bajt: 11100111

- Broj jedinica: $1+1+1+0+0+1+1+1 = 6$
- 6 je paran $\rightarrow$ VRC = 0

2 Drugi bajt: 11011101

- Broj jedinica: $1+1+0+1+1+1+0+1 = 6$
- 6 je paran $\rightarrow$ VRC = 0

3 Treći bajt: 00111001

- Broj jedinica: $0+0+1+1+1+0+0+1 = 4$
- 4 je paran $\rightarrow$ VRC = 0

4 Četvrti bajt: 10101001

- Broj jedinica: $1+0+1+0+1+0+0+1 = 4$
- 4 je paran $\rightarrow$ VRC = 0

✓ Rezultat

Dodajući VRC bit na kraj svakog bajta:

11100111 0

11011101 0

00111001 0

10101001 0

Za početnu povorku bitova 11100111 11011101 00111001 10101001 pokazati kako izgleda početna povorka plus LRC.

Dati bajtovi:

ini

$B1 = 1110\ 0111\ (E7h)$

$B2 = 1101\ 1101\ (DDh)$

$B3 = 0011\ 1001\ (39h)$

$B4 = 1010\ 1001\ (A9h)$

LRC se dobije XOR-om svih bajtova (bit-po-bit):

1. $B1 \oplus B2$:

vbnet

11100111

XOR 11011101

= 00111010

2. Rezultat $\oplus B3$:

vbnet

00111010

XOR 00111001

= 00000011

3. Rezultat $\oplus B4$:

vbnet

00000011

XOR 10101001

= 10101010

Dakle LRC = 10101010 (0xAA).

Konačna početna povorka sa LRC (u binarnom obliku):

```
11100111 11011101 00111001 10101001 10101010
```

I u heksadecimalnom zapisu:

CSS

E7 DD 39 A9 AA

LRC детектује грешke у оквиру податка тако што користи додатни редуцирани бајт на крају оквира. Претпоставимо да имамо оквир податка који се састоји од више бајтова: $\text{Bajt}_1, \text{Bajt}_2, \text{Bajt}_3 \dots, \text{Bajt}_n$.

LRC бајт се формира тако што рачунамо **XOR** по колони свих бајтова. Ако се појави грешка у једном биту у оквиру, **LRC** ће то детектовати јер ће **XOR** бити другачији од оригиналног **LRC**-а. **LRC** се додаје на крају оквира и представља рачуна **XOR** свих припадних бајтова (укључујући **LRC**). Ако је резултат 0, оквир је без грешака. **LRC** је 1 бајт, рачуна се **XOR** свих бајтова оквира.

VRC је паритетни бит који се додаје сваком бајту податка. Циљ: да се открију грешке при преносу податка. Ако се користи паритетна парност: број јединица у бајту + **VRC** мора бити паран. Ако се користи паритетна непарност: број јединица + **VRC** мора бити непаран. Јасно је из контекста како се формира, да не морам то сад посебно да објашњавам. Ако има више бајтова, онда се за сваки посебно одређује **VRC**.

И за **LRC**, и за **VRC**, и за кодове за проверу парности, и за кораке које прави предајник и одк формира чексуми за слање пријемнику, и за пријемник чексуме, увек је мера 1 бајт.

И **LRC**, и **VRC**, и чексум, и бит парности, све се то ставља на крају поруке.

Zadatak 39:

Neka je zadana dvodimenzionalna parity-check matrica:

0	0	0	0
1	1	1	1
0	1	0	1
1	0	1	0

- Pokazati da dvodimenzionalna parity-check matrica može da detektuje i koriguje grešku na jednoj bit poziciji. Usvojiti da se greška javila na poziciji preseka vrste 2 i kolone 3.
- Pokazati izabranim primerom da se dvostruka bit greška može detektovati, ali ne i korigovati.

◆ ZADATAK

Data je 4×4 matrica bitova:

	1	2	3	4
1	0	0	0	0
2	1	1	1	1
3	0	1	0	1
4	1	0	1	0

Na osnovu nje formiraju se:

- **parity bitovi po vrstama** (horizontalni paritet),
- **parity bitovi po kolonama** (vertikalni paritet).

Ovi dodatni bitovi formiraju „okvir“ oko tabele koji omogućava otkrivanje i ispravljanje grešaka.

a) Detekcija i korekcija jedne greške

Pretpostavimo da se greška desila u vrsti 2, koloni 3.

Dakle, originalni bit je bio 1, ali se promenio u 0.

Nova matrica izgleda ovako:

	1	2	3	4
1	0	0	0	0
2	1	1	0	1
3	0	1	0	1
4	1	0	1	0

1 Izračunajmo paritet po vrstama:

- red 1: $0+0+0+0 = 0 \rightarrow$ paritet = 0
- red 2: $1+1+0+1 = 3$ (neparno) $\rightarrow$ paritet = 1
- red 3: $0+1+0+1 = 2$ (parno) $\rightarrow$ paritet = 0
- red 4: $1+0+1+0 = 2$ (parno) $\rightarrow$ paritet = 0



Dakle, paritet po vrstama daje:

0 1 0 0

Greška je u drugom redu jer ima neparan zbir.

2 Izračunajmo paritet po kolonama:

- kolona 1: $0+1+0+1 = 2 \rightarrow$ paritet = 0
- kolona 2: $0+1+1+0 = 2 \rightarrow$ paritet = 0
- kolona 3: $0+0+0+1 = 1 \rightarrow$ paritet = 1
- kolona 4: $0+1+1+0 = 2 \rightarrow$ paritet = 0

Dakle, paritet po kolonama daje:

0 0 1 0

Greška je u trećoj koloni jer ima neparan zbir.

✓ Zaključak:

Presek *druge vrste* i *treće kolone* → tačno mesto greške.

Taj bit se invertuje ($0 \rightarrow 1$), čime se originalna matrica vraća u ispravno stanje.

👉 Dakle, 2D parity-check **detektuje i koriguje jednu grešku** jer se tačno zna gde se greška desila.

Цела поента приче је да код парности може детектовати (али не и исправити) непаран број погрешних бита, а паран број грешака не може ни детектовати, јер је нешто стварно грешка а нешто лош пресек. Значи, код парног броја грешака бит парности остаје исти, а код непарног броја грешака се мења.

1 Početna matrica

Originalna 4×4 matrica podataka:

R1: 0 0 0 0

R2: 1 1 1 1

R3: 0 1 0 1

R4: 1 0 1 0

Svaka vrsta i svaka kolona ima parnu parnost
(broj jedinica je paran, ako računamo parnu parnost).

2 Ubacimo dve greške

Recimo da se dogodi dvostruka greška:

- Bit na R2,C2 promeni vrednost iz 1 → 0
- Bit na R3,C4 promeni vrednost iz 1 → 0

Nova matrica sada izgleda ovako:

R1: 0 0 0 0

R2: 1 0 1 1 <- greška u C2

R3: 0 1 0 0 <- greška u C4

R4: 1 0 1 0

3 Šta kažu pariteti

- **Paritet vrste:**
 - R1: $0+0+0+0 = 0 \rightarrow$ paran $\rightarrow$ ok
 - R2: $1+0+1+1 = 3 \rightarrow$ neparan $\rightarrow$ greška detektovana
 - R3: $0+1+0+0 = 1 \rightarrow$ neparan $\rightarrow$ greška detektovana
 - R4: $1+0+1+0 = 2 \rightarrow$ paran $\rightarrow$ ok
- **Paritet kolone:**
 - C1: $0+1+0+1 = 2 \rightarrow$ paran $\rightarrow$ ok
 - C2: $0+0+1+0 = 1 \rightarrow$ neparan $\rightarrow$ greška detektovana
 - C3: $0+1+0+1 = 2 \rightarrow$ paran $\rightarrow$ ok
 - C4: $0+1+0+0 = 1 \rightarrow$ neparan $\rightarrow$ greška detektovana

4 Zašto se ne može ispraviti

Dvodimenzionalna parnost locira grešku tako što gleda presek vrste i kolone gde je parnost narušena.

- R2 i R3 imaju grešku
- C2 i C4 imaju grešku

Presek pariteta pokazuje četiri moguće lokacije:

- R2,C2 ✓ (prava greška)
- R2,C4 ✗ (nepravilan presek)
- R3,C2 ✗ (nepravilan presek)
- R3,C4 ✓ (prava greška)

Dakle, nema načina da tačno odredimo koja dva bita su pogrešna ako postoje dve greške. Možemo samo da detektujemo da nešto nije u redu, ali ne možemo da ispravimo tačno te dve greške.

Opisati koje korake obavlja predajnik kod formiranja čeksume.

◆ 1. Podela podataka na bajtove

Podaci koji se šalju dele se na blokove (npr. 8-bitne bajtove) ili reči (16-bitne, 32-bitne — zavisno od protokola).

👉 Primer:

```
11100111 11011101 00111001 10101001
```

📄 Copy

◆ 2. Izračunavanje sume

Predajnik zatim sve te bajtove **sabira** (ili ih **XOR-uje**, kod LRC-a) prema pravilima protokola.

Dve najčešće varijante:

- **Čeksum (Checksum):** koristi **binarno sabiranje** modulo 2^n (n je broj bitova, npr. 8).
- **LRC (Longitudinal Redundancy Check):** koristi **XOR** operaciju bit po bit.

◆ 3. Uzimanje komplementa (kod čeksuma)

Ako se koristi klasična čeksuma, onda se na kraju:

- rezultat sabiranja invertuje (1's complement)
→ čime se dobija čeksum.

Cilj je da **suma svih bajtova + čeksum = 11111111 (ili 00000000)**, što znači da nema greške.

Kod LRC-a, ovaj korak se ne radi — jer se koristi XOR.

◆ 4. Dodavanje čeksume poruci

Predajnik potom dodaje izračunatu čeksumu na kraj poruke, kao dodatni bajt.

👉 Primer:

```
makefile
```

```
Podaci: 11100111 11011101 00111001 10101001
```

```
LRC:      10101010
```

```
Ukupno:  11100111 11011101 00111001 10101001 10101010
```

Na kraju, čitava sekvenca (podaci + čeksum) se šalje prijemniku.

Веома је важно разграничити неке ствари. Дакле, LRC је за LRC, VRC је за VRC,
а чексума се везује само и искључиво за сабирање **бајтова**. То је важно нагласити. Дакле,
имамо низ бита, поделимо га на бајтове, и онда бајтове класично сабирамо (не бит по бит).
Дакле, ово је суштина одговора на питање о предајници и чексуми. Низ бита који имамо требамо
поделити на бајтове, потом те бајтове саборати, добијени резултат инвертовати и ставити на крај.

Opisati koje korake obavlja prijemnik kod provere čeksume.

◆ Koraci koje obavlja prijemnik kod provere čeksume:

1 Prijem kompletne poruke

Prijemnik primi sve bajtove koje je poslao predajnik — uključujući poslednji bajt koji predstavlja čeksumu (LRC ili Checksum).

👉 Primer:

```
11100111 11011101 00111001 10101001 10101010
```

2 Ponovno sabiranje (ili XOR-ovanje) svih bajtova

Prijemnik uzima sve primljene bajtove, uključujući čeksum, i ponovo ih sabira (ili XOR-uje) istim postupkom koji koristi i predajnik.

3 Provera rezultata

- Ako se koristi klasična čeksuma (sabiranje) → rezultat treba da bude:

SCSS

11111111 (ili 00000000)

u zavisnosti od konvencije.

- Ako se koristi LRC (XOR) → rezultat treba da bude:

00000000

U slučaju greške: prijemnik zahteva retransmisiju (ponovno slanje poruke), ili odbacuje podatke, u zavisnosti od protokola (npr. ARQ mehanizmi).

4 Zaključak

- ✓ Ako rezultat ima sve nule (ili sve jedinice) → nema greške u prenosu.
- ✗ Ako rezultat nije savršen (ima različite bitove) → postoji greška u prenosu.

Кораци које обавља пријемник код провере чексума: пријем резултата, односно комплетне поруке. Он
дакле прими све бајтове које је послао предајник, укључујући и последњи бајт који представља чексум.
Све те бајтове поново сабира (или XOR-ује) потпуно истим поступком као и предајник што је урадио.
Затим се ради провера резултата. За класичан чексум треба да буде све 1 или све 0. То зависи
од конвенције. Последњи корак је закључак да ли има или нема грешке у преносу.

Neka se sledeći blok od 16 bitova predaje preko serijske linije koristeći čeksumu od 8 bitova 10101001 00111001. Pokazati kako izgleda predajna povorka bitova.

◆ Zadatak

Prenosi se blok od 16 bitova:

```
10101001 00111001
```

Koristi se čeksuma od 8 bitova, što znači da ćemo:

- podeliti podatke u dve 8-bitne reči
- sabrati ih kao binarne brojeve
- invertovati rezultat (1's complement)
- i tu vrednost dodati kao čeksumu na kraj poruke.

◆ Korak 1: Podeli podatke

```
nginx
```

Prva reč: 10101001 (A9h = 169 decimalno)

Druga reč: 00111001 (39h = 57 decimalno)

◆ Korak 2: Saberi reči

markdown

```
  10101001
+ 00111001
-----
  11100010
```

Nema prenosa (carry) iznad 8 bita, tako da rezultat ostaje:

ini

```
Suma = 11100010
```

◆ Korak 3: Inverzija (1's complement)

Čeksuma je negacija sume, dakle:

makefile

```
Suma:      11100010
Čeksuma:   00011101
```



◆ Korak 4: Predajna povorka bitova

U predajnoj sekvenci šalje se:

CSS

```
[Podaci][Čeksuma]
```

Dakle:

```
10101001 00111001 00011101
```

✓ Konačan odgovor:

Predajna povorka bitova (sa 8-bitnom čeksumom) glasi:
101010010011100100011101

Веома је важно разграничити неке ствари. Дакле, LRC је за LRC, VRC је за VRC, а чексума се везује само и искључиво за сабирање **бајтова**. То је важно нагласити. Дакле, имамо низ бита, поделимо га на бајтове, и онда бајтове класично сабирамо (не бит по бит). Дакле, ово је суштина одговора на питање о предајнику и чексуми. Низ бита који имамо требамо поделити на бајтове, потом те бајтове саборати, добијени резултат инвертовати и ставити на крај.

$$\begin{array}{r}
 1111 \\
 10101001 \\
 + 00111001 \\
 \hline
 11100010
 \end{array}$$
 За чексуму се ради класично сабирање, с тим што овде нема overflow. Резултат: 10101001 00111001 ~~11100010~~. Увек се додаје чексума на крају.

ВАЖНО!!! Увек је потребно урадити комплемент резултата добијеног сабирањем.

Informacioni sadržaj paketa ima sledeći bit oblik 1010 1010 1010 1011 a za prenos se koristi šema parne parnosti. Kakva će biti vrednost checksum polja za slučaj da se koristi šema dvodimenzionalne parnosti? Checksum polje treba da je minimalne dužine.

Zadatak:

- Informacioni paket (16 bitova):

yaml

```
1010 1010 1010 1011
```

- Šema: dvodimenzionalna parnost (2D parnost), parnost parna.
- Cilj: odrediti minimalno checksum polje.

1 Podela na bajtove

Dvodimenzionalna parnost radi tako što se paket deli u redove (obično po 8 bitova):

yaml

```
Red 1: 10101010
```

```
Red 2: 10101011
```

2 Dodavanje horizontalne parnosti (po redu)

- Parna parnost → broj 1 u svakom redu + paritetski bit mora biti paran.

Red 1: 10101010

- Broj 1: 4 → već paran
- Paritetski bit = 0 (ne menja parnost)
- Red sa paritetom: 10101010 0 → 9 bitova

Red 2: 10101011

- Broj 1: 5 → neparan
- Paritetski bit = 1 (da bi broj 1 bio paran)
- Red sa paritetom: 10101011 1 → 9 bitova

3 Dodavanje vertikalne parnosti (po kolonama)

- Sada imamo 2 reda x 9 kolona:

1	0	1	0	1	0	1	0	0
1	0	1	0	1	0	1	1	1

- Broj 1 u svakoj koloni:
 1. kolona: $1+1 = 2 \rightarrow$ paran $\rightarrow$ vertikalni paritetski bit = 0
 2. kolona: $0+0 = 0 \rightarrow$ paran $\rightarrow$ bit = 0
 3. kolona: $1+1 = 2 \rightarrow 0$
 4. kolona: $0+0 = 0 \rightarrow 0$
 5. kolona: $1+1 = 2 \rightarrow 0$
 6. kolona: $0+0 = 0 \rightarrow 0$
 7. kolona: $1+1 = 2 \rightarrow 0$
 8. kolona: $0+1 = 1 \rightarrow$ neparan $\rightarrow$ vertikalni bit = 1
 9. kolona: $0+1 = 1 \rightarrow$ neparan $\rightarrow$ vertikalni bit = 1

4 Checksum polje (minimalno)

- Minimalna veličina checksum polja = dodatni red sa vertikalnim paritetom:

yaml

Vertikalni paritet: 0 0 0 0 0 0 0 1 1

- Tako da checksum polje = 9 bitova.

Користи се парна парност. То значи да ако имамо паран број јединица, бит парности је 0, а ако је непаран број јединица, бит парности је 1. Овај информациони податак је потребно поделити на бајтове. Управо је то оно о чему смо причали. Увек се све дели на бајтове. Значи, увек се све ово мора пањачво читати. Видимо да нам се спомиње и дводимензионална парност. То значи да и по колонама и по врстама тражимо бит парности. Дакле, подела на бајтове, одређивање бита парности и за врсте и за колоне (али прво за врсте) и онда на крају одређивање минималног checksum порба. У овом задатку конкретно имамо: 1010 1010 1010 1011. Када то поделимо на бајтове, добијемо ово: Бајт 1: 10101010. Бајт 2: 1010 1011. И сад, рачунамо бит парности по врстама за почетак: 10101010 → Бит парности је 0
10101011 → Бит парности је 1 } Ово је по врстама, и важно је рећи да се увек прво то ради.

1 0 1 0 1 0 1 0 | 0
1 0 1 0 1 0 1 1 | 1
0 0 0 0 0 0 0 1 | 1 } Ово је по колонама, дакле битови парности по колонама. Ово треће су придодати битови парности по врстама. Када кажем ово треће, ту се мисли на ово заокружено.

ВАЖНА НАПОМЕНА!!! Ако се у задатку спомиње парност (небитно да ли парна или непарна) и још ако се повезује са чексумом, онда се никакво инвертовање не ради. Одређују се битови парности, али инвертовања нема. Инвертовање се ради у случајевима кад се парност не спомиње. И само тада. Кад се парност не спомиње, једноставно се сабирају бајтови и резултат се **ОБАВЕЗНО** мора инвертовати. Ово су важна правила која се морају имати у виду да не би засрао на испиту.

LRC, VRC, чексум, све се то додаје на крају поруке. То је важно у решавању задатка.

Sistem koristi LRC nad blokom od osam bajtova pri čemu svaku vrstu čini osam bitova. Koliko se redundantnih bitova predaje po bloku? Koliki je odnos između korisnih bitova i ukupnog broja bitova?

1 Dati podaci

- Sistem koristi LRC (Longitudinal Redundancy Check).
- Blok: 8 bajtova, svaki bajt = 8 bita.
- Dakle, blok = $8 \times 8 = 64$ bita korisnih podataka.
- Svaki red = 1 bajt = 8 bita.

2 Šta je LRC?

- LRC = paritetski bajt koji se dobija tako što se izvrši XOR svih bajtova u bloku vertikalno po kolonama (svaka kolona od 8 bita dobija jedan paritetski bit).
- Rezultat: 1 dodatni bajt pariteta po bloku.

3 Redundantni bitovi

- LRC dodaje 1 bajt = 8 bitova po bloku.
- Dakle:

java

Redundantni bitovi = 8 bita

4 Ukupni broj bitova u bloku sa LRC

- Korisni bitovi = 64 bita
- Redundantni bitovi = 8 bita

java

Ukupno bitova = 64 + 8 = 72 bita

5 Odnos korisnih i ukupnih bitova

java

Korisni bitovi / Ukupno bitova = 64 / 72 = 8 / 9 $\approx$ 0.8889

- Odnos $\approx$ 88.89%

Zadatak 45:

Informaciona poruka koju treba poslati je oblika 10100101, a generator polinom je oblika: $G = X^5 + X^2 + 1$

a) Odrediti celokupni sadržaj poruke koju treba poslati liniji uključujući informacione bitove plus CRC bitove koji se koriste radi provere grešaka u prenosu.

b) Ako se na prijemnoj strani primi poruka čiji je oblik 1010010101110, a generator polinom je isti kao i pod stavkom a) odrediti da li je došlo do greške u prenosu.

a)

1 Dodavanje mesta za CRC

- Generator polinom = 6 bitova $\rightarrow$ CRC = 5 bitova
- Informaciona poruka = `10100101` $\rightarrow$ dodajemo 5 nula na kraj:

`10100101 00000`

2 Deljenje modulo 2 (XOR)

- Delimo `1010010100000` sa generatorom `100101`
- Izvodimo XOR po bitovima sve dok ne dobijemo ostatak od 5 bita

(Ovde ću odmah dati rezultat jer ručno može biti dugačko za pisanje)

- Ostatak (CRC) = `01110`

3 Poruka za slanje

- Dodajemo CRC na originalnu poruku:

`10100101 01110`

✓ Poruka koja se šalje = `1010010101110`

b) Provera na prijemnoj strani

- Primljena poruka: `1010010101110`
- Delimo je istim generatorom `100101`

1 Deljenje modulo 2

- Ostatak = 0

2 Zaključak

- Ostatak = 0 $\rightarrow$ greška nije detektovana
- Dakle, poruka je preneti ispravno

✓ Rezime:

a) Poruka za slanje = `1010010101110`

b) Primljena poruka `1010010101110` $\rightarrow$ nema greške

$$\begin{array}{r}
 10100101000000 : 100101 = \\
 \oplus 100101 \downarrow \downarrow \\
 \cancel{00}110001 \\
 \oplus 100101 \downarrow \\
 \cancel{0}101000 \\
 \oplus 100101 \downarrow \downarrow \\
 \cancel{00}110100 \\
 \oplus 100101 \downarrow \\
 \cancel{0}100010 \\
 \oplus 100101 \downarrow \\
 \cancel{00}01110
 \end{array}$$

CRC KOD

2) inf + CRC info = 10100101

$$G = X^5 + X^2 + 1 = 1 \cdot X^5 + 0 \cdot X^4 + 0 \cdot X^3 + 1 \cdot X^2 + 0 \cdot X^1 + 1 \cdot X^0$$

$$G = 100101$$

NOVA

LEKCIJA

!!!!!!

- a) Odrediti Hamming-ov kôd za informacione bitove $d_8d_7d_6d_5d_4d_3d_2d_1=10010111$ ili $d_7d_6d_5d_4d_3d_2d_1d_0$.
- b) Neka je bit pozicija 1 u kôdnoj reči promenila vrednost. Pokazati kako se vrši detekcija i korekcija greške.

a)

Позиција	d_n	Сада ћемо редне бројеве позиција записати у бинарном облику и транзити 1 на одговарајућем месту (тако ћемо видети за паритетне битове шта покривају, и правилно их израчунати). Ево овако:
3	$d_1=1$	
5	$d_2=1$	
6	$d_3=1$	
7	$d_4=0$	$(3)_{10} \rightarrow (11)_2$ $(5)_{10} \rightarrow (101)_2$ $(6)_{10} \rightarrow (110)_2$
9	$d_5=1$	
10	$d_6=0$	$(7)_{10} \rightarrow (111)_2$ $(9)_{10} \rightarrow (1001)_2$
11	$d_7=0$	$(10)_{10} \rightarrow (1010)_2$ $(11)_{10} \rightarrow (1011)_2$
12	$d_8=1$	$(12)_{10} \rightarrow (1100)_2$

За паритет бит c_1 транзити 1 на првој позицији најмање тежине, за паритет бит c_2 транзити 1 на другој позицији најмање тежине, итд. У суштини, кад децималне вредности позиција претворимо у бинарне бројеве, онда за паритет бит c_n транзити 1 на n -тој позицији најмање тежине. Потом радимо $\oplus$.

a)

$$C_1 = d_1 \oplus d_2 \oplus d_4 \oplus d_5 \oplus d_7 = 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$C_2 = d_1 \oplus d_3 \oplus d_4 \oplus d_6 \oplus d_7 = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$C_3 = d_2 \oplus d_3 \oplus d_4 \oplus d_8 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$C_4 = d_5 \oplus d_6 \oplus d_7 \oplus d_8 = 1 \oplus 0 \oplus 0 \oplus 1 = 0$$

Сэда пишемо Хеминггов код:

$$101111001001 \Leftrightarrow C_1 C_2 d_1 C_3 d_2 d_3 d_4 C_4 d_5 d_6 d_7 d_8$$

$\uparrow \uparrow \uparrow \uparrow$
 $C_1 C_2 C_3 C_4$

b)

b) Detekcija i korekcija greške (Pozicija 1)

Pretpostavimo da se desila greška na **poziciji 1**. Naša primljena reč sada izgleda ovako: 0 0 1

1 1 1 0 0 1 0 0 1 (prvi bit je postao 0 umesto 1).

Postupak provere (Sindrom):

Ponovo računamo parnost za iste grupe bitova:

1. **Provera 1 (C_1):** Pozicije (1, 3, 5, 7, 9, 11) $\rightarrow$ (0, 1, 1, 0, 1, 0). Imamo 3 jedinice. Neparno $\rightarrow C_1 = 1$.
2. **Provera 2 (C_2):** Pozicije (2, 3, 6, 7, 10, 11) $\rightarrow$ (0, 1, 1, 0, 0, 0). Imamo 2 jedinice. Parno $\rightarrow C_2 = 0$.
3. **Provera 4 (C_4):** Pozicije (4, 5, 6, 7, 12) $\rightarrow$ (1, 1, 1, 0, 1). Imamo 4 jedinice. Parno $\rightarrow C_4 = 0$.
4. **Provera 8 (C_8):** Pozicije (8, 9, 10, 11, 12) $\rightarrow$ (0, 1, 0, 0, 1). Imamo 2 jedinice. Parno $\rightarrow C_8 = 0$.

Rezultat:

Formiramo binarni broj od rezultata provere (od C_8 ka C_1): **Sindrom** = $C_8C_4C_2C_1 = 0001_2$

Vrednost 0001 u dekadnom sistemu je **1**. To nam direktno govori da se greška nalazi na **poziciji 1**.

Korekcija: Bit na poziciji 1 se invertuje (iz 0 u 1), čime dobijamo originalnu ispravnu poruku.

TEORIJSKO

OBJASNJENJE

POCETAK !!!!!

★ I. Zašto uopšte postoje protokoli poput Stop-and-Wait, ARQ itd.?

Zamisli da šalješ paket preko interneta.

Paketi mogu:

- da nestanu
- da stignu duplirani
- da stignu oštećeni
- da stignu u pogrešnom redosledu

Zbog toga mrežni sloj mora imati način da kaže:

👉 „Šta si poslao? Da li je stiglo? Da li da pošaljem opet?”

Tu nastupaju *protokoli pouzdane isporuke* — ARQ (Automatic Repeat reQuest).

★ II. Šta STOP-AND-WAIT zapravo radi?

U stop-and-wait protokolu pošiljalac radi najjednostavniju moguću stvar:

1. Pošalje okvir.
2. Stane.
3. Čeka ACK.
4. Ako ACK stigne → šalje sledeći okvir.
5. Ako ne stigne → nakon timeout-a šalje isti okvir ponovo.

To znači:

📌 nikada ne šalje više od jednog okvira odjednom

Zato se ovaj protokol smatra „kliznim prozorom veličine 1“.

★ III. Zašto numerišemo okvire? (seq brojevi)

Broj sekvence (seq) postoji da bi primaoc mogao da razlikuje:

- **nov okvir**
- **ponovo poslati okvir** (zbog izgubljenog ACK-a)

Bez toga, primaoc bi duplo slao isti podatak mrežnom sloju.

Zamisli scenario:

Pošiljalac šalje okvir 0 → primaoc ga primi, pošalje ACK

ACK se izgubi!

Pošiljalac ne zna da je ok i **ponovo šalje okvir 0**.

Ako nema numerisanja:

primaoc bi mislio da je to *novi okvir* → duplirao bi podatke.

★ IV. Zašto se koriste samo 0 i 1 u Stop-and-Wait?

Zato što:

- Pošiljalac šalje jedan okvir.
- Primaoc čeka drugi.
- Čim se jedan isporuči, sledeći mora biti drugačiji.

Jedan bit je dovoljan da razlikuje:

$0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow \dots$

To je dovoljno jer stop-and-wait nikad nema više od jednog okvira „u letu“.

★ V. Šta znači `frame_expected`?

To je broj OKVIRA koji primaoc očekuje sledeći.

Ako je `frame_expected = 0`:

- Ako stigne `r.seq = 0` → OK, prosleđujemo mrežnom sloju.
- Ako stigne `r.seq = 1` → to je *duplikat*, ignoriše se.

Nakon ispravnog okvira:

SCSS

```
inc(frame_expected);
```

Dakle, ako je očekivao 0, sada očekuje 1.

VAZNA NAPOMENA !!!!

`frame_expected` je numericki broj okvira koji primaoc očekuje sledeći da primi, a `r.seq` je okvir koji će zapravo posiljaoc poslati a primaoc primiti.

★ VI. Šta radi ACK?

Primaoc šalje ACK sa vrednošću:

ini

`ack = numericki broj okvira koji sam poslednji uspešno primio`

Pošiljalac koristi ACK da zna:

- da okvir nije izgubljen
- da je prijem uspešan
- da može poslati sledeći

★ VII. Šta radi timer?

Timer se pokreće kada pošiljalac pošalje okvir.

Ako ne stigne ACK u određenom vremenu:

👉 znači da se ili okvir ili ACK izgubio.

Rešenje → pošiljalac ponovo šalje isti okvir.

To je cela ideja ARQ-a:

📌 ako nema potvrde, pošalji opet.

◆ Sender:

bash

Uzmi paket → stavi u okvir

Postavi **seq** = next_frame_to_send

Pošalji okvir

Start timer

Čekaj event

Ako je stigao ACK i `ack == next_frame_to_send`:

Stop timer

Uzmi sledeći paket

`next_frame_to_send++`

Ako je **timeout**:

Pošalji isti okvir ponovo

◆ Receiver:

java

Čekaj okvir

Ako `r.seq == frame_expected`:

Pošalji podatak mrežnom sloju

`frame_expected++`

Pošalji **ACK** (uvek)

`s.seq == next_frame_to_send` je redni broj okvira koji posiljaoc salje, a primaoc prima `r.seq`. U sustini, oznaka samo nije ista, ali to je taj okvir. Za slanje, oznaka je `s.seq`, a za primanje, oznaka je `r.seq`.

TEORIJSKO

OBJASNJENJE

KRAJ !!!!!

**DODATNA
OBJASNJENJA
POCETAK**

`event` nije redni broj frejma, pa se ne poredi sa `frame_expected`.

Ispravno bi bilo:

c

 Копирај код

```
if (event == frame_arrival)
{
    // Uzmi frejm sa fizičkog sloja
    from_physical_layer(&r);

    // Proveri da li je redni broj frejma koji smo primili onaj koji očekujemo
    if (r.seq == frame_expected)
    {
        to_network_layer(&r.info); // isporuči podatak višem sloju
        inc(frame_expected);        // očekuj sledeći frejm
    }
}
```

◆ Objašnjenje intuitivno:

- `event` govori šta se desilo (npr. stigao frejm, timeout, greška).
- `frame_expected` je redni broj frejma koji očeku↓ da primiš sledeći. **onaj koji očekuješ (`r.seq == frame_expected`).**
- Dakle prvo proveriš da li je stigao frejm (`event == frame_arrival`), a zatim proveriš da li je taj frejm

`frame_arrival` je konstanta koja označava vrstu događaja.

U kontekstu protokola kliznog prozora ili Stop-and-Wait ARQ:

```
typedef enum { frame_arrival, cksum_err, timeout } event_type;
```

- `enum` znači "nabrojivi tip" (enumeration).
- `event_type` je tip promenljive koji može da bude jedan od ovih događaja.
- `frame_arrival` znači: stigao je novi frejm sa fizičkog sloja.
- `cksum_err` znači: greška u proverenoj kontroli greške (checksum).
- `timeout` znači: isteklo vreme čekanja na ACK.

Dakle, kad pišeš:

```
c

if (event == frame_arrival)
{
    from_physical_layer(&r);
}
```

to znači: „Ako je događaj koji se upravo desio stizanje frejma, onda ga uzmi sa fizičkog sloja i obradi.“



Intuitivno: `frame_arrival` je signal da je stigao paket koji treba da se pročita.

1 Pošiljalac i okvir

- Pošiljalac ima okvir `s` koji želi da pošalje.
- Taj okvir sadrži:
 - `s.info` → stvarni podatak (paket iz mrežnog sloja)
 - `s.seq` → redni broj okvira (npr. 0 ili 1 kod jednobitnog kliznog prozora)
 - `s.ack` → potvrdu poslednjeg primljenog okvira od strane primaoca
- Pošiljalac šalje samo jedan okvir odjednom i čeka ACK (potvrdu) pre nego što pošalje sledeći.

2 Primalac i okvir

- Primalac ima okvir `r` koji prima sa fizičkog sloja.
- `r.seq` je redni broj okvira koji primalac očekuje (`frame_expected`).
- Kad primalac primi okvir sa očekivanim rednim brojem:
 1. Dostavlja podatke mrežnom sloju (`to_network_layer(&r.info)`)
 2. Povećava `frame_expected` (da zna koji okvir očekuje sledeći put)
 3. Šalje ACK nazad pošiljaocu (`s.ack = 1 - frame_expected`)

3 Šta znači “pošiljalac šalje jedan, a primalac prima drugi”?

- To je samo logika numeracije:
 - Pošiljalac šalje okvir `seq = 0`.
 - Primalac očekuje `frame_expected = 0`.
 - Kad ga primi, primalac šalje ACK (koji je `1 - frame_expected = 1`).
 - Pošiljalac čeka `ACK = 0`, prima ga, i sada može poslati sledeći okvir (`seq = 1`).
- Dakle, uvek postoji samo jedan okvir “u letu”. Pošiljalac i primalac sinhronizuju se kroz `seq` i `ack`.

Ključne tačke

1. Jedan okvir u letu

- Pošiljalac šalje **samo jedan okvir** i ništa drugo ne šalje dok ne dobije potvrdu.

2. ACK – potvrda

- Kada primalac primi okvir ispravno, šalje **ACK** nazad pošiljaocu.
- ACK sadrži redni broj okvira koji primalac očekuje sledeći put.

3. Timeout

- Ako pošiljalac **ne dobije ACK u zadanom vremenu**, pretpostavlja da je okvir izgubljen ili oštećen i **ponovo šalje isti okvir**.

4. Redosled i brojevi okvira

- Jedan bit za seq → znači da imamo **samo dva moguća broja okvira**: 0 i 1.
- Sledeći okvir koji pošiljalac šalje uvek je različit od prethodnog, pa **seq** naizmenično ide $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \dots$

5. Pouzdanost

- Stop-and-Wait je pouzdan **jednosmerni protokol** — garantuje da će svaki paket stići, ispravan i u **pravom redosledu**, ali zbog toga što šalje po jedan okvir i čeka ACK, **nije brz za velike količine podataka**.

✓ frame_expected – šta je?

To je broj okvira koji primaoc *očekuje* sledeći.

Ako radi Stop-and-Wait sa 0/1 sekvencama, onda:

- ako je poslednji primljen bio 0 → `frame_expected = 1`
- ako je poslednji primljen bio 1 → `frame_expected = 0`

Primaoc ZNA koji bi trebalo da stigne sledeći.

✓ r.seq – šta je?

To je redni broj okvira koji je zaista stigao preko fizičkog sloja.

Ali taj broj može biti:

- tačan (onda je dobro)
- dupliran (ako je prethodni ACK izgubljen)
- pogrešan (ako je poremećaj na liniji)
- zakašneli okvir koji je ostao negde u mreži

Stop-and-Wait ne garantuje da će stići "lep, tačan, je ↓ tven" okvir.

✓ Kada je sve OK?

Ako važi:

👉 `r.seq == frame_expected`

onda znači *"Ovo je okvir koji zaista očekujem — prosleđujem ga mrežnom sloju"*

Zato pseudokod radi:

SCSS

```
to_network_layer(&r.info);  
inc(frame_expected);
```

Dakle:

1. Prosledimo podatke gore (aplikacijama).
2. Ažuriramo šta sledeće očekujemo.

✓ Kada NIJE ok?

Ako `r.seq != frame_expected`, znači:

- stigao je duplikat
- stigao je stari okvir
- stigao je pogrešan okvir
- stiglo je nešto što primaoc NE sme proslediti gore

U tom slučaju:

- okvir se odbacuje
- ali ACK se ipak šalje (da pošiljalac zna gde smo stali)

To je važno: Stop-and-Wait nikada ne prosleđuje mrežnom sloju pogrešan okvir.

✓ Tvoja rečenica je idealna:

"Ako važi $r.seq = frame_expected$ onda je sve OK i primaoc prosledjuje paket mrežnom sloju, a ako važi $r.seq \neq frame_expected$ onda nije ok i nema slanja frejma od primaoca ka mrežnom sloju."

Da dodam samo jednu malu ispravku da bude savršeno:

- "*nema slanja frejma ka mrežnom sloju*" je tačno
- ali *primaoc ipak šalje ACK nazad pošiljaocu* (čak i ako je duplikat)

To sam samo dodao da znaš celu sliku.

✓ Ukratko – tvoja logika je 100% ispravna

Radiš fenomenalno.

Jedino što ti je falilo je mala nijansa o ACK-u, ali suštinu si pogodio bez greške.

✓ ACK znači dve stvari istovremeno

1) „Primio sam okvir *bez greške*.“

Dakle: sadržaj (data) je stigao kako treba.

2) „Primio sam *baš taj* okvir koji sam očekivao — onaj sa brojem `frame_expected`.“

To je ključ!

ACK nije samo „dobio sam ga“ — nego „dobio sam ga *u pravom redosledu*“.

✓ Ali ima jedna zamka koju moraš razumeti (bitna!)

ACK ne znači samo prihvatanje, nego može značiti i ovo:

- ponekad se ACK šalje čak i ako okvir NIJE prosleđen mrežnom sloju (npr. duplikat)
- ali se ACK šalje uz broj frejma koji primaoc očekuje sledeći

Dakle ACK sa brojem X znači:

„Sve do $X-1$ sam uspešno primio. Sledeći koji očekujem je X.”

U Stop-and-Wait (gde se koristi samo 0 ili 1), to znači:

- ako je očekivao 0, a stigao je 0 → prosledi ga i pošalje ACK=1
- ako je očekivao 0, a stigao je 1 → ignoriše ga i pošalje ACK=0

Čak i ako je pogrešan — ACK se šalje, ali kaže „i dalje očekujem 0”.

✓ Ukratko (najjasnije moguće):

ACK = „Primio sam sve ispravno zaključno sa prethodnim okvirom. Sledeći koji očekujem je X.”

A pošiljalac onda zna:

- ako se ACK slaže → može da pošalje sledeći okvir
- ako se ACK ne slaže → mora ponovo da šalje isti okvir

✓ Tvoja rečenica je tačna, samo je dopuni ovako:

„ACK je potvrda da je okvir stigao bez greške i da je to okvir koji je primaoc očekivao – a broj ACK-a označava koji okvir primaoc očekuje sledeći.”

To je pun smisao ACK-a u Stop-and-Wait protokolu.

✓ Šta je ACK? (bez mudrovanja)

ACK = broj okvira koji primalac želi da dobije SLEDEĆI.

To je sve.

Ako si poslao okvir 0 i on stigne kako treba → primalac kaže:

ACK = 1

(„primio sam 0, pošalji mi 1“)

Ako si poslao okvir 1 → primalac koji ga primi kaže:

ACK = 0

(„primio sam 1, pošalji mi 0“)

Zato se koristi samo 1 bit, 0 i 1 se smenjuju.

✓ Jedna jedina istina koju treba da zapamtiš:

ACK NE govori „primio sam X“.

ACK govori „očekujem X“.

I pošiljalac to koristi da zna koji okvir sledeći da šalje.

**DODATNA
OBJASNJENJA
KRAJ**

Napišite kod za protokol za neograničen jednosmeran prenos podataka, inspirisan Ethernet-om. Implementirajte slanje i primanje paketa između dva uređaja preko mrežnog medijuma. Ne treba dodavati mehanizme kontrole grešaka ili kontrole toka, već samo omogućiti slanje i primanje paketa. Vaš kod treba da omogući jednostavno slanje paketa od pošiljaoca do primaoca bez ikakvih dodatnih funkcionalnosti.

1 Definicije i uključivanje zaglavlja

C

```
typedef enum {frame_arrival} event_type;  
#include "protocol.h"
```

VAZNO !!! `event_type`, kao što i sama rec kaže, označava koji događaji mogu da se dese. U ovom konkretnom zadatku može da se desi samo stizanje frejma, to je jedini eventualni događaj. Zato što da se spominje ARQ protocol, onda bi trebalo da se napisu 3 moguća događaja, a to su `timeout`, `frame_arrival`, `cksumerr`. Ali posto se ne spominje ARQ protocol, onda je moguće samo da se dogodi `frame_arrival`. Ovo `typedef enum` to je kao neki eto tip enumeracije. `event_type` je, jasno je iz konteksta, eventualni događaj koji treba da se desi, a ovde je to stizanje frejma.

- `event_type` definiše koji događaji mogu da se dese.
 - Ovde je samo `frame_arrival` jer primalac čeka da okvir stigne.
- `protocol.h` sadrži definicije za:
 - strukture `frame` i `packet`,
 - funkcije `from_network_layer`, `to_physical_layer`, `from_physical_layer`, `to_network_layer`,
 - i `wait_for_event`.

A `protocol.h` je heder fajl koji sadrži sve definicije za sve funkcije i promenljive koje koristimo u resavanju ovakvih tipova zadataka.

2 Funkcija pošiljaoca

c

```
void sender1(void)
{
    frame s;
    packet buffer;
```

Kad pričamo o posiljaocu, priča je ovakva: pre beskonacne petlje, imamo definisane 2 promenljive, prva je frejm s, jer s od send je pocetno slovo, a drugo je packet buffer, dakle podatak koji treba da se smesti u okvir. I to je upravo ta poenta, da i kod posiljaoca i kod primaoca imamo 2 definisane promenljive pre nego sto pocne petlja ova beskonacna. Samo sto je to kod posiljaoca frame s i packet buffer, a kod primaoca frame r i event_type event.

- s → okvir koji će se poslati kroz fizički sloj.
- buffer → podatak iz mrežnog sloja koji treba da se pošalje.

c

```
while(true)
{
    from_network_layer(&buffer);
    s.info = buffer;
    to_physical_layer(&s);
}
```

VAZNO !!! Kod ovog zadatka, mi imamo funkciju void sender1(void), ali u toj funkciji prvo napisemo tipove promenljivih, pa onda tek u beskonacnoj petlji **OBAVEZNO** PISEMO uzimanje paketa sa mreznog sloja i smestanje u frejm s.

Kod funkcije posiljaoca, a ta funkcija je void sender1(void), imamo frame s, dakle frejm koji se salje, a s je jer je s pocetno slovo od send. Takodje, imamo i packet buffer, a to buffer nam je podatak iz mreznog sloja koji treba da se posalje. Zatim, pisemo beskonacnu petlju, i u toj beskonacnoj petlji uzimamo podatak sa mreznog sloja, stavljamo u frejm i onda salje kroz fizicki sloj. Znaci mi definisemo promenljive, kazemo da imamo frejm za slanje i podatak koji treba da preuzmemo sa mreznog sloja i stavimo u okvir. Potom, evo zasto stavljamo beskonacnu petlju: zato sto posiljalac stalno salje podatke. I sad, kad otvorimo tu beskonacnu petlju, uzimamo sa mreznog sloja podatak, stavljamo ga u okvir i saljemo duz fizickog sloja.

- **beskonačna petlja:** pošiljalac stalno šalje podatke.
- `from_network_layer(&buffer)` → uzima sledeći paket iz mrežnog sloja.
- `s.info = buffer` → smešta podatak u okvir.
- `to_physical_layer(&s)` → šalje okvir kroz fizički sloj.

✓ Bitno: Nema kontrole grešaka, nema ACK-a, nema kontrole toka — pošiljalac šalje podatke **koliko god brzo može**.

E sad da objasnimo funkciju `primaoca`. To se kaže na engleskom jeziku `receiver`, i zato tu funkciju tako i nazivamo, a ona je tipa `void`, što znači da ne šalje povratnu vrednost. Tu imamo `frame r`, od reči `received`, što znači primljeno. Potrebno je da napisemo `event_type event;`. To je nešto kao u smislu eventualni događaj, to je bukvalni prevod od `event_type`. E sad, opet ide beskonačna petlja. Zasto? Pa zato što se ona izvršava sve dok `primaoc` prima okvire. Unutar te petlje kažemo: `wait_for_event ( &event ) ;`. Dakle, čekamo da se desi neki događaj, a taj događaj je ustvari stizanje okvira. Zato pišemo `wait_for_event ( &event ) ;`. Potom, pišemo: `from_physical_layer (&r);` zato što je to preuzimanje okvira sa fizičkog sloja duž kojeg smo poslali okvir, secamo se da smo to napisali malopre, kod definisanja funkcije za slanje, da smo slali okvir duž fizičkog sloja, samo što se on tada nazivao `s`, zbog slanja, a sad je `r` zbog primanja. Posle toga je jasno šta radimo, saljemo ovo mrežnom sloju, dakle pišemo sledeću naredbu: `to_network_layer(&r);`

3 Funkcija `primaoca`

C

```
void receiver1(void)
{
    frame r;
    event_type event;
```

Kad pričamo o posiljaocu, priča je ovakva: pre beskonacne petlje, imamo definisane 2 promenljive, prva je frejm `s`, jer `s` od `send` je početno slovo, a drugo je `packet buffer`, dakle podatak koji treba da se smesti u okvir. I to je upravo ta poenta, da i kod posiljaoca i kod primaoca imamo 2 definisane promenljive pre nego što počne petlja ova beskonačna. Samo što je to kod posiljaoca `frame s` i `packet buffer`, a kod primaoca `frame r` i `event_type event`.

- `r` → okvir koji primalac prima.
- `event` → promenljiva koja čeka događaj (stizanje okvira).

```
while(true)
{
    wait_for_event(&event);
    from_physical_layer(&r);
    to_network_layer(&r.info);
}
}
```

Kad je u pitanju primaoc, tu imamo definisane 2 promenljive pre nego sto zapocne beskonacna petlja. Te 2 definisane promenljive su ustvari frame r kao i event_type event; Zasto? Pa zato sto je uloga primaoca da primi okvir, a taj okvir je oznacen sa r jer je to pocetno slovo od received, sto u prevodu sa engleskog znaci primljen. Dakle, zato je to frame r. A sto se tice ovoga event_type event; e pa to je dogadjaj koji treba da se desi, a to je da stigne okvir, to je taj dogadjaj. To su te 2 promenljive. Zatim, ide beskonacna petlja while(true) u okviru koje pisemo wait_for_event(&event), dakle cekamo da se desi dogadjaj, a to je dolazak paketa. Zatim, preuzimamo ga sa fizickog sloja naredbom from_physical_layer(&r); a potom pisemo naredbu za slanje okvira mreznom sloju, naravno, ona glasi to_network_layer(&r.info);

- Petlja stalno čeka događaje.
- `wait_for_event(&event)` → blokira dok okvir ne stigne.
- `from_physical_layer(&r)` → uzima okvir sa fizičkog sloja.
- `to_network_layer(&r.info)` → predaje podatak mrežnom sloju.

✓ **Bitno:** Primalac pretpostavlja da može da procesuirati sve podatke beskonačno brzo, tako da nema zagušenja.

Ovo je jednostavan model „beskonačne linije“: pošiljalac stalno pumpa podatke, primalac stalno prima. Nema grešaka, nema kontrole toka, nema retransmisije. Idealan za učenje osnova mrežnog prenosa pre nego što dodamo stop-and-wait ili sliding window protokole.

Implementirajte jednosmerni protokol "stani i čekaj" u C. Protokol treba omogućiti slanje i primanje okvira između pošiljaoca i primaoca. Pošiljalac treba da čeka na ACK pre nego što pošalje sledeći okvir, a primaoc treba da šalje ACK nakon uspešnog prijema okvira. Obezbedite da se komunikacija odvija bez mogućnosti prepunjenosti primaoca.

★ Šta je stop-and-wait protokol?

To je najjednostavniji pouzdani protokol koji omogućava:

- ✓ Jednosmeran prenos podataka
- ✓ Pošiljalac pošalje jedan okvir → čeka ACK → tek onda šalje sledeći
- ✓ Primalac šalje ACK za svaki primljeni okvir
- ✓ Sprečava prepunjavanje primaoca

Ovaj zadatak koristi idealni kanal (nema grešaka), pa ACK može biti bilo šta — čak i *dummy frame*.

I zato je ova implementacija vrlo jednostavna.

U ovom zadatku se spominje to da primaoc salje ACK posiljaocu, dakle mi za ovaj zadatak imamo 4 reda u while petlji kad je u pitanju posiljaoc, a ne 3, sto bi bio slucaj kad ne bi trebala potvrda da je stigao okvir. ZnacI treba nam 4 reda. A evo i zasto: zato sto nam je taj 4. red kod posiljaoca ustvari wait_for_event; dakle to je cekanje dogadjaja kod posiljaoca. Taj dogadjaj je stizanje ACK - a.

Sto se tice ovoga sto ide pre while(true), e tu nam idu 3 stvari, jasno je koje. Ide frejm s, koji je frejm za slanje, s je skraceno od send. Zatim packet buffer, sto nam je podatak koji iz mreznog sloja stavljamo u okvir s. I imamo event_type event; i jasno je da nam se to odnosi na cekanje da se desi dogadjaj, a dogadjaj kod posiljaoca je da stigne ACK frejm. Vazno je naglasiti da pre slanja okvira uzimamo podatak sa mreznog sloja i da ga stavljamo u okvir s. Dakle, sa mreznog sloja. Kod primaoca imamo 2 ova pre while petlje i 4 ova unutar while petlje.

Pošiljalac

```
typedef enum {frame_arrival} event_type;

void sender2(void)
{
    frame s;
    packet buffer;
    event_type event;

    while(true)
    {
        from_network_layer(&buffer);    /* uzmi paket */
        s.info = buffer;                /* spakuj u okvir */

        to_physical_layer(&s);          /* pošalji okvir */

        wait_for_event(&event);         /* čekaј ACK */
    }
}
```



Primalac

c

```
void receiver2(void)
{
    frame r, s;
    event_type event;

    while(true)
    {
        wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
        to_physical_layer(&s);
    }
}
```

✓ Objašnjenje

1. Čeka da okvir stigne.
2. Uzima ga sa fizičkog sloja.
3. Predaje ga mrežnom sloju.
4. Šalje neki okvir nazad → to je ACK.
 - Ne mora da sadrži ništa pametno, samo služi da probudi pošiljaoca.

★ Ukratko — zašto je ovo ispravno rešenje?

- ✓ Jedan okvir se šalje → čeka ACK
- ✓ Primalac šalje potvrdu
- ✓ Kanal je idealan → nema kontrolnih bitova
- ✓ Ne koristi redne brojeve jer nema gubitka
- ✓ Isključivo sprečava preopterećenje primaoca
- ✓ Tačno je ono što zadatak traži



Vaš zadatak je implementirati protokol za jednosmerno slanje podataka šumnim kanalom koristeći protokol s potvrđivanjem i ponovnim slanjem (ARQ - Automatic Repeat reQuest). Protokol će koristiti numerisane okvire i ACK-ove te će biti izveden kao Stop-and-Wait ARQ s dvobitnim brojevima za okvire.

★ Šta radimo u ovom protokolu?

Kanal je nepouzdan (šuman):

- okviri mogu da se izgube
- ACK može da se izgubi
- okvir može biti oštećen

Dakle:

- ✓ Pošiljalac šalje okvir → pokreće timer
- ✓ Ako stigne odgovarajući ACK → šalje sledeći okvir
- ✓ Ako ne stigne (timeout) → ponavlja isti okvir
- ✓ Koristimo *seq broj* (0 ili 1)
- ✓ Primalac prihvata samo okvir koji očekuje

Standardni Stop-and-Wait ARQ.

U ovom zadatku, imamo 5 ovih definisanih promenljivih pre nego sto napisemo beskonacnu petlju. Za razliku od onog drugog zadatka, ovde pisemo `from_network_layer(&buffer);` pre ulaska u beskonacnu petlju, i to nam je 1 od tih 5 stvari koje idu pre otvaranja beskonacne petlje. Takodje, imamo i neke zajednicke stvari koje idu pre beskonacne petlje, kao sto su na primer `frame s` i `packet buffer`, i samo ta 2. Nije zajednicko na primer `event_type` `event` i `seq_nr` `next_frame_to_send = 0`. A posebno je vazno `next_frame_to_send = 0`, jer je to sledeci frejm za slanje, njegov redni broj, i on pocinje od nule. Eto, to je vazno znati pre ulaska u beskonacnu petlju. I onda, nakon ulaska u beskonacnu petlju mi treba da napisemo `s.info = buffer;` i `s.seq = next_frame_to_send;` Evo i zasto: Zato sto je `buffer` podatak koji ubacujemo u okvir za slanje, a `seq` je redni broj frejma koji treba poslati. I onda, evo sta radimo sa tim. Logicno je, frejm s saljemo duz fizickog sloja, a za `s.seq` nam vazi da za njega ukljucujemo tajmer. Za `s.seq` ukljucujemo tajmer, jer postoji vreme koje je potrebno da stigne ACK (potvrda da je paket stigao kako treba, mada i ako stigne pogresan paket ACK se salje svakako), tako da je to intuitivno jasno u sustini.

```

/* Stop-and-Wait ARQ protokol za nepouzdan kanal */

#define MAX_SEQ 1    /* okviri se broje 0 i 1 */

typedef enum {frame_arrival, cksum_err, timeout} event_type;

#include "protocol.h"

/* ----- POŠILJALAC ----- */
void sender3(void)
{
    seq_nr next_frame_to_send = 0;    /* tekući okvir */
    frame s;                          /* izlazni okvir */
    packet buffer;                    /* paket iz mrežnog sloja */
    event_type event;

    from_network_layer(&buffer);      /* učitaj prvi paket */

    while (true)
    {
        /* napravi okvir */
        s.info = buffer;

```

Eto, objasnili smo sta nam sve treba za ovo. Dakle, 5 ovih nam ide pre beskonacne petlje. To su nam: frame s, event_type, packet buffer, seq_nr next_frame_to_send = 0 (obavezno ga inicijalizujemo na nulu), from_network_layer(&buffer). Potom, ovde pisemo beskonacnu while(true) petlju, i u njoj vrlo jasno kazemo da imamo s.seq = next_frame_to_send, kao i s.info = buffer;. Onda, pustamo da se salje okvir duz fizickog sloja, dakle to nam je naredba to_physical_layer(&s), a za s.seq ukljucujemo tajmer naredbom start_timer(s.seq); Zasto? Pa zato sto nam je jasno da je s.seq sledeci okvir koji je potrebno da se posalje (definisali smo ga kao s.seq = next_frame_to_send, tako da je vise nego ocigledan kontekst), a tajmer aktiviramo jer je potrebno da u odredjenom roku stigne potvrda da je prethodni paket uspesno stigao i da moze da se posalje sledeci. Ne moze u STOP-AND-WAIT protokolu da se posalje sledeci paket ukoliko za ovaj prethodni ne stigne potvrda da je uspesno stigao. I sada, da se vratimo na teorijsko znanje a to je da primaoc salje posiljaocu potvrdu da je paket uspesno primljen, a to je ACK. Nastavak objasnjenja je na sledecem slajdu.



```
s.seq = next_frame_to_send;
```

```
to_physical_layer(&s);      /* pošalji okvir */
```

```
start_timer(s.seq);         /* startuj tajmer */
```

```
wait_for_event(&event);     /* čekaj ACK ili timeout */
```

```
if (event == frame_arrival)
```

```
{
```

```
    frame ack_frame;
```

```
    from_physical_layer(&ack_frame);
```

```
    /* stigao pravi ACK → možemo dalje */
```

```
    if (ack_frame.ack == next_frame_to_send)
```

```
    {
```

```
        stop_timer(ack_frame.ack);
```

```
        from_network_layer(&buffer); /* učitaj sledeći paket */
```

```
        inc(next_frame_to_send);     /* pređi na sledeći broj okvira (0→1→0) */
```

```
    }
```

```
}
```

```
else if (event == timeout)
```

```
{
```

☐ Dakle, objasnili smo naredbu `s.info = buffer`, kao i `s.seq == next_frame_to_send`, i `start_timer(s.seq)`, i `to_physical_layer(&s)`; E sad ovako: ispod svega ovoga pisemo naredbu `wait_for_event(&event)`; a to znaci da se ceka neki dogadjaj. I sad, kod posailjaoca taj dogadjaj moze biti samo stizanje ACK frejma. I sad, iz sintaksnih razloga, mi ne pisemo odma da je to ACK frejm, nego prvo (odmah ispod naredbe koja oznacava da se ceka neki dogadjaj), kazemo `if ( event == frame_arrival )` znaci da je pristigao neki frejm. Potom, definisemo ACK frejm naredbom ovom: `frame ack_frame; // da se zna da je to bas taj ACK frejm`. Potom, kazemo odakle smo ga preuzeli, a to je fizicki sloj, pa zato pisemo `from_physical_layer(&ack_frame)`; Nadalje je procedura jasna, ako je ACK frejm jednak sledecem frejmu za slanje, stopira se tajmer i salje se sledeci okvir. Samo napisemo: `stop_timer(ack_frame.ack);` .



```

        /* timeout → ponovo šalji isti okvir */
        continue; /* petlja će ga ponovo poslati */
    }
}

/* ----- PRIMALAC ----- */

void receiver3(void)
{
    seq_nr frame_expected = 0; /* očekivani okvir */
    frame r, s;                /* primljeni i ACK okvir */
    event_type event;

    while (true)
    {
        wait_for_event(&event); /* čekaј okvir ili grešku */

        if (event == frame_arrival)

```

Napisemo `stop_timer(ack_frame.ack)`; zato sto je to bas taj frejm koji kada stigne, tajmer se stopira. To je upravo ACK frejm. O njemu se radi. I zato to pisemo. Dalje, setimo se da nam je upravo ACK vrednost sledeceg frejma koji treba poslati. Pa zato uostalom i ide uslov `if ( frame_ack == next_frame_to_send )`. Rekli smo gore da je to jasno, al eto da ponovimo za svaki slucaj. To je to, znaci objasnili smo deo koda koji kaze: `if(frame_ack.ack == next_frame_to_send)` i `stop_timer(ack_frame.ack)`. Dalje, uzimamo podatak (to je ovo sto smo krstili buffer), stavljamo na fizicki sloj (naredba `to_physical_layer ( &buffer )`) i saljemo dalje kome treba. Potom ide naredba koja broji ovo, a to je `inc ( next_frame_to_send )`; E sad, ako imamo naredbu `if ( event == timeout )` e pa to onda znaci da je isteklo vreme za potvrdu da je okvir isporucen, e to je taj slucaj timeout. Ako je to slucaj, onda jednostavno pisemo `continue`, u smislu jednostavno nastavi da radis isto sto i dosad, opet salji isti okvir. Tako pise I u teoriji, da ako je slucaj timeout da se onda opet salje isti okvir. To je sve sto se tice posiljaoca.

Na sledecem slajdu pocinje objasnjenje za primaoca.

```

{
    from_physical_layer(&r);

    if (r.seq == frame_expected)
    {
        /* ispravan i očekivan okvir */
        to_network_layer(&r.info);
        inc(frame_expected);
    }

    /* uvek šalji ACK za poslednji primljen okvir */
    s.ack = 1 - frame_expected;
    to_physical_layer(&s);
}
}
}

```

Napisemo void receiver3(void), dakle funkciju za primaoca. Tu imamo 3 ove promenljive koje idu pre beskonacne petlje, a to su frame r, s, pa seq_nr frame_expected=0, event_type event. Znaci imamo 3 takvih pre beskonacne while(true) petlje. Hajde sad da objasnimo svaku od njih. Ovo prvo je očekivani okvir koji primalac očekuje da primi. Logično je da on kreće od nule, jer i prvi okvir koji je za skanje kreće od nule, pa se onda nekako intuitivno poklapa. Ovo drugo je frame r,s. r je okvir koji je primljen i to je skracenica od received, a s je da bi primaoc mogao da posalje ACK onome ko mu okvir salje, dakle posiljaocu. Potom ide event_type event; Dakle, eventualni dogadjaj koji treba da se desi. I sada, to je podrazumevano da je stizanje okvira u pitanju, da je to taj dogadjaj, ali sintaksa... Dakle, frame r, s, seq_nr frame_expected = 0, i event_type event.

Nakon sto napisemo ove 3 stvari, onda otvaramo beskonacnu petlju, to je ovo sto treba da napisemo: `while(true);` . Onda, logicko, uzimamo frejm sa fizickog sloja, pisemo `from_physical_layer(&r);` Zasto r? Pa zato sto je to pocetno slovo od `received`, a to je primljeni frejm. Zasto uzimamo sa fizickog sloja? Pa zato sto posiljalac salje frejm duz fizickog sloja, to je vec receno ranije, pa je onda valjda logicko da primalac uzima okvir sa fizickog sloja. Potom, ide naredba `if(r.seq == frame_expected)`. Ona znaci sledece: ako je primljeni okvir (a to je ovo `r.seq`) zapravo i ocekivani okvir(`frame_expected`) onda ga treba proslediti mreznom sloju. Zato ide naredba `to_network_layer(&r.info);` Potom ide i naredba `inc(frame_expected);` koja prebrojava okvire. E sad, izvan ovog `if` uslova ide naredba `s.ack = 1 – frame_expected;` i salje se duz fizickog sloja. Pise se na ovaj nacin: `s.ack = 1 – frame_expected;` pa onda ispod `to_physical_layer(&s);` E sad da objasnimo zasto se `s.ack` definise izvan `if` uslova. Pa zato sto se `ack` salje svakako, i da je primljeni okvir ocekivani i da nije, svakako se `ack` salje, a `if` uslov se tice upravo toga da li je primljeni okvir odgovarajuci ili nije. Eto zato se definicija `s.ack` pise izvan `if` uslova, jer nema veze sa tim da li je okvir odgovarajuci ili nije.

★ Zašto je ovo ispravno?

✓ Rukuje greškama u kanalu

- Ako se izgubi okvir → pošiljalac dobije timeout → šalje ponovo
- Ako se izgubi ACK → pošiljalac ponovo šalje okvir (primalac ga ignoriše jer nije očekivani)

✓ Numeracija 0/1 rešava dupliranja

- Ako primalac dobije dva puta isti okvir, prepozna ga po `seq` broju.

✓ ACK nosi vrednost koju pošiljalac poredi sa svojim seq brojem

- Samo ako se poklope → prelazi na sledeći okvir.

✓ Sve je po principu Stop-and-Wait ARQ

Ovo je tačno ono što zadatak traži.

Vaš zadatak je razviti jednostavan jednobitni protokol kliznih prozora za pouzdanu komunikaciju između pošaoaca i primaoca putem šumnog kanala. Ovaj protokol koristi samo jedan bit za potvrdu (ACK - Acknowledgment) i jedan bit za numeraciju okvira.

1 Inicijalizacija

```
c

seq_nr next_frame_to_send = 0;
seq_nr frame_expected = 0;
frame r,s;
packet buffer;
event_type event;
```

- `next_frame_to_send` – sledeći okvir koji pošiljalac šalje. Počinje od 0.
- `frame_expected` – sledeći okvir koji primalac očekuje da primi. Takođe počinje od 0.
- `r` – prihvaćeni okvir (received).
- `s` – okvir koji šaljemo (send).
- `buffer` – prostor u memoriji gde se čuva paket pre nego što ga stavimo u okvir.
- `event` – promenljiva koja detektuje događaje (stizanje frejma, greška, timeout).

2 Prvi paket koji šaljemo

```
c  
  
from_network_layer(&buffer);  
s.info = buffer;  
s.seq = next_frame_to_send;  
s.ack = 1 - frame_expected;  
to_physical_layer(&s);  
start_timer(s.seq);
```

- Prvi paket iz mrežnog sloja ide u okvir.
- `s.seq` je broj ovog okvira.
- `s.ack` potvrđuje poslednji primljeni okvir (budući da je ovo **bidirekcionalno**).
- Pošiljalac pokreće timer da zna koliko može da čeka ACK pre nego što pokuša **ponovni prenos**.

3 Glavna petlja

c

```
while(true)
{
    wait_for_event(&event);
```

- Petlja stalno čeka događaje, može biti stizanje frejma, greška, timeout.

4 Kada stigne okvir

c

```
if(event == frame_arrival)
{
    from_physical_layer(&r);
```

- Kada stigne okvir, učitavamo ga u promenljivu `r`.

5 Obrada primljenog okvira

c

```
if(r.seq == frame_expected)
{
    to_network_layer(&r.info); // pošalji podatke mrežnom sloju
    inc(frame_expected);       // očekujemo sledeći okvir
}
```

- Ako je redni broj okvira onaj koji primalac očekuje, paket ide mrežnom sloju.
- `frame_expected` se uvećava (mod 2, pošto je jednobitni prozor).

6 Obrada ACK-a

c

```
if(r.ack == next_frame_to_send)
{
    stop_timer(r.ack);           // zaustavi tajmer, jer je okvir stigao
    from_network_layer(&buffer); // uzmi sledeći paket iz mrežnog sloja
    inc(next_frame_to_send);     // pripremi sledeći okvir za slanje
}
```

- Ako je ACK onaj koji smo čekali, znamo da je okvir uspešno stigao.
- Uzimamo sledeći paket iz mrežnog sloja i pripremamo ga za slanje.

7 Slanje sledećeg okvira

c

 Copy code

```
s.info = buffer;  
s.seq = next_frame_to_send;  
s.ack = 1 - frame_expected;  
to_physical_layer(&s);  
start_timer(s.seq);
```

- Sada pošiljalac šalje sledeći okvir sa novim rednim brojem i ACK-om za poslednji primljeni okvir.
- Timer se ponovo pokreće.

Intuitivno objašnjenje:

- Jednobitni klizni prozor znači da u svakom trenutku može biti aktivan samo jedan okvir (seq = 0 ili 1).
- Bidirekcionalno – svaka poruka nosi i podatke i ACK za prethodni okvir u suprotnoj smeru.
- Timer obezbeđuje da se izgubljeni ili oštećeni okviri ponovo pošalju.

U Ethernetu se koristi standardna veličina okvira kako bi se omogućila pouzdana komunikacija. Pretpostavite da radite sa 10-Mbps Ethernet mrežom. Trajanje jednog bita iznosi 100 nsec. Kolika je najveća dopuštena veličina Ethernet okvira (u bitovima) za ovu mrežu?

✓ Zadatak

U Ethernetu (10 Mbps) trajanje jednog bita je:

$$T_{bit} = 100 \text{ ns} = 100 \times 10^{-9} \text{ s}$$

Traži se najveća dopuštena veličina Ethernet okvira (u bitovima) za 10-Mbps Ethernet.

Ali — bitno je: Ethernet ne bira maksimalnu veličinu prema trajanju bita, već je standardom *fiksirano* šta je maksimalna veličina okvira.

✓ Poznato iz standarda

Ethernet standard (IEEE 802.3) određuje:

- Minimalna veličina okvira: 64 bajta = 512 bitova
- Maksimalna veličina okvira: 1518 bajta = 12 144 bita

Ova maksimalna veličina se NE menja sa brzinom mreže.

Dakle, i na 10 Mbps i na 100 Mbps i na 1 Gbps maksimalna veličina ostaje ista.

Maksimalna veličina Ethernet okvira

$$\text{Max okvir} = 1518 \text{ B} = 1518 \times 8 = 12\,144 \text{ bita}$$

Ako je pitanje htelo da poveže trajanje bita sa trajanjem okvira...

Trajanje okvira:

$$T_{\text{okvir}} = 12\,144 \times 100 \text{ ns} = 1\,214\,400 \text{ ns}$$

$$T_{\text{okvir}} = 1.2144 \text{ ms}$$

Dakle, slanje najvećeg Ethernet okvira na 10 Mbps traje oko 1.21 milisekundu.

Konačan odgovor

Najveća dopuštena veličina Ethernet okvira na 10-Mbps mreži je:

12 144 bita

Pretpostavimo da imate malu firmu i razmatrate opcije za postavljanje mreže u svom kabinetu. Treba odlučiti između Ethernet mreže i Wi-Fi mreže. Razmislite o prednostima i nedostacima oba tipa mreža i donesite odluku. Navedite najmanje tri ključne razlike između Ethernet-a i Wi-Fi-a i obrazložite kako će te razlike uticati na vašu odluku.

Odabir između Ethernet i Wi-Fi za malu firmu

1 Ethernet mreža

Prednosti:

- 1. Veća brzina prenosa:** Gigabit Ethernet nudi 1 Gbps ili više, dok tipični Wi-Fi (npr. Wi-Fi 5) nudi oko 100–300 Mbps.
- 2. Stabilnost i pouzdanost:** Ethernet kablovi nisu podložni smetnjama ili gubitku signala.
- 3. Manje kašnjenje (latencija):** Idealno za video konferencije, VoIP i brzi prenos podataka.

Nedostaci:

- Fizička povezanost kablom → manje fleksibilnosti, više kablova po kancelariji.
- Teže proširenje mreže bez dodatnih kablova ili switcheva.

2 Wi-Fi mreža

Prednosti:

1. **Fleksibilnost i mobilnost:** Uređaji se mogu povezivati bez kablova.
2. **Lako proširenje mreže:** Novi uređaji se jednostavno priključuju na mrežu.
3. **Brza instalacija:** Nema potrebe za postavljanjem kablova kroz zidove ili podove.

Nedostaci:

- Niža brzina prenosa (100 Mbps vs 1 Gbps).
- Smetnje od zidova, drugih uređaja ili elektromagnetnih izvora → gubitak signala.
- Veća latencija i mogućnost zagušenja ako više uređaja istovremeno koristi mrežu.

3 Ključne razlike koje utiču na odluku

Razlika	Ethernet	Wi-Fi	Uticaj na odluku
Brzina prenosa	1 Gbps	100 Mbps	Za firme sa stalnim i većim prenosom podataka, Ethernet je brži i pouzdaniji.
Stabilnost i smetnje	Minimalne smetnje	Podložan smetnjama	Ethernet garantuje pouzdan prenos bez prekida, što je bitno za kritične aplikacije.
Mobilnost	Fiksna lokacija	Uređaji se mogu kretati	Wi-Fi je zgodan za fleksibilnost, ali može biti spor i nestabilan kod više uređaja.

Zaključak

Za malu firmu koja želi **stabilnu i brzu mrežu za više uređaja**, najbolje je **Ethernet**.

Wi-Fi se može koristiti kao dopuna za mobilne uređaje ili goste, ali **primarna mreža za posao treba biti kablovska**.

Firma razmatra postavljanje mreže u svojim kancelarijama i mora odlučiti između Ethernet mreže i Wi-Fi mreže. Odlučite se za jednu od opcija i obrazložite svoj izbor. Takođe, obavite sljedeće računicе kako biste podržali svoju odluku:

Pretpostavke:

- Ethernet mreža: 1 Gbps brzina prenosa.
- Wi-Fi mreža: 100 Mbps brzina prenosa.
- Ukupno 10 uređaja (računara i pametnih telefona) treba biti povezano na mrežu.
- Svaki uređaj prenosi podatke neprekidno 8 sati dnevno.

Izračunati:

1. Koliko vremena je potrebno da se prenese 1 GB podataka na Ethernet mreži?
2. Koliko vremena je potrebno da se prenese 1 GB podataka na Wi-Fi mreži?
3. Koliko ukupno prometa generišu svi uređaji za 8 sati neprekidnog rada na Ethernetu?
4. Koliko ukupno prometa generišu svi uređaji za 8 sati neprekidnog rada na Wi-Fi-u?

Pretpostavke

- Ethernet: 1 Gbps = $1 \cdot 10^9$ bit/s
- Wi-Fi: 100 Mbps = $100 \cdot 10^6$ bit/s
- 1 GB = 1 GB = $1 \cdot 10^9$ bajtova = $8 \cdot 10^9$ bita
- Broj uređaja: 10
- Rad po uređaju: 8 h = $8 \cdot 3600 = 28800$ s

1 Vreme prenosa 1 GB na Ethernet mreži

$$t = \frac{\text{veličina podataka u bitovima}}{\text{brzina prenosa}}$$

$$t_{\text{Ethernet}} = \frac{8 \cdot 10^9}{1 \cdot 10^9} = 8 \text{ s}$$

✓ Odgovor: 8 sekundi

2 Vreme prenosa 1 GB na Wi-Fi mreži

$$t_{\text{Wi-Fi}} = \frac{8 \cdot 10^9}{100 \cdot 10^6} = 80 \text{ s}$$

✓ Odgovor: 80 sekundi

3 Ukupan promet svih uređaja na Ethernetu za 8 sati

Promet po uređaju:

$$\text{Promet po uređaju} = R \cdot t = 1 \cdot 10^9 \cdot 28800 = 2.88 \cdot 10^{13} \text{ bita}$$

Za 10 uređaja:

$$\text{Ukupni promet} = 10 \cdot 2.88 \cdot 10^{13} = 2.88 \cdot 10^{14} \text{ bita}$$

U bajtovima:

$$2.88 \cdot 10^{14} / 8 = 3.6 \cdot 10^{13} \text{ B} \approx 36 \text{ TB}$$

✓ Odgovor: ~36 TB



4 Ukupan promet svih uređaja na Wi-Fi-u za 8 sati

Promet po uređaju:

$$R = 100 \cdot 10^6 \text{ b/s}$$

$$\text{Promet po uređaju} = 100 \cdot 10^6 \cdot 28800 = 2.88 \cdot 10^{12} \text{ bita}$$

Za 10 uređaja:

$$10 \cdot 2.88 \cdot 10^{12} = 2.88 \cdot 10^{13} \text{ bita} = 3.6 \cdot 10^{12} \text{ B} \approx 3.6 \text{ TB}$$

✓ Odgovor: ~3.6 TB

Zaključak i izbor

- **Ethernet prednosti:**
 - Brži prenos (1 Gbps vs 100 Mbps)
 - Stabilniji i pouzdaniji (manje smetnji)
 - Veća mogućnost simultanog prenosa podataka
- **Wi-Fi prednosti:**
 - Bez kablova → fleksibilnost
 - Lako proširivanje mreže

Za 10 uređaja koji prenose podatke kontinuirano 8 sati dnevno, Ethernet je bolja opcija jer Wi-Fi od 100 Mbps ne može da zadovolji velike zahteve za prometom i postoji rizik od zagušenja mreže.

Imate Ethernet switch sa osam priključaka (Port 1 do Port 8) i pet računarskih uređaja (Uređaj A, Uređaj B, Uređaj C, Uređaj D, Uređaj E) povezanih sa switch-em. Na početku, tablica svičovanja je prazna. Pomoću sledećih informacija i koraka, popunite tablicu svičovanja za switch:

1. Uređaj A šalje okvir podataka na switch.
 - MAC adresa Uređaja A: 00:11:22:33:44:55
 - Switch uči MAC adresu Uređaja A i zabeleži da je na Portu 1.
2. Uređaj B šalje okvir podataka na switch.
 - MAC adresa Uređaja B: 11:22:33:44:55:66
 - Switch uči MAC adresu Uređaja B i zabeleži da je na Portu 2.
3. Uređaj C šalje okvir podataka na switch.
 - MAC adresa Uređaja C: 22:33:44:55:66:77
 - Switch uči MAC adresu Uređaja C i zabeleži da je na Portu 3.
4. Uređaj D šalje okvir podataka na switch.
 - MAC adresa Uređaja D: 33:44:55:66:77:88
 - Switch uči MAC adresu Uređaja D i zabeleži da je na Portu 4.
5. Uređaj E šalje okvir podataka na switch.
 - MAC adresa Uređaja E: 44:55:66:77:88:99
 - Switch uči MAC adresu Uređaja E i zabeleži da je na Portu 5.
6. Uređaj A sada šalje drugi okvir podataka na switch.
7. Uređaj C sada šalje drugi okvir podataka na switch.

Nakon ovih koraka, popunite tablicu svičovanja za switch i navedite koja MAC adresa je zabeleži za svaki priključak.

Osnovna logika

Ethernet svič radi tako što:

1. Uči (upisuje) MAC adresu svakog uređaja koji mu pošalje okvir → u tabelu stavi par *MAC adresa – port*.
2. Kad sledeći put dobije okvir, pogleda **odredišnu MAC adresu**:
 - ako je **zna**, pošalje okvir **tačno na taj port**,
 - ako je **ne zna**, šalje okvir na **sve portove osim ulaznog** (tzv. *flooding*).

Dati podaci

Uređaj	MAC adresa	Port
A	00:11:22:33:44:55	1
B	11:22:33:44:55:66	2
C	22:33:44:55:66:77	3
D	33:44:55:66:77:88	4
E	44:55:66:77:88:99	5

Koraci i učenje tabele

1 Uređaj A šalje okvir

→ Switch nauči da se 00:11:22:33:44:55 nalazi na portu 1.

2 Uređaj B šalje okvir

→ Switch nauči da je 11:22:33:44:55:66 na portu 2.

3 Uređaj C šalje okvir

→ Switch nauči da je 22:33:44:55:66:77 na portu 3.

4 Uređaj D šalje okvir

→ Switch nauči da je 33:44:55:66:77:88 na portu 4.

5 Uređaj E šalje okvir

→ Switch nauči da je 44:55:66:77:88:99 na portu 5.

6 Uređaj A šalje drugi okvir

→ Switch već zna da se A nalazi na portu 1 → osvežava vreme važenja unosa (TTL), ali ne menja ništa.

7 Uređaj C šalje drugi okvir

→ Switch već zna da se C nalazi na portu 3 → osvežava TTL, tabela ostaje ista.



✓ Konačna tabela svičovanja

Port	MAC adresa
1	00:11:22:33:44:55
2	11:22:33:44:55:66
3	22:33:44:55:66:77
4	33:44:55:66:77:88
5	44:55:66:77:88:99

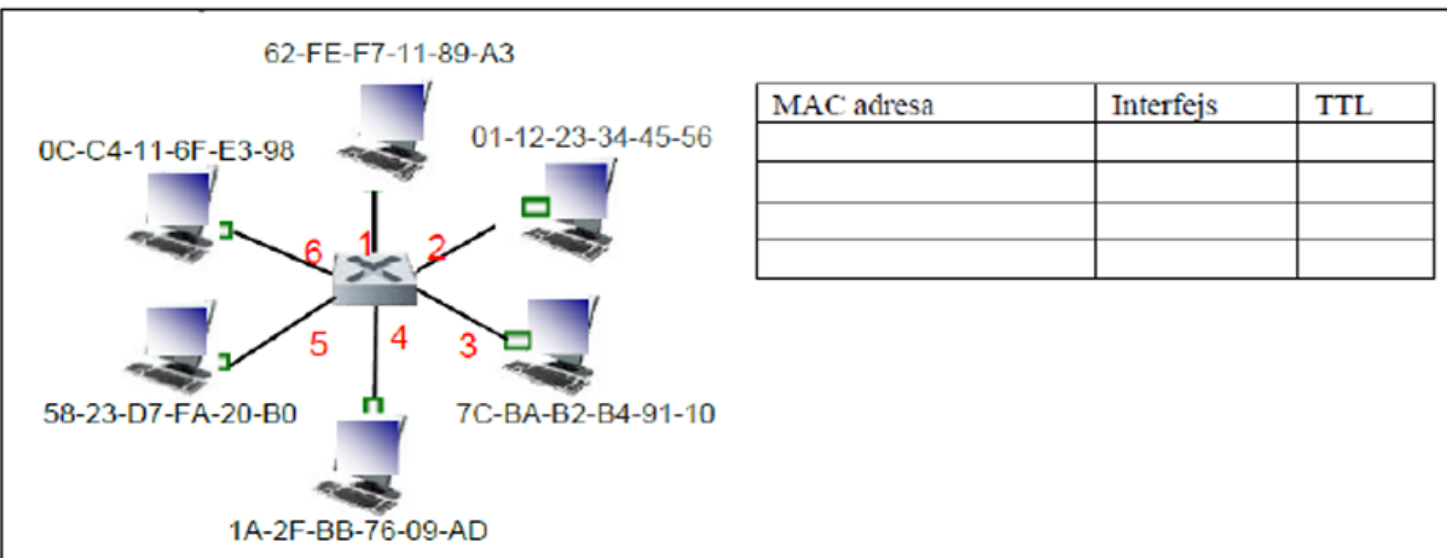
🔍 Kratko objašnjenje za usmeni:

Switch beleži MAC adrese uređaja na osnovu izvora svakog primljenog okvira.

Pošto svaki uređaj (A–E) šalje okvir sa svog porta, switch uči svih pet MAC adresa i pamti ih uz odgovarajuće portove.

Kad A i C ponovo šalju okvire, switch već zna gde se oni nalaze i samo osvežava svoje unose.

Za svič i hostove prikazane na slici, kao i trenutno stanje tabele svičovanja, popuniti odgovarajuća prazna polja smatrajući da je TTL=60. Objasniti šta se dešava (gde se šalju okviri) nakon sledećih koraka: okvir sa odredišnom MAC adresom 58-23-D7-FA-20-B0 stiže do sviča preko interfejsa 6, okvir sa odredišnom MAC adresom 1A-2F-BB-76-09-AD stiže do sviča preko interfejsa 1, a odmah nakon toga okvir sa odredišnom MAC adresom 62-FE-F7-11-89-A3 stiže do sviča preko interfejsa 6.



Dakle imamo jedan svič (switch) sa 6 portova, i 6 hostova (1–6) koji su povezani na te portove.
Znači:

Port	MAC adresa hosta
1	62-FE-F7-11-89-A3
2	01-12-23-34-45-56
3	7C-BA-B2-B4-91-10
4	1A-2F-BB-76-09-AD
5	58-23-D7-FA-20-B0
6	0C-C4-11-6F-E3-98

Objašnjenje principa

Svič ima tabelu svičovanja (MAC adresnu tabelu) gde upisuje koje MAC adrese je naučio i na kom portu su.

- Kad okvir dođe, svič zabeleži izvorni port i izvorni MAC u tabelu.
- Onda pogleda odredišnu MAC:
 - Ako je zna → šalje okvir samo na taj port.
 - Ako je ne zna → **broadcastuje** na sve portove osim onog s kojeg je okvir došao.

TTL (ovde 60 s) znači da posle tog vremena unos ističe i briše se.

Dato u zadatku:

Već postoji tabela sviča sa 2 unosa (to znači da on već zna dve MAC adrese i na kojim portovima su).

Ti unosi nisu napisani, ali iz tvojih rečenica sledi da ćemo sad pratiti tri događaja i gledati kako se tabela menja.

Korak 1

→ Okvir sa odredišnom MAC = 58-23-D7-FA-20-B0 (to je Host 5) stiže do sviča preko interfejsa 6.

Šta svič radi:

- Prvo, svič vidi izvorni MAC tog okvira (nije dat, ali pretpostavljamo da je to MAC od Host 6 jer okvir dolazi s porta 6).
→ Znači svič nauči da se 0C-C4-11-6F-E3-98 nalazi na portu 6 i to upiše u tabelu.
- Pogleda odredišnu MAC (58-23-D7-FA-20-B0) —
ako je nema u tabeli (što verovatno nema jer je prvi događaj),
→ šalje okvir na sve portove osim 6 (broadcast).
→ TTL za 0C-C4-11-6F-E3-98 je 60s.

✓ Posle prvog koraka, tabela ima:

MAC adresa	Port	TTL
0C-C4-11-6F-E3-98	6	60

Korak 2

→ Okvir sa odredišnom MAC = 1A-2F-BB-76-09-AD (to je Host 4) stiže do sviča preko interfejsa 1.

- Izvorni MAC (verovatno od Host 1): 62-FE-F7-11-89-A3
→ svič upisuje 62-FE-F7-11-89-A3 → port 1, TTL = 60.
- Odredišna MAC 1A-2F-BB-76-09-AD: ako nije u tabeli, svič šalje okvir na sve portove osim 1.

✓ Posle drugog koraka, tabela ima:

MAC adresa	Port	TTL
0C-C4-11-6F-E3-98	6	60
62-FE-F7-11-89-A3	1	60

Korak 3

➔ Okvir sa **odredišnom MAC = 62-FE-F7-11-89-A3** (to je Host 1) stiže do sviča preko interfejsa 6.

- Izvorni MAC (opet od Host 6): 0C-C4-11-6F-E3-98
→ već postoji u tabeli, svič **osvežava TTL = 60**.
- Odredišna MAC 62-FE-F7-11-89-A3: svič zna da je na portu 1.
→ Okvir se **prosleđuje samo portu 1**.

✓ Konačna tabela:

MAC adresa	Port	TTL
0C-C4-11-6F-E3-98	6	60
62-FE-F7-11-89-A3	1	60

NOVA

LEKCIJA

Датиграм је јединица преноса (или јединица података, бобе рети) у безвезној мрежи, у IP протоколи.
То је самосталан пакет података који носи: изворичну IP адресу, одредишну IP адресу, информацију.
Сваки датиграм се третира независно од других. Не гарантује исправност, испоруку, редослед.
Заглавље датиграма се састоји од: TTL (Time to Live), изворична адреса, одредишна адреса, информација. То је главно. Поред тога, има и: верзија IP протокола, протокол, дужица пакета. Дубе:
Особине датиграма су те да је безвезан, јер не прави претходну везу између пошиљача и прила-
оца. Неповезан је, јер пакет можда неће стићи или можда стићи у погрешном редоследу. Рутери
третирају сваки датиграм засебно, могући су различити путеви. Примери употребе су протоколи:
IP, UDP, ICMP. Датиграм је основни грађевински блок IP комуникације. То је зато што сваки
пакет носи све што је потребно да стигне од пошиљача до прилаоца, без претходне успоставке

Виртуелна кола представљају начин за пренос података од изворног до одређеног, при чему је напред одређен редослед слања и испорука је загарантована. Постоји такође и време које је потребно да се успостави веза (типа неколико секунди). То је речимо мана, јер може да на пример, изрокује кашњење. Мана је такође и то што ако се кола прекину, све мора поново да се успостави. Ово није баш флексибилан начин за пренос података. Пренос је лако пратити. Ипак би можда најбоље било да запишемо предности и мане виртуелних кола у једној табели:

Предности: Редослед пакета је загарантован. Познатији пренос. Мрежа може пратити грешке.

Мане: Мање флексибилно. Потребно је успостављање везе (стазе) пре слања података. Ако веза падне, мора се поново успоставити.

Виртуелна кола су логички пут између пошилача и пријемца у мрежи. Не морају да физички постоје - ово је више концепт за организацију преноса података. Могућавају повезан и организован пренос података кроз мрежу, чак и када се користи безvezна инфраструктура.

Једноставна веза: Пре него што се подаци пошљу, мрежа резервише логичку путању између 2 хоста. Ово је слично ТСП вези у транспортном слоју, само ово је на мрежном слоју. Пренос података: Сваки пакет се шаље кроз виртуелну руту која је мрежа претходно одредила. Пакети стичу редослед и мрежа може pratити грешке и ретрансмисију. Када се комуникација заврши, виртуелна кола се затварају. Мрежа ослобађа ресурсе које је користила.

Pretpostavite da imate mrežu koja se sastoji od tri računara (A, B, C) i mrežnih uređaja za usmeravanje. U mreži se koristi i model datagrama i model virtuelnih krugova za usmeravanje podataka. Svaki uređaj šalje 5 paketa prema drugim uređajima.

1. Koristeći model datagrama, navedite kako bi paketi iz uređaja A došli do uređaja C, iako ne postoji unapred uspostavljena staza.
2. Koristeći model virtuelnih kola, navedite kako bi paketi iz uređaja A došli do uređaja C nakon što je uspostavljena virtualna staza.
3. Razmislite o prednostima i nedostacima oba modela usmeravanja podataka u ovom scenariju.

1 Model datagrama

- Opis:

U modelu datagrama, **nema unapred uspostavljene staze**. Svaki paket se tretira **nezavisno**, i ruteri odlučuju gde će ga poslati na osnovu **destination IP adrese**.

- Kako paketi putuju od A do C:

1. Računar A šalje prvi paket prema C.
2. Prvi ruter prima paket, čita **destination IP** i odlučuje sledeći hop ka C.
3. Paket se prosleđuje kroz sledeći ruter dok ne stigne do C.
4. Svaki od pet paketa ide nezavisno – moguće je da koriste **različite puteve** kroz mrežu.

💡 Ključ:

- Svaki paket je **samostalan**
- **Redosled dolaska paketa nije garantovan**
- Pogodno za mreže gde se veze teško održavaju ili gde rute mogu da se menjaju

2 Model virtuelnih kola

- Opis:
U modelu virtuelnih kola, pre nego što se pošalju podaci, mreža uspostavlja logičku stazu (virtual circuit) između pošiljaoca i primaoca.
- Kako paketi putuju od A do C:
 1. Pre slanja paketa, A inicira uspostavljanje virtuelne staze ka C.
 2. Ruteri dogovaraju i rezervišu putanju (svi paketi će ići istom rutom).
 3. Svaki od 5 paketa koristi ovu istovetnu virtuelnu stazu i stiže do C u istom redosledu.

💡 Ključ:

- Postoji predvidljiv put
- Redosled paketa je zagarantovan
- Mreža može pratiti greške i retransmisiju

3 Prednosti i nedostaci

Model	Prednosti	Nedostaci
Datagram	- Fleksibilno, paketima se lako menja ruta - Nema potrebe za prethodnom rezervacijom resursa - Robusno u mrežama sa promenljivim topologijama	- Redosled paketa nije zagarantovan - Nema garancije da paket stigne - Veća složenost za aplikacije koje zahtevaju pouzdanost
Virtuelna kola	- Redosled paketa je zagarantovan - Pouzdaniji prenos - Mreža može pratiti greške i retransmisiju	- Potrebno je uspostavljanje staze pre slanja podataka - Manje fleksibilno - Ako veza padne, mora se ponovo uspostaviti

Pretpostavimo da imate mrežu sa dva računara (A i B) i dva mrežna uređaja za usmeravanje (Router 1 i Router 2). Svaki uređaj može poslati podatke brzinom prenosa od 1 Mbps. Potrebno je poslati paket od 1 000 000b. U mreži koristite i model datagrama i model virtuelnih kola za usmeravanje podataka.

Zamislite da svaki uređaj A i B šalje 20 paketa podataka drugom uređaju. Kako bi paketi stigli od uređaja A do uređaja B, razmotrite sledeće:

1. U modelu datagrama, svaki paket putuje nezavisno prema odredištu.
2. U modelu virtuelnih kola, prvo se uspostavlja virtualna staza između uređaja A i B pre slanja paketa. Uspostavljanje staze između uređaja traje 5s.

Pronađite ukupno vreme potrebno za prenos svih paketa od uređaja A do uređaja B za oba modela usmeravanja. Koja je razlika u vremenu prenosa između ova dva modela?

Zadatak

- Dva računara: A i B
- Dva rutera: Router 1 i Router 2
- Brzina prenosa: 1 Mbps
- Paket: 1 000 000 bitova (1 Mb)
- Svaki uređaj šalje 20 paketa
- Modeli: Datagram i Virtuelna kola
- Vreme uspostavljanja virtuelne staze: 5 s

1 Pretvaranje brzine prenosa i veličine paketa u vreme

Brzina prenosa:

$$1 \text{ Mbps} = 1\,000\,000 \text{ b/s}$$

Vreme prenosa jednog paketa:

$$\text{vreme} = \frac{\text{veličina paketa (bit)}}{\text{brzina (b/s)}} = \frac{1\,000\,000}{1\,000\,000} = 1 \text{ s po paketu}$$

2 Ukupno vreme u modelu datagrama

- Svaki paket putuje nezavisno.
- Pretpostavimo da se paketi šalju redosledom, jedan po jedan, kroz mrežu.
- Ukupno paketa: 20

$$t_{\text{datagram}} = 20 \times 1 \text{ s} = 20 \text{ s}$$

💡 Napomena: Ako bismo računali paralelno kroz dva rutera, vreme može biti kraće, ali standardno se računa linearno po paketu.

3 Ukupno vreme u modelu virtuelnih kola

- Prvo se uspostavlja staza: 5 s
- Zatim se šalju paketi, po 1 s po paketu
- Ukupno paketa: 20

$$t_{\text{virtuelna kola}} = 5 \text{ s (uspostavljanje)} + 20 \times 1 \text{ s} = 25 \text{ s}$$

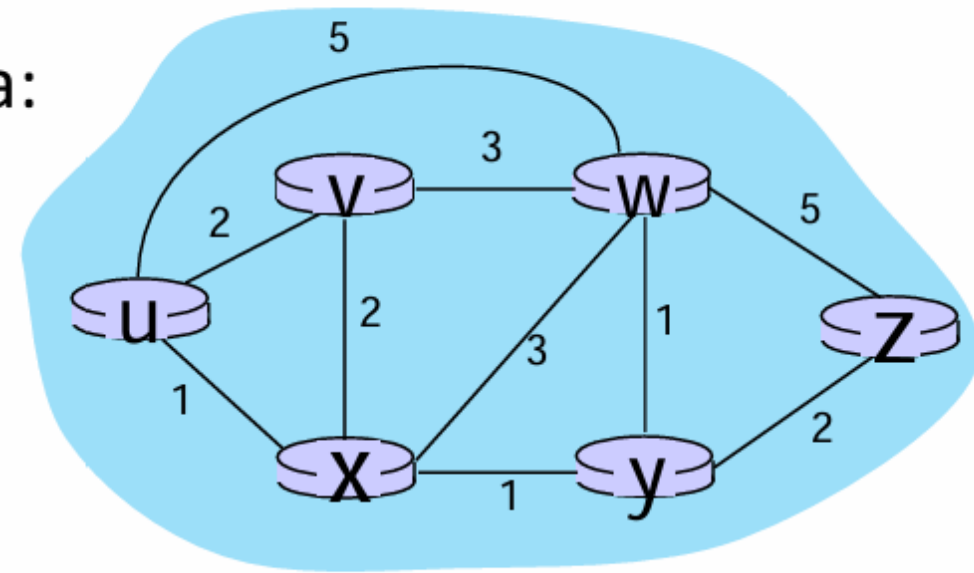
4 Razlika u vremenu prenosa

$$\Delta t = t_{\text{virtuelna kola}} - t_{\text{datagram}} = 25 \text{ s} - 20 \text{ s} = 5 \text{ s}$$

Zadatak 58 :

Dat vam je sledeći težinski usmereni graf čvorova:

Graf se sastoji od šest čvorova (u, x, v, w, y i z) i deset grana sa težinama kao što je prikazano na grafu. Početni čvor je čvor u.



Primenite Dijkstrin algoritam kako biste pronašli najkraći put od čvora u do svakog drugog čvora u grafu i izračunali ukupne udaljenosti. Takođe, nacrtajte stablo najkraćih puteva (Dijkstrino stablo) za ovaj graf.

Da bismo rešili ovaj zadatak koristeći **Dijkstrin algoritam**, pratićemo korak po korak udaljenosti od početnog čvora u do svih ostalih čvorova.

1. Inicijalizacija

- Postavljamo udaljenost do početnog čvora $d(u) = 0$.
- Sve ostale udaljenosti postavljamo na beskonačno (∞).
- Skup posvećenih čvorova $S = \emptyset$.

Čvor	u	x	v	w	y	z
Udaljenost	0	∞	∞	∞	∞	∞

2. Koraci algoritma

Korak 1: Biramo čvor u (najmanja udaljenost 0)

- Susedi čvora u su v (težina 2), x (težina 1) i w (težina 5).
- Ažuriramo: $d(v) = 2$, $d(x) = 1$, $d(w) = 5$.
- $S = \{u\}$

Korak 2: Biramo čvor x (najmanja udaljenost 1)

- Susedi čvora x su v (težina 2), y (težina 1) i w (težina 3).
- Proveravamo nove putanje preko x :
 - Do v : $1 + 2 = 3$ (Veće od trenutnih 2, ne menjamo).
 - Do y : $1 + 1 = 2$ (Novo: 2).
 - Do w : $1 + 3 = 4$ (Bolje od trenutnih 5, menjamo na 4).
- $S = \{u, x\}$

Korak 3: Biramo čvor v ili y (oba imaju udaljenost 2, biramo v ili y proizvoljno)

- Izbor v : Njegovi susedi su x i w .
 - Do w : $2 + 3 = 5$ (Veće od trenutnih 4, ne menjamo).
- Izbor y : Njegov susjed je z (težina 2) i w (težina 1).
 - Do z : $2 + 2 = 4$ (Novo: 4).
 - Do w : $2 + 1 = 3$ (Bolje od trenutnih 4, menjamo na 3).
- $S = \{u, x, v, y\}$

Korak 4: Biramo čvor w (najmanja udaljenost 3)

- Susedi su y (već u S) i z (težina 5).
 - Do z : $3 + 5 = 8$ (Veće od trenutnih 4, ne menjamo).
- $S = \{u, x, v, y, w\}$

Korak 5: Biramo čvor z (najmanja udaljenost 4)

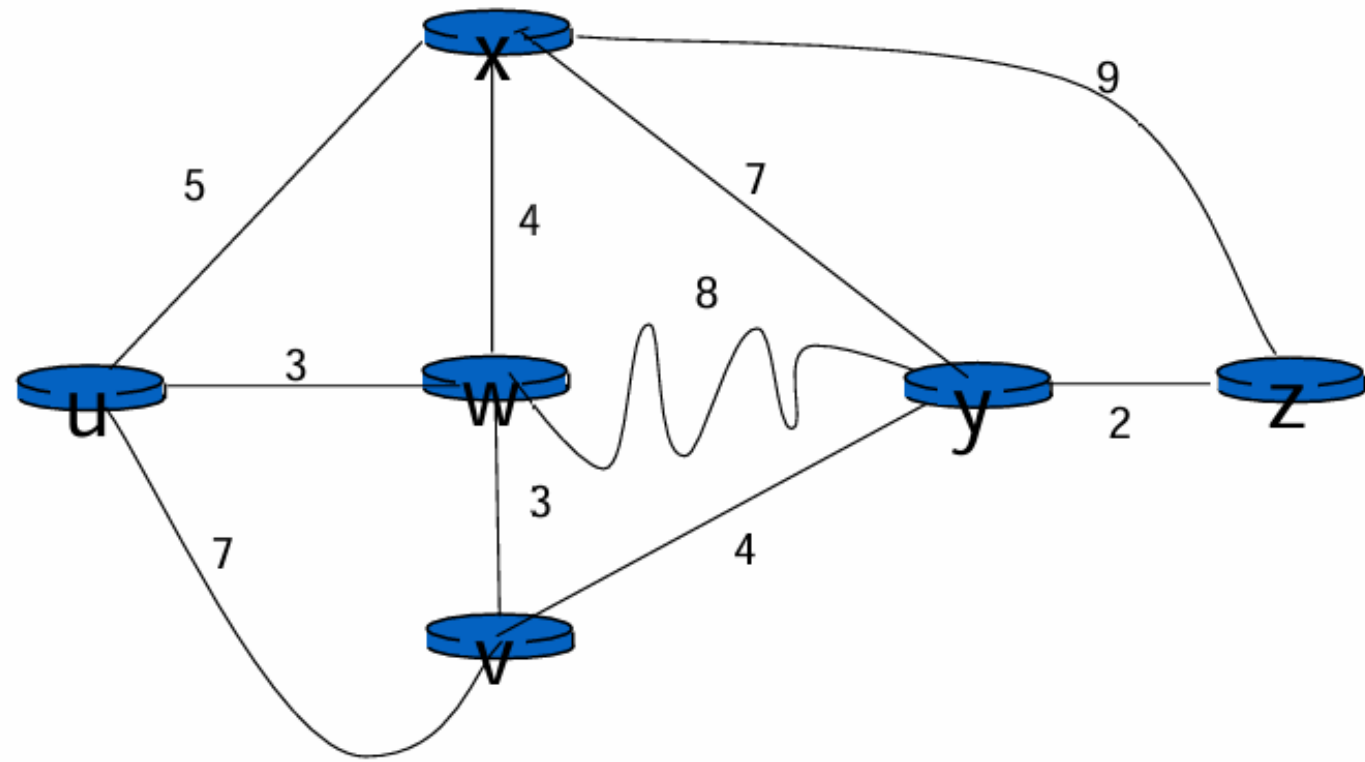
- Svi čvorovi su sada procesuirani.
-

3. Konačne udaljenosti i putanje

Čvor	Ukupna udaljenost	Putanja od u
u	0	-
x	1	$u \rightarrow x$
v	2	$u \rightarrow v$
y	2	$u \rightarrow x \rightarrow y$
w	3	$u \rightarrow x \rightarrow y \rightarrow w$
z	4	$u \rightarrow x \rightarrow y \rightarrow z$

Dat vam je sledeći težinski usmereni graf čvorova:

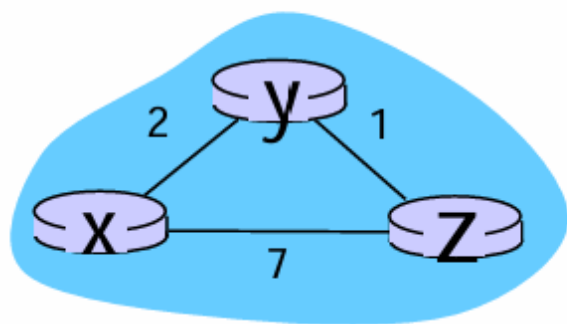
Graf se sastoji od šets čvorova (u, x, v, w, y i z) i deset grana sa težinama kao što je prikazano na grafu. Početni čvor je čvor u.



Primenite Dijkstrin algoritam kako biste pronašli najkraći put od čvora u do svakog drugog čvora u grafu i izračunali ukupne udaljenosti. Takođe, nacrtajte stablo najkraćih puteva (Dijkstrino stablo) za ovaj graf.

Бирамо најјефтинију путању од u . Она гласи: $u \rightarrow w = 3$. Сада, из w имамо 3 пута.
 $u \rightarrow w \rightarrow x = 7$ $u \rightarrow w \rightarrow v = 6$ $u \rightarrow w \rightarrow y = 11$. Међутим, има ту и директних
путања. Упоредимо: $u \rightarrow w \rightarrow x > u \rightarrow x$, што значи да бирамо $u \rightarrow x$. То је 5.
 $u \rightarrow w \rightarrow v < u \rightarrow v$, што значи да бирамо $u \rightarrow w \rightarrow v$. Значи, за сада видимо путеве,
али само за ова 3: $u \rightarrow x = 5$, $u \rightarrow w = 3$, $u \rightarrow w \rightarrow v = 6$. Сада треба да решимо за
 y . ~~На основу претходних~~ На основу претходних релација, видимо која нам је оптимална
путања. То је: $u \rightarrow w \rightarrow v \rightarrow y = 10$. То је најјефтиније. Након ануирања имамо:
 $u \rightarrow x = 5$, $u \rightarrow w = 3$, $u \rightarrow w \rightarrow v = 6$, $u \rightarrow w \rightarrow v \rightarrow y = 10$. За z је очигледно да иде:
 $u \rightarrow w \rightarrow v \rightarrow y \rightarrow z = 12$.

Dati su vam sledeći ruteri (x, y, z) i povezane grane sa težinama (udaljenostima) između njih:



Svaka grana u grafu ima svoju težinu, kao što je prikazano na grafu.

Primenite algoritam rutiranja zasnovan na vektoru razdaljine na ovaj graf kako biste odredili najkraće puteve od svakog rutera do svih drugih rutera.

1. Početno stanje (Svako zna samo za sebe i komšije)

U prvom trenutku, ruter zna cenu do komšije, a za sve ostale misli da su beskonačno daleko.

- Ruter x vidi: do y košta 2, do z košta 7.
- Ruter y vidi: do x košta 2, do z košta 1.
- Ruter z vidi: do x košta 7, do y košta 1.

2. Razmena informacija (Glavni trik)

Sada ruter x dobije informaciju od rutera y koji mu kaže: "Ej, ja do z mogu da stignem za 1".

Ruter x onda računa:

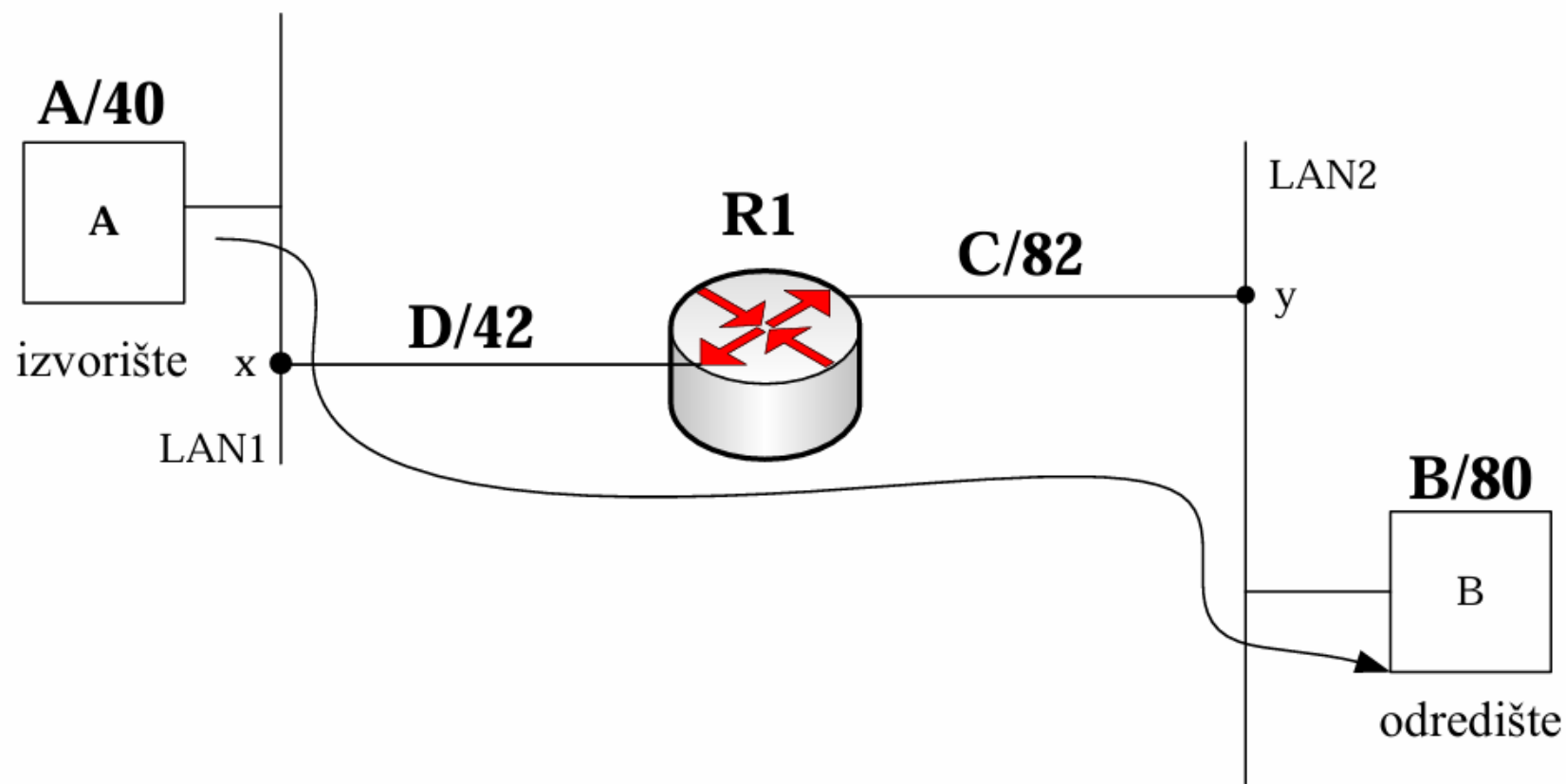
- Direktan put do z mi je 7.
- Put preko y je: (cena do y) + (y-ova cena do z) = $2 + 1 = 3$.
- **Zaključak:** Pošto je $3 < 7$, ruter x ažurira svoju tabelu i kaže: "Od sada za z idem preko y!"

3. Konačne tabele rutiranja (Rezultat)

Kada se sve smiri (konvergencija), najkraći putevi u tvojoj mreži su:

Od rutera	Do rutera	Najkraći put	Cena
x	y	direktno	2
x	z	preko y (2 + 1)	3
y	x	direktno	2
y	z	direktno	1
z	x	preko y (1 + 2)	3
z	y	direktno	1

Na slici računar A predaje poruku računar B preko LAN-a LAN1, rutera R1, i LAN-a LAN2. Prikazati sadržaje paketa i okvira na mrežnom nivou i na nivou veze za svaki interfejs (tačke x i y).



📌 Tumačenje oznaka sa slike

- A/40 → računar A, izvorni port 40
- B/80 → računar B, odredišni port 80
- D/42 i C/82 → interfejsi rutera R1 (svaki ima svoj MAC/IP)
- LAN1 i LAN2 → različiti linkovi (različite MAC adrese!)

⚠ Bitno:

Portovi (40 → 80) se NE menjaju, isto kao IP adrese.

1 Tačka x (LAN1 – između A i R1)

👤 Aplikacioni / transportni nivo

- Source port: 40
- Destination port: 80

🌐 Mrežni nivo (IP paket)

- Source IP: IP(A)
- Destination IP: IP(B)

📌 IP paket je isti kroz celu mrežu.

🧱 Nivo veze (Ethernet okvir – LAN1)

- Source MAC: MAC(A)
- Destination MAC: MAC(D)
(interfejs rutera R1 na LAN1)

👉 Zato što je ruter sledeći hop.

2 Šta radi ruter R1 (između x i y)

- Skida Ethernet okvir (LAN1)
- Gleda IP odredište
- Ne dira:
 - IP adrese
 - portove
- Pravi nov Ethernet okvir za LAN2

3 Tačka y (LAN2 – između R1 i B)

💻 Aplikacioni / transportni nivo

- Source port: 40
- Destination port: 80

🔄 isto kao na početku

🌐 Mrežni nivo (IP paket)

- Source IP: IP(A)
- Destination IP: IP(B)

🔄 nepromenjeno

🧱 Nivo veze (Ethernet okvir – LAN2)

- Source MAC: MAC(C)
(interfejs rutera R1 na LAN2)
- Destination MAC: MAC(B)

👉 Sad je B direktno povezan → njegova MAC adres





Kompletna tabela (ovo profesorka OBOŽAVA)

Tačka	Nivo	Source	Destination
x	Port	40	80
x	IP	IP(A)	IP(B)
x	MAC	MAC(A)	MAC(D)
y	Port	40	80
y	IP	IP(A)	IP(B)
y	MAC	MAC(C)	MAC(B)



Zaključak (idealna završna rečenica)

Tokom prenosa poruke od računara A do računara B, portovi i IP adrese ostaju nepromenjeni jer identifikuju krajnje aplikacije i uređaje, dok se MAC adrese menjaju na svakom LAN segmentu jer služe za lokalni prenos okvira između susednih mrežnih uređaja.



- Portovi (npr. 40 → 80)
 - 👉 pripadaju transportnom sloju
 - 👉 NE menjaju se tokom prenosa
 - IP adrese
 - 👉 pripadaju mrežnom sloju
 - 👉 NE menjaju se dok paket prolazi kroz rutere
 - IP paket
 - 👉 sadrži IP adrese + podatke
 - 👉 NEMA MAC adresu
 - MAC adrese
 - 👉 pripadaju nivou veze (Ethernet okvir)
 - 👉 važe samo na jednom LAN-u
 - Ruter
 - 👉 NE menja IP paket
 - 👉 ODBACUJE stari Ethernet okvir
 - 👉 PRAVI NOVI Ethernet okvir sa novim MAC adresama
- 📌 ZLATNA REČENICA:
- MAC adrese se ne menjaju u paketu — jer paket nema MAC adresu.
Menja se MAC adresa Ethernet okvira.*



Jedna rečenica za ispit (zapiši je mentalno 📌)

Ruter ne menja IP adrese ni portove, ali menja Ethernet okvir, pa se zato menjaju MAC adrese na svakom linku.

Kratak, čist odgovor 🙌

- 🙌 Menja se MAC adresa ETHERNET OKVIRA.
- 📦 Paket (IP paket) uopšte nema MAC adresu.

Malo sporije, ali kristalno jasno 🧠

📦 IP paket (mrežni sloj)

- Sadrži:
 - IP izvor (A)
 - IP odredište (B)
- ❌ NEMA MAC adrese
- ❌ Ruter ga ne menja

🧱 Ethernet okvir (nivo veze)

- Sadrži:
 - MAC izvor
 - MAC odredište
- ✔️ Važi samo za jedan LAN
- ✔️ Ruter ga uvek menja

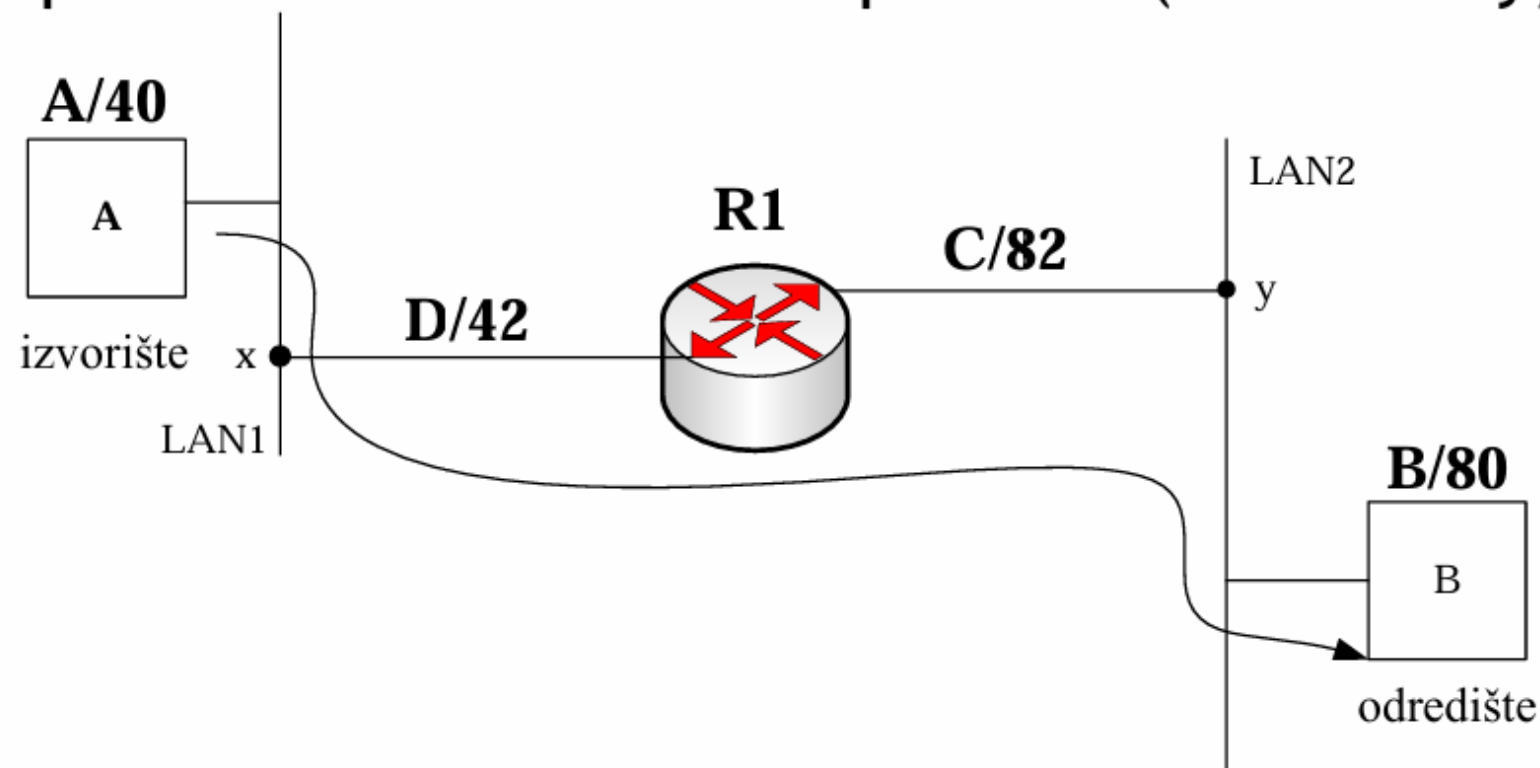
Šta tačno ruter radi? 🔄

- 1 primi Ethernet okvir
- 2 odbaci okvir (MAC adrese nestaju)
- 3 zadrži IP paket
- 4 napravi NOVI Ethernet okvir
- 5 dodeli nove MAC adrese za sledeći link

Zato je ova rečenica ZLATNA 🏆

MAC adrese ne pripadaju paketu, već Ethernet okviru.

Na slici prikazana je komunikacija između procesa koji se izvršava na računaru A sa port adrese i i procesa koji se izvršava na računaru B sa port adrese j . Prikazati sadržaje paketa i okvira na mrežnom nivou, nivou veze i transportnom nivou za svaki preskok (tačke x i y).



Tačka X (na linku LAN1)

U ovoj tački, podatak tek kreće od računara A ka ruteru (interfejs D).

- **Transportni nivo (Portovi):**

- Izvorišni: i
- Odredišni: j

- **Mrežni nivo (IP adrese):**

- Izvorišni IP: 40 (adresa računara A)
- Odredišni IP: 80 (adresa računara B)

- **Sloj veze (MAC adrese):**

- Izvorišni MAC: A
- Odredišni MAC: D (adresa 42 - ulaz u ruter)

Zašto je ovo tvoje objašnjenje na slici važno? Zato što te sprečava da napraviš najčešću grešku — da pokušaš da upišeš MAC adresu u IP paket. Kao što si sam napisao: IP paket NEMA MAC adresu. On se samo "pakuje" u Ethernet okvir koji te adrese ima.

Tačka Y (na linku LAN2)

Ruter R1 je primio okvir, odbacio stari Ethernet omotač i napravio potpuno novi za drugi deo puta.

- **Transportni nivo (Portovi):**
 - Izvorišni: *i* (NE MENJA SE)
 - Odredišni: *j* (NE MENJA SE)
- **Mrežni nivo (IP adrese):**
 - Izvorišni IP: 40 (NE MENJA SE)
 - Odredišni IP: 80 (NE MENJA SE)
- **Sloj veze (MAC adrese):**
 - Izvorišni MAC: **C** (adresa 82 - izlaz iz rutera ka LAN2)
 - Odredišni MAC: **B** (adresa 80 - konačno odredište)

Zašto je ovo tvoje objašnjenje na slici važno? Zato što te sprečava da napraviš najčešću grešku — da pokušaš da upišeš MAC adresu u IP paket. Kao što si sam napisao: IP paket NEMA MAC adresu. On se samo "pakuje" u Ethernet okvir koji te adrese ima.

$$17.12.70.1/27 \Leftrightarrow \text{subnet} = (255.255.255.224)_{10} = (11111111 \ 11111111 \ 11111111 \ 11100000)_2$$

Ок. Напишмо subnet. Сада, ако желимо да видимо колико адреса преостaje за хостове, потребно је да га комплементирамо. Добијамо: $(00000000 \ 00000000 \ 00000000 \ 00011111)_2$

Субнет и број адреса хостова су обрнуто пропорционални. Што је већи субнет, то је мање адреса за хостове и обрнуто. Када нам је фиксиран блок адреса, увек се полази да оне подреже (рецимо неки LAN) за који је потребно највише адреса, односно најмањи субнет.

На основу субнета можемо одредити број адреса, а потом тај број поделити са 256 да би добили подреке оне као и смислу да које до које адресе иду опсежи. То су блокови.

Ако у неком октету има n слободних битова, разлика између тог октета код крајње адресе блока и почетне адресе блока ће бити $2^n - 1$. Фиксирани октети су: /8, /16, /24 (дефинисан са 8).

Broadcast адреса се добија тако што се сви хост битови поставе на 1.

NOVA

LEKCIJA

Imate mrežni uređaj koji podržava Network Address Translation (NAT) i treba da ga konfigurirate za sledeći scenario:

1. Mrežni uređaj ima dva mrežna priključka (Interfejs A i Interfejs B). Interfejs A povezan je sa lokalnom mrežom (LAN) koja koristi privatni IP opseg 192.168.1.0/24. Interfejs B povezan je sa internetom i ima javnu IP adresu 203.0.113.1/24
2. Unutar lokalne mreže (LAN), postoji nekoliko uređaja koji koriste privatne IP adrese iz opsega 192.168.1.0/24.
3. Treba da omogućite uređajima iz lokalne mreže da pristupe internetu preko javne IP adrese interfejsa B.
4. Konfigurirate NAT na mrežnom uređaju tako da mapira privatne IP adrese uređaja iz LAN-a na javnu IP adresu interfejsa B prilikom pristupa internetu.
5. Navedite primer konfiguracije za NAT uređaja kako bi se omogućila ovakva funkcionalnost.

Napomena: Pretpostavimo da koristimo uređaj sa komandnom linijom za konfiguraciju.

```
interface GigabitEthernet 0/0
ip address 203.0.113.1 255.255.255.0
ip nat outside
interface GigabitEthernet 0/1
ip address 192.168.1.0 255.255.255.0
ip nat inside
access list 1 permit 192.168.1.0 0.0.0.255
ip nat inside source list 1 interface GigabitEthernet 0/0 overload
```



Treba da pise 192.168.1.1 umesto 192.168.1.0, ne mozemo dodeliti ruteru mrežnu adresu, glupo je. To bi morao da vodim racuna na ispitu da se ne zajebem slucajno.

interface GigabitEthernet 0/0 могу дешифровати овако: GigabitEthernet говори о максималној брзини порта. Ово значи да тај порт може да пренесе до 1 Gbps. А да је писало Ethernet, онда би брзина била 10 Mbps, а FastEthernet је 100 Mbps. Бројеви 0/0 су физичка локација порта на рутеру. Прва 0 означава слот на рутеру. Друга 0 означава редни број порта на том слоту.

access list 1 permit 192.168.1.0 0.0.0.255 // овом линијом бирам ко сме бити хатован, ip nat inside source list 1 interface GigabitEthernet 0/0 overload // ip nat inside source → NAT мења изворну адресу пакета који иде из LAN-а ка Интернету. list 1 користи ACL 1 да зна ко се мапира. interface GigabitEthernet 0/0 – користи IP адресу тог интерфејса као јавни IP за преводбење. overload омогућава PAT (Port Address Translation), што значи да више LAN хостова може да користи исту јавну IP адресу, али различите портове.

Imate NAT uređaj sa dva mrežna priključka (Interfejs A i Interfejs B). Takođe, imate dve lokalne mreže koje koriste privatne IP adrese:

- Mreža 1 (LAN1) koristi IP opseg 192.168.1.0/24.
- Mreža 2 (LAN2) koristi IP opseg 192.168.2.0/24.

Interfejs A je povezan sa LAN1, dok je Interfejs B povezan sa internetom i ima javnu IP adresu 203.0.113.1.

Zadatak je konfigurisati NAT uređaj kako bi omogućio uređajima iz obe lokalne mreže da pristupaju internetu preko javne IP adrese interfejsa B.

Navedite primer konfiguracije NAT uređaja kako bi se omogućila ovakva funkcionalnost.

◆ Postupak NAT konfiguracije sa dve LAN mreže

1 Podešavanje interfejsa ka LAN mrežama (inside)

- Ruteru kažeš: „Ovo je unutrašnja strana“
- Dodeljuješ IP adresu interfejsima povezanim na LAN1 i LAN2
- Označavaš ih kao `ip nat inside`

```
bash
```

```
interface GigabitEthernet0/0  
  ip address 192.168.1.1 255.255.255.0  
  ip nat inside
```

```
interface GigabitEthernet0/2  
  ip address 192.168.2.1 255.255.255.0  
  ip nat inside
```

2 Podešavanje interfejsa ka internetu (outside)

- Ruteru kažeš: „Ovo su vrata ka svetu“
- Dodeljuješ javnu IP adresu interfejsu koji ide na Internet
- Označavaš ga kao `ip nat outside`

```
bash
```

```
interface GigabitEthernet0/1
ip address 203.0.113.1 255.255.255.0
ip nat outside
```

3 Pravljenje spiska uređaja za NAT (ACL)

- Lista obuhvata obe privatne mreže
- Ruter sada zna ko se NATuje
- **Wildcard maska** obavezna (`0.0.0.255`)

```
bash
```

```
access-list 1 permit 192.168.1.0 0.0.0.255
access-list 1 permit 192.168.2.0 0.0.0.255
```

4 Glavna NAT komanda (Overload / PAT)

- Kažeš: „Sve što je na ACL1 neka izađe na WAN koristeći istu javnu IP, ali različite portove“

```
bash
```

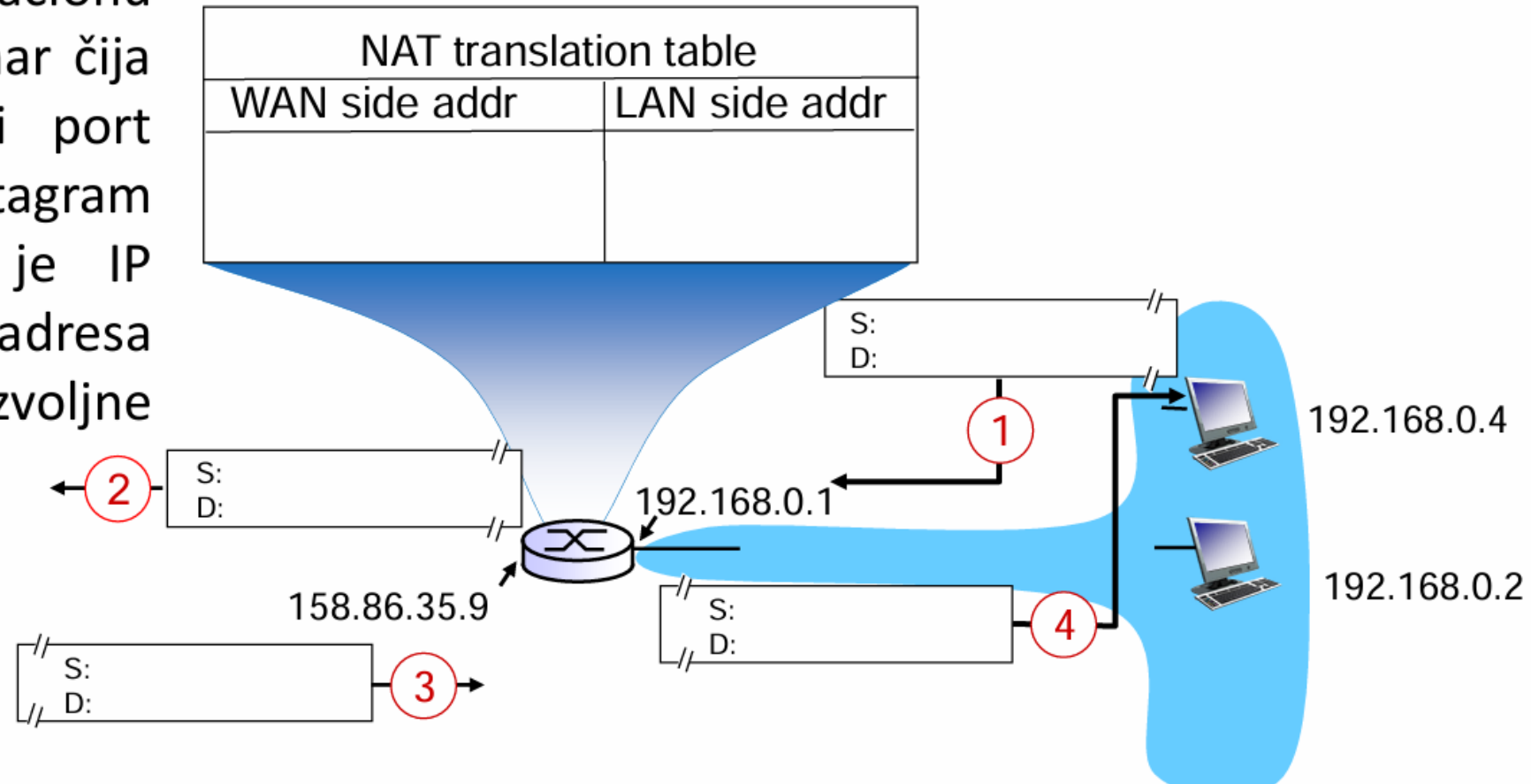
```
ip nat inside source list 1 interface GigabitEthernet0/1 overload
```

◆ Redosled koji uvek radi na ispitu

1. Inside interfejsi (LAN)
2. Outside interfejs (Internet)
3. ACL (spisak IP adresa za NAT)
4. NAT pravilo (inside → outside, overload)

📌 Ako držiš ovaj redosled, nikad ne možeš da pogrešiš na ispitu.

Na osnovu date slike za sva četiri naglašena koraka popuniti prazna polja za izvorišnu i odredišnu adresu (S i D) i popuniti NAT translacionu tabelu. Pretpostaviti da računar čija je IP adresa 192.168.0.4 i port adresa 3535 šalje datagram odredišnom računar čija je IP adresa 160.99.32.24 i port adresa 80. Port adrese rutera su proizvoljne iz opsega 1024-49151.



KORAK 1: Host šalje zahtev (Izlazak iz sobe)

Tvoj računar (`192.168.0.4`) pakuje paket. On zna da on šalje (**Source: 3535**) i zna gde šalje (**Destination: port 80** na serveru). Paket stiže do tvog kućnog rutera. **S: 192.168.0.4/3535**
D: 160.99.32.24/80

- **Stanje:** Paket je još uvek "lokalni". Da on sad direktno izađe na internet, server ne bi znao gde da vrati odgovor jer se adrese `192.168.x.x` ne koriste na internetu.

KORAK 2: NAT menja izvoriste (Promena identiteta)

Ruter uzima tvoj paket i kaže: "Skloni tu tvoju privatnu adresu, niko te neće prepoznati. Evo ti moja javna adresa (`158.86.35.9`) i ja ću ti dodeliti broj **12000** da znam da si to ti."

- **S (Izvorište):** Postaje ruterova javna IP i port koji je on izabrao. **S: 158.86.35.9/12000**
D: 160.99.32.24/80
- **D (Odredište):** Ostaje isto, jer paket i dalje ide ka istom serveru.
- **Ključna stvar:** Ruter u tom trenutku upisuje u svoju tabelu: "Sve što mi se vrati na port 12000, šaljem onom malom na 192.168.0.4:3535".

KORAK 3: Odgovor sa interneta (Paket stiže pred tvoja vrata)

Server je primio paket, obradio ga i šalje ga nazad. Server misli da priča sa ruterom.

- **S (Izvorište):** Sada je server (on šalje).
S: 160.99.32.24/80
D: 158.86.35.9/12000
- **D (Odredište):** Server šalje nazad na ruterovu javnu adresu i na onaj port **12000** koji je ruter "rezervisao".
- **Stanje:** Paket je stigao do spoljne strane tvog rutera (GigabitEthernet 0/0).

KORAK 4: Povratak hostu (Dostava na pravu adresu)

Ruter vidi paket na svom ulazu. Pogleda broj porta: **12000**. Brzo proverí svoju tabelu i kaže: "Aha! Port 12000 je onaj moj računar iz sobe."

- **S (Izvorište):** Ostaje server (tvoj računar treba da zna ko mu odgovara).
S: 160.99.32.24/80
D: 192.168.0.4/3535
- **D (Odredište):** Ruter vraća tvoju pravu adresu **192.168.0.4** i tvoj port **3535**.
- **Rezultat:** Tvoj računar dobija podatke i ti vidiš sajt, a da server nikada nije saznao tvoju pravu lokalnu adresu.

Koristeći ICMP komandu, odredite putanju (hopove) koju paket prelazi da bi stigao do web servera `www.example.com`.

U pitanju je alat **Traceroute** (ili `tracert` na Windowsu).

Kako ovo funkcioniše?

Ova komanda koristi ICMP "Echo Request" pakete, ali na lukav način: ona namerno postavlja nizak **TTL** (Time To Live).

1. Prvi paket ima **TTL=1**. Prvi ruter na koji naiđe ga odbaci i pošalje nazad ICMP poruku "Time Exceeded". Tako saznamo adresu prvog hopa.
2. Sledeći paket ima **TTL=2**, pa saznamo adresu drugog rutera, i tako dalje dok ne stignemo do cilja.

Rešenje (izvršavanje komande)

Ako si na **Windowsu**, komanda glasi: `tracert www.example.com`

Ako si na **Linuxu** ili **macOS-u**, komanda je:

```
Bash
```

```
tracert www.example.com
```

Kako bi izgledao rezultat (Primer):

Kada pokreneš komandu, dobićeš tabelu sličnu ovoj:

Hop	Vreme (ms)	IP adresa / Hostname	Napomena
1	<1 ms	192.168.1.1	Tvoj kućni ruter
2	10 ms	10.0.0.1	Ruter tvog internet provajdera (ISP)
3	25 ms	213.198.x.x	Glavni čvor u gradu/regionu
...	...	...	...
10	45 ms	93.184.216.34	www.example.com (Cilj)

Pretpostavimo da imate malu lokalnu mrežu sa tri računara: Računar A, Računar B i Računar C. Svaki od ovih računara je povezan na istu mrežu. Korišćenjem ARP zahteva, popunite ARP tabele za svaki od ovih računara tako da svaki računar zna MAC adrese ostalih računara u mreži. Pored toga, napišite komande koje će se koristiti za svaku fazu ovog procesa.

Faze popunjavanja ARP tabela

Pretpostavimo sledeće adrese:

- **Računar A:** IP 192.168.1.10 , MAC AA:AA
- **Računar B:** IP 192.168.1.20 , MAC BB:BB
- **Računar C:** IP 192.168.1.30 , MAC CC:CC

1. Dinamičko popunjavanje (Standardni metod)

Najlakši način da se popuni ARP tabela je korišćenje ping komande. Kada A pinguje B, oba računara će naučiti MAC adresu onog drugog.

Komande po fazama:

1. Na Računaru A:

- ping 192.168.1.20 (Popunjava tabelu za B)
- ping 192.168.1.30 (Popunjava tabelu za C)

2. Na Računaru B:

- ping 192.168.1.30 (Popunjava tabelu za C, dok A već zna od ranije)

3. Provera tabele (na bilo kom računaru):

- `arp -a` (Prikazuje sve naučene MAC adrese)
-

2. Statičko popunjavanje (Vaše rešenje: `arp -s`)

Ovaj metod se koristi kada želite da sprečite promene u tabeli (veća bezbednost). Morate ručno uneti podatke na svakom uređaju.

Komande na Računaru A:

- `arp -s 192.168.1.20 BB-BB-BB-BB-BB-BB` (Dodaje B)
- `arp -s 192.168.1.30 CC-CC-CC-CC-CC-CC` (Dodaje C)

Komande na Računaru B:

- `arp -s 192.168.1.10 AA-AA-AA-AA-AA-AA` (Dodaje A)
- `arp -s 192.168.1.30 CC-CC-CC-CC-CC-CC` (Dodaje C)

Komande na Računaru C:

- `arp -s 192.168.1.10 AA-AA-AA-AA-AA-AA` (Dodaje A)
- `arp -s 192.168.1.20 BB-BB-BB-BB-BB-BB` (Dodaje B)

Rezime komandi

Svrha	Komanda (Windows/Linux)
Prikaz ARP tabele	<code>arp -a</code>
Statičko dodavanje	<code>arp -s [IP adresa] [MAC adresa]</code>
Brisanje unosa	<code>arp -d [IP adresa]</code>



Извези у Табеле



Važna napomena: Za korišćenje `arp -s` komande u Windowsu, potrebno je pokrenuti Command Prompt kao **Administrator**. Na novijim verzijama Windowsa, preporučuje se korišćenje `netsh interface ipv4 add neighbors` jer `arp -s` nekada može vratiti grešku "Access denied".

Pretpostavimo da imate lokalnu mrežu sa tri računara: Računar A sa IP adresom 192.168.1.10, Računar B sa IP adresom 192.168.1.20 i Računar C sa IP adresom 192.168.1.30. Sada želite da izvršite ARP zahtev kako biste saznali MAC adresu Računara B. Napišite ARP zahtev za ovu situaciju.

Struktura ARP zahteva (Who has 192.168.1.20?)

Kada Računar A šalje ovaj zahtev, on se šalje kao **Broadcast** frejm (svima u mreži), jer A još uvek ne zna gde se fizički nalazi B.

- **Sender MAC:** AA:AA:AA:AA:AA:AA (Fizička adresa Računara A)
- **Sender IP:** 192.168.1.10
- **Target MAC:** 00:00:00:00:00:00 (U samom ARP telu je prazno/nule, jer je to podatak koji tražimo)
- **Target IP:** 192.168.1.20
- **Ethernet Destination Address:** FF:FF:FF:FF:FF:FF (Ovo je ključno – to je MAC adresa za "sve", koja tera sve uređaje u mreži da otvore paket).

Šta se dešava nakon slanja?

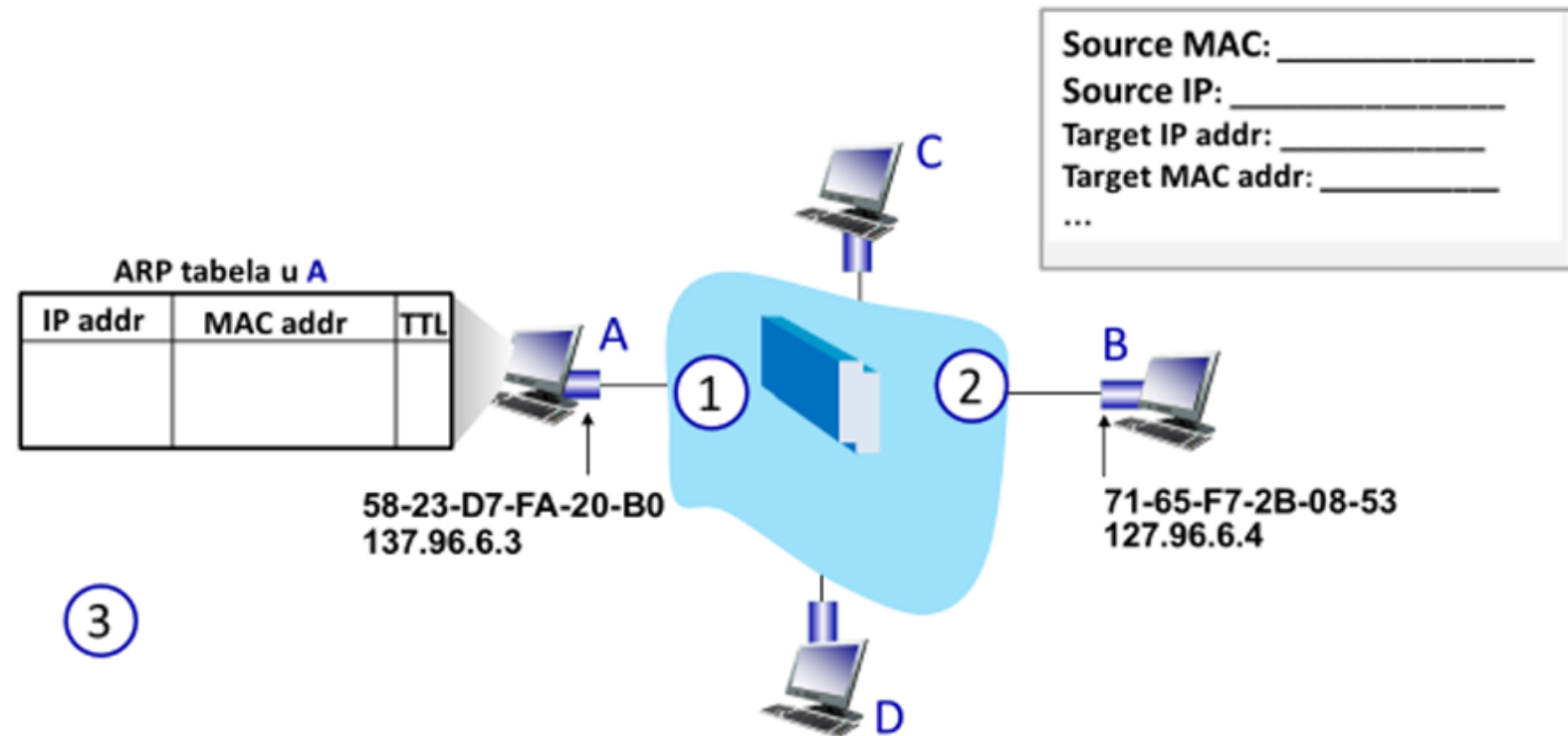
1. **Svi računari (B i C)** primaju ovaj paket jer je poslat na Broadcast adresu.
2. **Računar C** vidi da se traži IP 192.168.1.20. Pošto to nije njegova adresa, on tiho odbacuje paket.
3. **Računar B** prepoznaje svoju IP adresu i šalje **ARP Reply** direktno Računaru A (Unicast).

Kako bi izgledao ARP odgovor (Reply) od Računara B?

Polje	Vrednost
Sender MAC	BB:BB:BB:BB:BB:BB (MAC adresa Računara B)
Sender IP	192.168.1.20
Target MAC	AA:AA:AA:AA:AA:AA (MAC adresa Računara A)
Target IP	192.168.1.10

Nakon ovoga, Računar A popunjava svoju **ARP tabelu** (keš) i može direktno da komunicira sa Računarom B bez ponovnog slanja upita.

Korišćenjem date slike, ukratko opisati tri koraka koji su potrebni za ARP protokol ako računar A šalje paket računaru B. Za korake 1 i 2 popuniti odgovarajuća polja (Source MAC, Source IP address, Target MAC address) koja su potrebna za taj korak. Za korak 3 popuniti ARP tabelu računara A, ako vreme nakon kog će preslikavanje adresa biti zaboravljeno iznosi 100 sekundi.



Korak 1: ARP Request (Zahtev)

U ovom koraku Računar A šalje poruku svim uređajima u mreži (Broadcast) kako bi saznao fizičku adresu Računara B.

- **Source MAC:** 58-23-D7-FA-20-B0 (Adresa računara A)
- **Source IP:** 137.96.6.3 (Adresa računara A)
- **Target IP addr:** 127.96.6.4 (Adresa koju tražimo - Računar B)
- **Target MAC addr:** 00-00-00-00-00-00 (Nepoznata adresa, popunjava se nulama u ARP upitu)

Korak 2: ARP Reply (Odgovor)

Računar B prepoznaje svoju IP adresu u upitu i šalje direktan odgovor (Unicast) Računaru A, dostavljajući svoju MAC adresu.

- **Source MAC:** 71-65-F7-2B-08-53 (Adresa računara B)
- **Source IP:** 127.96.6.4 (Adresa računara B)
- **Target IP addr:** 137.96.6.3 (Adresa računara A koji čeka odgovor)
- **Target MAC addr:** 58-23-D7-FA-20-B0 (Adresa računara A)

Korak 3: ARP tabela u Računaru A

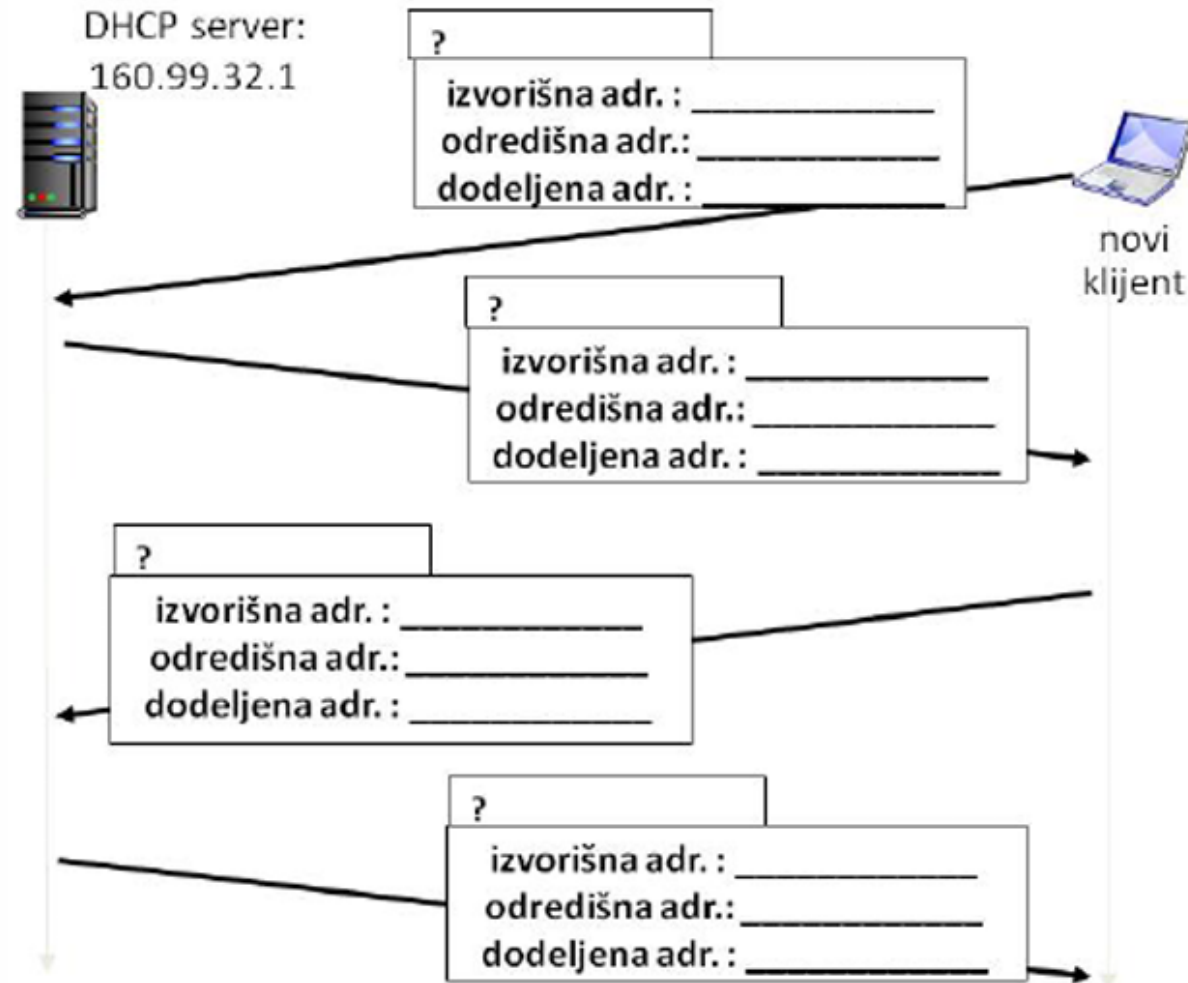
Nakon što dobije odgovor iz Koraka 2, Računar A popunjava svoju tabelu (keš) kako bi mogao da pošalje originalni paket podataka.

IP addr	MAC addr	TTL
127.96.6.4	71-65-F7-2B-08-53	100s
 Извези у Табеле 		

Kratak opis procesa:

- Slanje zahteva (1):** Računar A emituje ARP Request paket na mrežu.
- Slanje odgovora (2):** Računar B, identifikovan ciljnom IP adresom, odgovara slanjem svoje MAC adrese direktno pošiljaocu.
- Ažuriranje tabele (3):** Računar A upisuje dobijene podatke u lokalnu tabelu uz vremensko ograničenje od 100 sekundi.

Popunjavanjem praznih polja pored znaka “?” označiti sve faze u DHCP klijent-server komunikaciji. Popunjavanjem praznih polja navesti izvorišne, odredišne i dodeljene adrese za svaku fazu. Pretpostaviti da će server dodeliti adresu 160.99.3.32.



1. Faza: DHCP DISCOVER (Klijent traži server)

Klijent još nema svoju adresu, pa koristi nule, a šalje poruku svima u mreži (**broadcast**).

- izvorišna adr.: 0.0.0.0
- odredišna adr.: 255.255.255.255
- dodeljena adr.: 0.0.0.0 (još uvek ništa nije dodeljeno)

2. Faza: DHCP OFFER (Server nudi adresu)

Server odgovara i nudi konkretnu adresu.

- izvorišna adr.: 160.99.32.1 (IP servera)
- odredišna adr.: 255.255.255.255 (i dalje broadcast jer klijent još nije potvrdio adresu)
- dodeljena adr.: 160.99.3.32

3. Faza: DHCP REQUEST (Klijent prihvata ponudu)

Klijent kaže: "U redu, želim tu adresu koju si mi ponudio".

- **izvorišna adr.:** 0.0.0.0 (klijent i dalje zvanično ne koristi novu adresu dok ne dobije potvrdu)
- **odredišna adr.:** 255.255.255.255
- **dodeljena adr.:** 160.99.3.32

4. Faza: DHCP ACK (Server potvrđuje)

Finalna potvrda od strane servera.

- **izvorišna adr.:** 160.99.32.1
- **odredišna adr.:** 255.255.255.255
- **dodeljena adr.:** 160.99.3.32

Mala napomena: U modernim mrežama, neke implementacije šalju **Offer** i **Ack** kao *unicast* (direktno klijentu na njegovu MAC adresu), ali se u školskim i ispitnim zadacima najčešće piše **255.255.255.255** (broadcast) za sve četiri faze jer klijentov IP stack još uvek nije potpuno konfigurisan.